

基于多因素建模的 NIPT 时点选择与胎儿异常判定研究

摘要

本文针对NIPT（无创产前检测）技术中最佳检测时点选择与胎儿异常判定的问题，通过建立数学模型，对孕妇进行合理分组并确定最佳检测策略，以最小化潜在风险并提高检测准确性。我们综合利用了数理统计等方法，系统地解决了题目提出的四个问题。

针对问题一，本文首先对数据进行了探索与预处理，清洗后保留同一受试者检测次数大于三次的样本。初步散点图分析表明，从总体上看，Y染色体浓度与孕周和BMI之间未显示出明显的相关性；然而在个体层面，Y染色体浓度普遍随孕周增加呈上升趋势，且个体Y染色体浓度的增长率与BMI增长率之间存在高度正相关。这一发现说明，尽管在个体内部存在一致的变化规律，但由于个体间差异显著，在整体层面难以识别出明确关联。为量化该现象，本文对比了多种混合模型，最终选用广义可加混合模型（GAMM）。基于GAMM的方差分析结果显示：孕周平滑项、BMI平滑项以及个体随机效应项均达到极显著水平，表明三者对Y染色体浓度均具有显著影响。模型进一步揭示，Y染色体浓度随孕周呈非线性增长，与BMI之间存在复杂的非线性负相关关系，同时个体间差异具有高度统计学意义。

针对问题二，本研究基于临床已知的男胎孕妇BMI是影响胎儿Y染色体浓度达标时间的主要因素，通过对孕妇BMI进行科学分组并确定最佳NIPT检测时点，以最小化孕妇潜在风险。首先，我们依据Y染色体浓度达标情况将孕妇分为三组：始终达标组、中间达标组和从不达标组；接着采用 K-means聚类算法对孕妇平均BMI进行聚类分析，通过肘部法则确定最佳聚类数为5类；然后基于 Kaplan-Meier生存分析方法构建累积达标函数，并引入动态效用函数（结合早期与晚期风险函数），通过有界优化算法（`minimize_scalar`）在每组中寻找使效用函数最小化的最优检测时间点；最后通过可视化分析展示各BMI区间的达标曲线与效用变化，综合评价检测时点的合理性。结果表明，基于BMI分组的个性化检测时点推荐能够有效降低临床风险，为NIPT临床实践提供了科学依据。

针对问题三，本研究在问题二基础上进行了多因素拓展与建模策略优化，系统性融合年龄、身高、体重等因素，深入分析其对男胎Y染色体浓度达标时间的影响。在开始的区间划分的过程中，通过对比多参数聚类与天数/BMI聚类方法，最终选定具有明确临床解释性的天数/BMI聚类法，将孕妇划分为5个BMI区间，为后续分层建模奠定基础。研究基于Kaplan-Meier生存分析构建累积达标函数，并结合早期检测效用与晚期检测风险建立总效用最小化模型，通过有界优化算法（`minimize_scalar`）精准求解各组最优NIPT检测时点。在预测建模方面，创新性地采用随机森林回归与线性回归相结合的策略：对样本充足的组别使用随机森林以捕捉非线性关系与交互效应；对样本较少的组别采用线性回归确保稳定性。模型评估通过K折交叉验证（动态调整折数）与多指标体系（ R^2 、MAE）严格进行，并引入SHAP价值分解技术量化各因素边际贡献，显著增强模型透明度与临床可解释性。误差分析显示，模型在 $\pm 5\%$ 检测时间误差下仍保持良好稳定性，最终提出的基于BMI分层的检测推荐策略可信度高。

针对问题四，本文首先基于GC含量、X染色体Z值等七项指标对原始数据进行了清洗。在此基础上，构建了一种针对女胎染色体非整倍体异常（T21、T18、T13）的智能判定模型。具体流程包括：对目标变量进行one-hot编码，并采用Z-score方法对数据进行标准化处理；利用点二列相关系数评估各特征的重要性，并依据top-k规则筛选关键指标；针对样本不均衡问题，引入SMOTE-ENN混合采样算法以提升数据质量。最终，构建以随机森林为核心的分类模型，结合交叉验证与超参数优化，使模型准确率超过92.5%，召回率达到95%以上，表现出优秀的预测性能与临床适用性。因此，建

议孕妇仅需按照问题三推荐的时间节点进行血液检测并提供年龄等相关信息，即可借助该模型实现高效、可靠的女胎染色体异常筛查。

关键词：广义可加混合模型，K-means聚类算法，有界优化算法（`minimize_scalar`），Kaplan-Meier生存分析，SHAP值增强模型，SMOTEENN混合采样算法，随机森林

一 问题重述

1.1 问题背景

NIPT (Non-invasive Prenatal Testing, 无创产前检测) 作为一项革命性的产前筛查技术, 通过采集孕妇外周血中的胎儿游离DNA进行测序分析, 能够有效评估胎儿常见染色体非整倍体异常的风险[1]。该技术具有无创、安全、准确性高等特点, 已成为临床产前筛查的重要手段。

在实际临床应用中, NIPT检测的准确性受到多种因素影响, 其中胎儿游离DNA浓度 (特别是Y染色体浓度对于男胎) 是关键因素之一。临床实践表明, 胎儿Y染色体浓度与孕妇孕周数及身体质量指数 (BMI) 存在显著相关性。同时, 检测时机的选择对于尽早发现胎儿异常、降低临床风险至关重要: 早期发现 (12周以内) 风险较低, 中期发现 (13-27周) 风险较高, 而晚期发现 (28周以后) 风险极高。

目前临床通常根据孕妇BMI值进行简单分组并确定统一的检测时点, 但这种方法未能充分考虑孕妇年龄、体重等个体差异, 可能导致部分孕妇错过最佳检测时机, 增加临床风险。因此, 需要建立更加科学、个性化的 NIPT时点选择模型, 为不同特征的孕妇群体制定最优检测策略。

1.2 问题要求

附件提供了某地区 (大多为高BMI) 孕妇的NIPT检测数据, 包括孕妇年龄、BMI、孕周数、胎儿染色体浓度、Z值、GC含量、读段数等相关指标。现需要根据这些数据建立数学模型, 解决以下问题:

问题一: 基于附件中的数据, 分析胎儿Y染色体浓度与孕妇孕周数、BMI等指标的相关特性, 建立合适的数学模型描述它们之间的关系, 并对模型的显著性进行统计检验。

问题二: 临床证明男胎孕妇的BMI是影响胎儿Y染色体浓度达标时间 (浓度 $\geq 4\%$ 的最早时间) 的主要因素。请对男胎孕妇的BMI进行合理分组, 确定每组的BMI区间和最佳NIPT检测时点, 使得孕妇的潜在风险最小, 并分析检测误差对结果的影响。

问题三: 综合考虑体重、年龄等多种因素对男胎Y染色体浓度达标时间的影响, 同时考虑检测误差和胎儿Y染色体浓度达标比例, 根据男胎孕妇的BMI进行合理分组, 确定每组的最佳NIPT检测时点, 使孕妇潜在风险最小, 并分析检测误差对结果的影响。

问题四: 针对女胎异常的判定问题, 以女胎孕妇的21号、18号和13号染色体非整倍体为判定结果, 综合考虑X染色体及上述染色体的Z值、GC含量、读段数及相关比例、BMI等因素, 建立女胎异常的判定模型和方法。

二 问题分析

2.1 问题一的分析

问题一要求分析Y染色体浓度与孕周数、BMI之间的相关关系, 由于数据来源于同一孕妇的多次检测, 存在明显的组内相关结构, 传统回归方法无法处理该类数据且会导致估计偏差。同时, 生物学背景

表明孕周与浓度间可能存在非线性增长模式，而BMI亦可能通过稀释效应产生非线性影响。因此，需采用能同时兼容个体随机效应与变量非线性关系的混合建模策略。我们首先通过个体线性回归初步探索趋势，发现普遍正增长但存在个体差异，且BMI与浓度增长率线性相关不明显，暗示简单线性假设不足。进而，分别建立LMM、NLME和 GAMM三类混合模型进行系统比较，最终依据GAMM显著的非线性识别能力和更高的拟合优度，选择其作为揭示变量间复杂相关特性的核心模型。

2.2 问题二的分析

针对问题二，其要求根据男胎孕妇的BMI进行合理分组并确定最佳检测时点。我们根据Y染色体浓度是否达标（ $\geq 4\%$ ）将孕妇分为三类，反映不同孕妇的Y染色体浓度变化规律。然后采用K-means聚类算法对孕妇平均BMI 进行客观分组，避免主观划分的随意性。通过肘部法则确定最佳聚类数为5，确保了分组的科学性和合理性。接着对于每个BMI分组，计算组内孕妇的Y染色体浓度达标时间（对于中间达标组采用插值法预测），以中位数作为该组的最佳检测时点，减少异常值影响。最后考虑检测过程中可能存在的测量误差和模型误差，通过误差传播模型评估这些误差对达标时间预测的影响程度，确保推荐时点的可靠性。

2.3 问题三的分析

问题三要求在问题二基础上引入年龄、身高、体重等多因素，并考虑检测误差与达标比例，以更全面确定男胎孕妇的最佳NIPT检测时点。难点在于多因素之间存在复杂交互效应，且数据中体重变化显著，直接使用均值会丢失动态信息。因此，本研究首先对BMI进行Z标准化，并采用多参数聚类与天数-BMI聚类两种方法进行对比。结果表明，天数-BMI聚类在聚类质量、结构稳定性和临床可解释性上均优于多参数聚类，故以其作为分组依据。随后构建多阶段建模框架，整合生存分析、效用-风险函数优化和机器学习建模（随机森林/线性回归），系统分析各因素影响，并通过蒙特卡洛模拟验证模型鲁棒性。最终实现科学分组与个性化时点推荐，为临床提供可靠决策支持。

2.4 问题四的分析

问题四要求建立女胎染色体非整倍体异常判定模型，需综合考量X、21、18、13号染色体的Z值、GC含量、读段数及相关比例、BMI等多种因素。核心挑战在于数据存在严重类别不平衡（异常样本仅占10.7%），且特征维度高、冗余性强。为此，本研究首先进行严格数据清洗与质控，剔除异常样本，并使用点二列相关系数筛选出15 个最具判别力的特征。为克服类别不平衡，引入SMOTEENN混合采样算法，有效提升少数类样本代表性。在模型选择上，系统比较随机森林、XGBoost和SVM三类主流算法，最终随机森林以其最优的综合性能（准确率、召回率、AUC等）胜出。通过超参数调优与交叉验证进一步优化模型，并结合SHAP值分析增强可解释性，确保模型结果临床可信、可用。

三 模型假设

1.假设附件所提供的孕妇NIPT检测数据真实、准确，且数据样本足以反映胎儿染色体浓度与孕妇孕周、BMI等指标间的统计规律。

- 2.假设题目中给出的Y染色体浓度4% 的临界值是可靠且普适的。
- 3.假设具有相似特征（如处于同一BMI区间）的孕妇群体，其胎儿Y染色体浓度的增长规律和达标时间具有相似的统计特征。
- 4.假设题目所提供的特征（包括但不限于X、21、18、13号染色体的Z值、GC含量、读段数比例及孕妇BMI等）包含了足以有效判别女胎染色体是否异常的信息。

四 符号说明

符号	说明
$k_i^{(c)}$	孕妇代码为 i 的个体Y染色体浓度与孕周线性回归方程斜率
$k_i^{(b)}$	孕妇代码为 i 的个体BMI指数与孕周线性回归方程斜率
Y_{ij}	孕妇代码为 i 的个体第 j 次检测得到的Y染色体浓度
G_{ij}	孕妇代码为 i 的个体第 j 次检测的检测孕周（天为单位）
BMI_i	表示第 i 个孕妇的身体质量指数（常数对于每个孕妇）

五 问题一模型建立求解

5.1 模型建立思路

问题一要求分析胎儿Y染色体浓度与孕妇孕周数和BMI等指标的相关特性。我们先观察通过分析数据，对每个人线性回归，发现了绝大多数人 y 和时间关系与 y 和增长率和BMI增长率关系，以此我们判定，所有曲线都遵循一个共同的、内在的生物学规律，但由于个体差异，曲线的高度和形状会各不相同。为精确刻画个体增长规律并避免群体平均带来的巨大偏差，我们决定采用基于个体时间序列的混合效应方法。核心思路是：首先对数据进行筛选，保证每个个体的数据点数量足以支持回归分析；然后通过使用三种线性与非线性混合效应模型，比较得出最优是GAMM，并且根据这个得出得出胎儿Y染色体浓度与孕妇孕周数和BMI的相关特性。

5.2 数据预处理

为探究Y染色体浓度与孕妇孕周数和BMI的关系，我们分别以检测孕周（天数）和BMI为x轴，Y染色体浓度为y轴，绘制了散点图（其中相同颜色的点代表同一个人），如下：

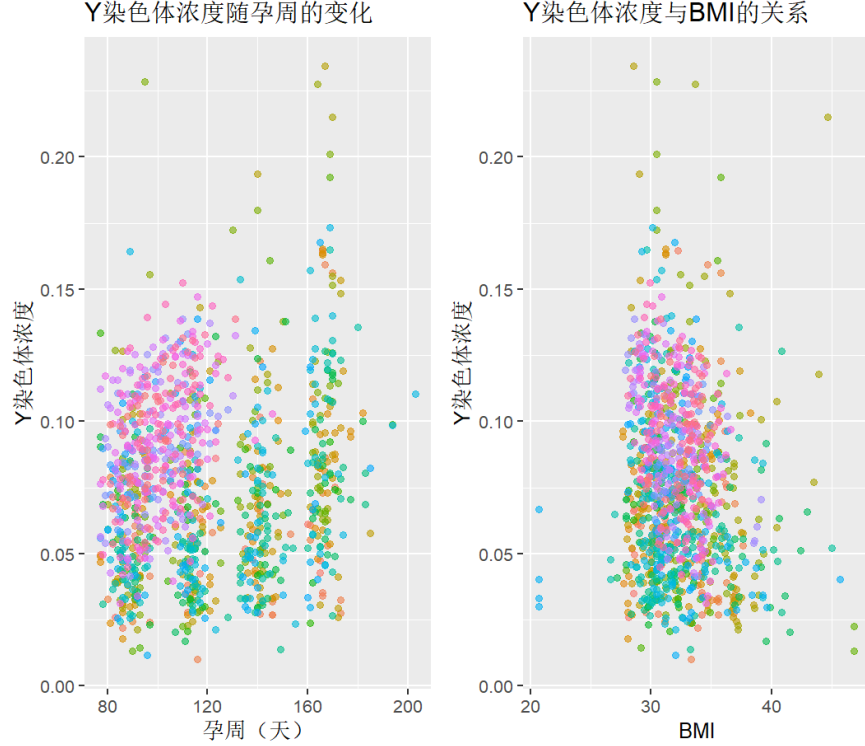


图 1: y染色体浓度散点图

分析左图（Y染色体浓度随孕周变化）可见对于一个个体而言，随着孕周从80天增加到200天，Y染色体浓度整体呈上升趋势，散点分布呈现明显的“向右上方扩散”特征。特别是在孕周大于160天后，高浓度点（ $Y \geq 0.15$ ）的出现频率显著增加，这表明Y染色体浓度与孕周之间存在正相关关系

分析右图（Y染色体浓度与BMI关系）显示，随着BMI值在20-40范围内变化，Y染色体浓度的散点分布未呈现明显的单向变化趋势。然而，结合后续模型得到的负系数估计值（ $c = -0.0042$ ），提示Y染色体浓度可能随BMI增加而轻微下降，但这种趋势在散点图中不够明显，可能需要通过模型来进一步验证。

我们打算先建立回归模型，故对于原数据处理，仅保留检测同一孕妇代码检测次数大于3次的数据，以保证建立的回归模型的可靠性，最终符合条件的人是242，在总人数中占90.6%。

5.3 线性回归模型

分析胎儿 Y 染色体浓度与孕妇的孕周数和 BMI 等指标的相关特性，本研究为每位孕妇建立了双变量时间序列回归模型。从预处理后的有效数据中，筛选出每位孕妇多次检测的记录，分别建立以下两个一元线性回归模型：

1. Y染色体浓度-时间关系模型：

以检测时间（ t_{ij} ）为自变量，对应的Y染色体浓度（ c_{ij} ）为因变量，建立回归模型：

$$c_{ij} = k_i^{(c)} * t_{ij} + b_i^{(c)} + \epsilon_{ij}^{(c)} \quad (1)$$

其中： $k_i^{(c)}$ 表示第 i 位孕妇的胎儿Y染色体浓度的日增长率，这是我们关注的核心参数； $b_i^{(c)}$ 为截距

项; $\epsilon_{ij}^{(c)}$ 为随机误差项。

2. BMI-时间关系模型:

以检测时间 (t_{ij}) 为自变量, 对应的Y染色体浓度 (c_{ij}) 为因变量, 建立回归模型:

$$c_{ij} = k_i^{(b)} * t_{ij} + b_i^{(b)} + \epsilon_{ij}^{(b)} \quad (2)$$

其中: $k_i^{(b)}$ 表示第 i 位孕妇的胎儿Y染色体浓度的日增长率, 这是我们关注的核心参数; $b_i^{(b)}$ 为截距项; $\epsilon_{ij}^{(b)}$ 为随机误差项。

为评估拟合优度, 我们采用最小二乘法进行参数估计, 并计算确定系数 R_i^2

5.4 Y染色体浓度与时间关系的个体分析

对于基于Y染色体浓度与时间关系建立的所有个体回归模型, 我们首先对其关键参数 (斜率, 截距, 确定系数) 进行了统计分析, 结果如下表所示:

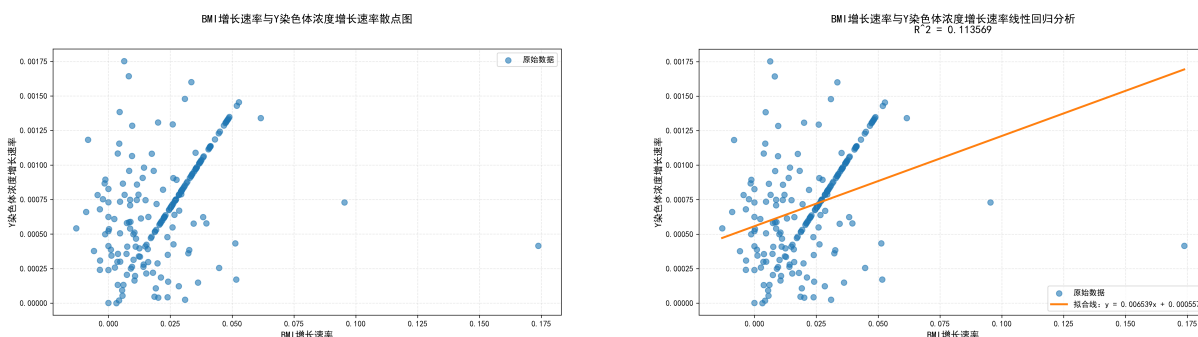
统计量	斜率 k_i (%/天)	截距	确定系数 R_i^2
平均值	0.000800	-0.005467	0.861443
标准差	0.00316	0.038959	0.123487
最小值	0.001752	0.088208	0.999747
最大值	0.000205	-0.151320	0.505933

从表中可知, 个体的Y染色体浓度均随孕天增加呈现明显的上升趋势,绝大部分模型的拟合优度较高 (平均 $R^2 = 0.824$), 这表明线性模型能够很好地描述浓度随时间变化的规律。

5.5 BMI增长率与Y染色体浓度增长率的全局相关性分析

为探究BMI是否导致了上述增长速率的差异, 我们计算了每位孕妇的BMI增长率个体Y染色体浓度增长率 k_i 的相关性。

我们绘制了标准化BMI增长速率与斜率 k_i 在相应编号下的散点图与两者在不同维度下绘制的图像, 如下



为探究BMI是否影响Y染色体浓度的增长速率, 我们分析了每位孕妇的BMI增长率与Y染色体浓度增长率的相关性。散点图与线性拟合显示 R^2 值较小(0.113569), 拟合效果不理想, 因此需要考虑其他模型形式。

5.6 新模型选择思路

为建立能够准确描述Y染色体浓度与孕周数、BMI之间关系的数学模型，我们基于生物学机制与数据结构特征两个方面进行了综合论证，以此确定模型选择的基本方向。

1.从生物学机理出发：

题目背景与临床知识表明，胎儿游离DNA浓度（此处体现为Y染色体浓度）随孕周增加而增长。这是因为随着胎儿发育，其细胞凋亡释放到母体血液中的DNA总量增加。这种增长通常不是线性的，在早期增长缓慢，中后期可能加速，符合指数增长或逻辑增长的生物学规律。因此，我们优先考虑非线性模型来捕捉孕周与浓度的关系。

而对于BMI与Y染色体的浓度关系，较高的BMI可能意味着更多的血液量和脂肪组织，理论上会对胎儿游离DNA浓度产生一定的“稀释”效应。因此，我们假设BMI对Y染色体浓度存在一个负向影响将这一项加入模型，符合我们的生理学直觉。

2.从数据结构出发：

本研究的核心特征在于数据结构：每个孕妇有多次重复测量记录，导致数据点之间存在明显的组内相关性。若采用普通最小二乘回归，将严重违背模型独立性假设，导致参数估计偏差和标准误低估。

混合效应模型则能有效处理此类层次结构数据：通过引入刻画个体差异的随机效应，模型能够区分组内与组间变异，从而提升固定效应估计的稳健性与准确性。因此，从数据结构的角度看，混合效应模型是更适合本研究的建模框架。

5.7 混合模型建立

基于前述生物学机理与数据结构特征，本研究采用混合效应模型（Mixed-Effects Model）作为核心建模框架。该模型能有效刻画个体随机变异，区分组内与组间差异，从而提升固定效应估计的稳健性。我们分别构建以下三类模型进行系统比较：线性混合模型（LMM）、非线性混合模型（NLME）与广义可加混合模型（GAMM），详细性能对比见表1：

综合对比分析如下

5.7.1 模型复杂度与拟合优度

线性混合模型（LMM）结构简单、解释性强，但强制的线性假设可能导致对潜在非线性关系的拟合不足。非线性混合模型（NLME）通过引入指数项更好地捕捉Y浓度随孕周的增长趋势，其AIC值（-5026.9）略低于LMM，显示在复杂度调整后拟合更优，但该模型未包含BMI变量，限制了其整体解释力。广义可加混合模型（GAMM）则兼具极佳的灵活性与拟合能力，同时刻画BMI和孕周的非线性效应，其调整 R^2 达0.753，可解释81%的偏差，显著优于前两类模型。

5.7.2 孕周效应

三类模型均证实孕周对Y染色体浓度存在极显著影响（ $p \leq 0.001$ ）。特别地，GAMM中孕周平滑项的有效自由度（ $edf = 4.682$ ）显著大于1，进一步表明其与浓度之间呈显著非线性关系，也从数据层面支持了NLME中采用指数增长形式的合理性。

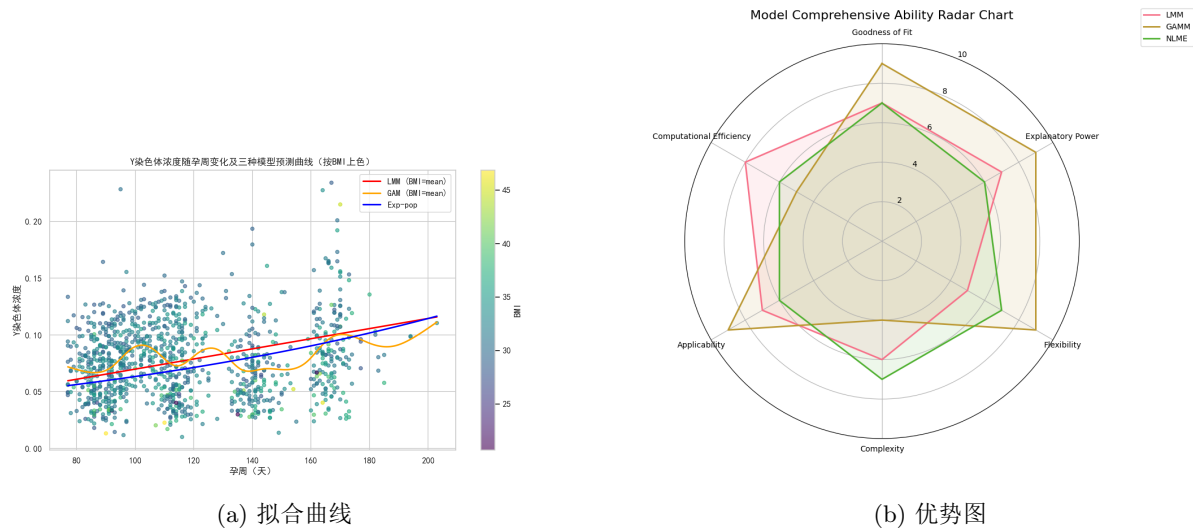
表 1: 三种混合建模对比分析表

特征指标	线性混合模型 (LMM)	非线性混合模型 (NLME) - 广义可加混合模型 (GAMM) 指数形式	
模型形式	$Y \sim \text{BMI} + \text{G_days} + (1 \text{Subject})$	$Y \sim a * \exp(b * \text{G_days})$ $Y \sim s(\text{BMI}) + s(\text{G_days}) + s(\text{Subject}, \text{bs}="re")$	
模型复杂度/灵活性	低 (严格线性假设)	中 (预设非线性形式)	高 (数据驱动非线性形式)
固定效应 - BMI	-0.001376($p \leq 0.01$)	-	平滑项显著($p = 0.00764$)
固定效应 - 孕周	0.0004466($p \leq 0.001$)	0.0088($p \leq 0.001$)	平滑项高度显著($p \leq 2e-16$)
随机效应 - 个体方差	0.0008285(Std.Dev: 0.02878)	0.0001279(Std.Dev: 0.01131) (仅a)	包含在 $s(\text{Subject})$ 中 (高度显著, $p \leq 2e-16$)
残差方差	0.0002930(Std.Dev: 0.01712)	0.0002807(Std.Dev: 0.01675)	Scale est. = 0.00027726
拟合优度 - AIC	-5025.5 (REML)	-5026.912	REML = -2536.1 (与AIC/BIC不可直接比较)
拟合优度 - R^2 /解释度	-	-	Adj. $R^2 = 0.753$, Deviance Explained = 81%
核心优势	简单直观，解释性强	捕捉指数增长趋势，生物学解释性好	无需预设形式，自动捕捉复杂非线性关系，拟合优度最高
主要局限	可能忽略重要非线性关系，拟合度相对较低	函数形式预设可能带来偏差，未包含BMI	结果为平滑曲线，缺乏参数化解析表达式

5.7.3 BMI效应

LMM检测到BMI存在显著的线性负效应 ($p \leq 0.01$), 而GAMM进一步揭示出BMI的影响具有显著非线性特征 (edf = 5.416, $p = 0.00764$)。这表明LMM中简单的线性设定可能不足以充分捕捉BMI在整个取值范围内对Y浓度的复杂影响模式。

此外我们分别绘制了这三者对于原散点图的拟合曲线并绘制了这三者的各自的优劣图, 如下:



综合模型对比指标与拟合图示，GAMM不仅具有最优的统计拟合性能，也能更充分捕捉变量间复杂的非线性关系，因此被选作后续建模的基础框架。

5.8 GAMM模型的建立

为充分探索胎儿Y染色体浓度与孕妇孕周数和BMI之间可能存在的复杂非线性关系，避免预设模型形式的偏差，我们建立了广义可加混合模型（Generalized Additive Mixed Model, GAMM）。该模型不强制要求变量间满足特定的参数关系（如线性或指数关系），而是通过平滑函数灵活地拟合数据中潜在的非线性模式，同时通过随机效应项处理个体重复测量数据带来的相关性。

建立GAMM模型的具体形式如下：

$$Y_{ij} = \beta_0 + s_1(BMI_i) + s_2(G_{ij}) + s_3(Subject_i) + \epsilon_{ij} \quad (3)$$

其中： β_0 为模型截距项； $s_1(BMI_i)$ 为BMI的平滑函数，用于捕捉BMI对Y染色体浓度的非线性影响； $s_2(G_{ij})$ 为孕周数的平滑函数，用于捕捉孕周对Y染色体浓度的非线性影响； $s_3(Subject_i)$ 为针对孕妇个体的随机效应平滑项，用于捕捉个体特异性差异； ϵ_{ij} 为残差项，假定服从正态分布： $\epsilon_{ij} \sim N(0, \sigma_\epsilon^2)$

模型采用薄板回归样条（Thin Plate Regression Splines）作为平滑函数的基础，使用REML（Restricted Maximum Likelihood）方法进行参数估计，以保证方差分量估计的无偏性。

5.9 显著性检验

对建立的GAMM模型进行全面的显著性检验，结果如下：

5.9.1 平滑项显著性检验

基于方差分析原理的显著性检验显示：

1. BMI平滑项 ($s_1(BMI_i)$)：有效自由度 (edf) = 5.416，参考自由度 = 6.263，F值 = 3.02，p值 = 0.00764**。结果表明BMI对Y染色体浓度的影响具有高度显著的非线性特征 ($p \leq 0.01$)，而edf ≥ 1 表明BMI与Y浓度间存在复杂的非线性关系，而非简单的线性关系

2. 孕周平滑项 ($s_2(G_{ij})$)：有效自由度 (edf) = 4.682，参考自由度 = 5.711，F值 = 77.86，p值 $\leq 2e-16$ *** 结果表明孕周对Y染色体浓度的影响极显著 ($p \leq 0.001$)，edf = 4.682 ≥ 1 证实了孕周与Y浓度间存在显著的非线性关系

3. 个体随机效应项 ($s_3(Subject_i)$)：有效自由度 (edf) = 240.736，参考自由度 = 266.000，F值 = 10.58，p值 $\leq 2e-16$ *** 结果表明不同孕妇个体间存在极显著的差异性 ($p \leq 0.001$)，证明使用混合效应模型处理重复测量数据的必要性。

5.9.2 模型拟合优度检验

1. 调整后 $R^2 = 0.753$ ，表明模型能够解释Y染色体浓度75.3%的变异
2. 偏差解释率 = 81%，表明模型对数据偏差的解释程度较高
3. 尺度参数估计值 = 0.00027726，反映了模型的残差方差
4. REML准则值 = -2536.1，用于模型比较（数值越小表示拟合越好）

GAMM模型的结果为后续参数化模型（非线性混合效应模型）的选择提供了坚实的数据驱动依据，证实了孕周和BMI与Y染色体浓度间均存在显著的非线性关系。

六 问题二模型建立求解

6.1 数据预处理

本问题旨在探究孕妇BMI对胎儿Y染色体浓度最早达标时间的影响。考虑到同一孕妇的BMI随时间变化幅度较小（个体内变异），且个体间差异远大于时间变化量，为提高分析的准确性和可靠性，我们采用每位孕妇多次BMI 测量的平均值作为其代表值进行后续统计分析。

为合理区分样本并便于计算NIPT最佳时点，我们依据Y染色体浓度达标情况对孕妇进行了科学分组：

- **第一组（始终达标组）**：从首次检测开始Y染色体浓度即达到或超过4%的样本。该组数据保存在‘bmi_Y_always_can_test_result.xlsx’中，包含孕妇代码、平均BMI和最早达标天数（即首次检测时间）。
- **第二组（中间达标组）**：初始检测未达标但后续检测达标的样本。该组数据保存在‘bmi_Y_middle_result.xlsx’中，包含孕妇代码、平均BMI和预测达标天数（通过插值法计算获得，详见附录代码Line XX）。
- **第三组（从不达标组）**：所有检测中Y染色体浓度均未达标的样本。该组数据保存在‘bmi_Y_cannot_test_result.xlsx’中，包含孕妇代码、平均BMI和最晚达标天数（即末次检测时间）。

为确保数据质量，若某样本曾达标但后续检测又低于4%，则视为无效样本并予以剔除。经上述处理，最终获得有效样本总量为230例。其中，第一组186例（占比80.87%），第二组37例（占比16.09%），第三组7例（占比3.04%）。这一分组策略确保了后续分析的可靠性和代表性。

6.2 BMI划分与聚类分析

为探究BMI对Y染色体浓度达标时间的影响模式，我们采用聚类算法对孕妇平均BMI进行科学细分。首先，通过肘部法则确定最佳聚类数量（图4）。结果显示，当聚类数超过5时，距离平方和下降速率显著降低，故选择聚类数 $k=5$ 作为最优解。

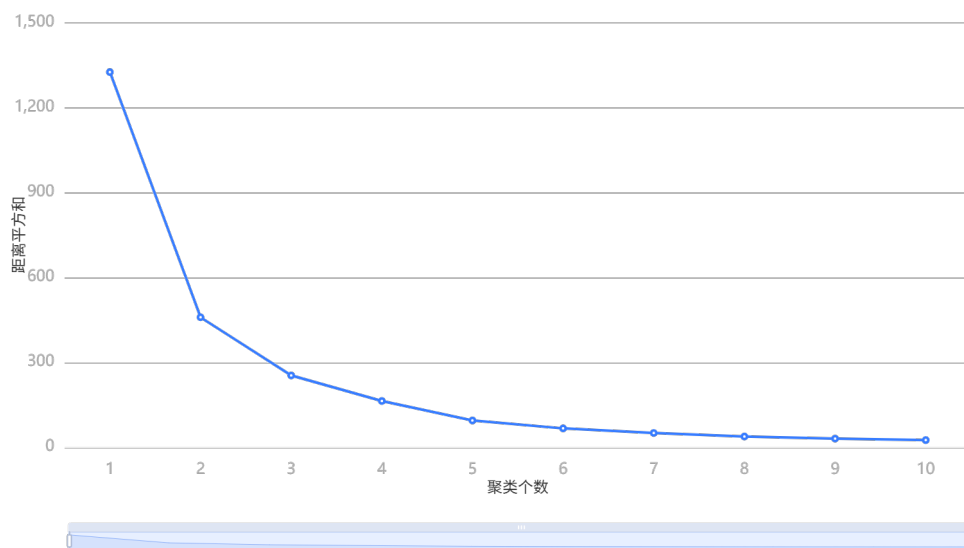


图 4: 肘部法则图

基于此，我们对孕妇平均BMI与Y染色体浓度进行了K-means聚类分析，结果如图5，图6所示。聚类结果清晰地展示了不同BMI区间与Y染色体浓度的关联模式。

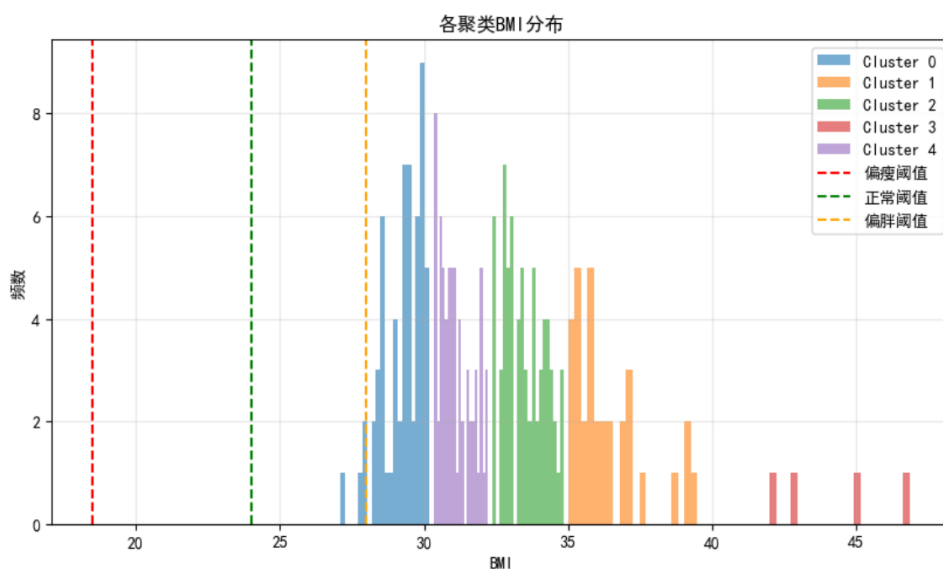


图 5: 聚类结果柱状图图

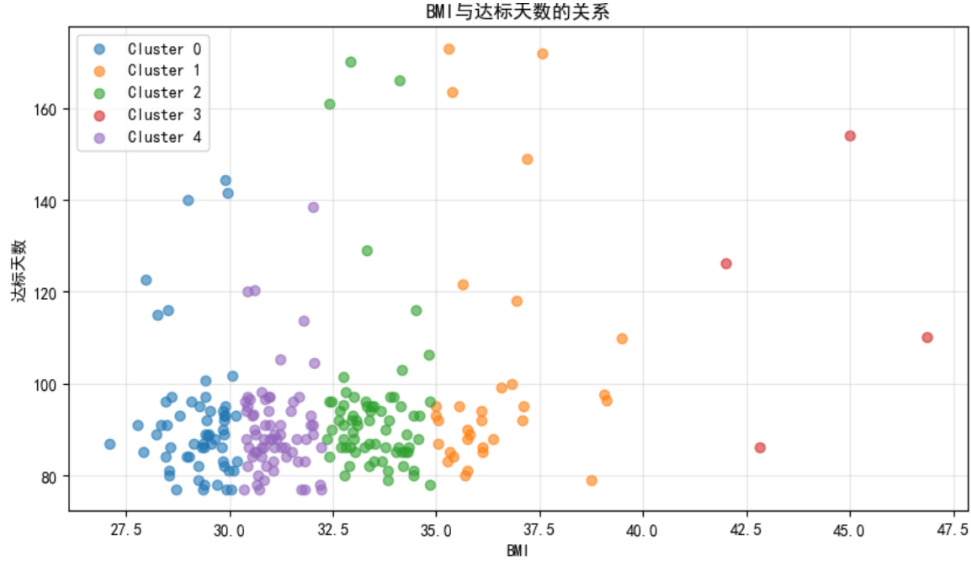


图 6: 聚类结果散点图

6.3 聚类分类评价

为进一步评估聚类质量，我们计算了三个内部评价指标：

1.轮廓系数：0.551（轮廓系数取值范围[-1,1]，值越大表示同类样本越相近，不同类样本越远离，一般不可能大于0.7）

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (4)$$

$a(i)$: 样本 i 到同一簇内所有其他样本的平均距离。（凝聚度） $b(i)$: 样本 i 到其他某个簇的所有样本的平均距离。计算样本 i 与所有非本身所在簇的的平均距离，然后取其中的最小值。（分离度）

2.DBI指数：0.517（值越小表示类内紧密度越高，类间分离度越好， ≤ 0.7 可接受）

$$DBI = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} R_{ij} \quad (5)$$

K : 簇的个数。 R_{ij} : 簇 i 和簇 j 之间的相似度 s_i : 簇 i 的簇内散度，即簇内所有样本到其质心 c_i 的平均距离。 d_{ij} : 簇 i 的质心 c_i 与簇 j 的质心 c_j 之间的距离（通常使用欧氏距离）。

3.CH指数：701.814（值越大表示聚类效果越优）

$$CH = \frac{SS_B / (K - 1)}{SS_W / (N - K)} \quad (6)$$

N : 数据集中的总样本数。 K : 簇的个数。 SS_B : 簇间离散度（Between-Cluster Sum of Squares）。所有簇的质心 c_k 与全局质心 c 的距离平方和，并按簇大小加权。 SS_W : 簇内离散度（Within-Cluster Sum of Squares）。所有样本点到其所属簇的质心的距离平方和。

上述指标综合表明，本次聚类效果良好，各类别内部样本相似度高，类别之间差异明显，分类结果可靠有效，为后续分析奠定了坚实基础。

6.4 NIPT时点计算

6.4.1 方法原理与模型构建

为确定各BMI分组的最佳NIPT检测时点，本研究构建了一个基于风险-效用权衡的优化模型。该模型综合考虑了早期检测与晚期检测的双重风险，通过数学优化方法寻找使总体风险最小化的检测时间点。

1. 风险函数定义：

- 早期检测风险(R_{early})：检测时间过早可能导致Y染色体浓度尚未达标，造成假阴性结果。我们采用指数衰减函数模拟这一风险：

$$R_{early}(t) = 2.0 \cdot e^{-t/50} \quad (7)$$

该函数从初始值2.0开始，随孕期增加呈指数衰减，反映早期预测的高风险性。

- 晚期检测风险(R_{late})：检测时间过晚则可能错过最佳干预时机。我们采用分段函数描述这一风险：

$$R_{late}(t) = \begin{cases} 0.1 & t < 84 \\ 0.1 + 0.9 \cdot \left(\frac{t-84}{189-84}\right)^2 & 84 \leq t \leq 189 \\ 1.0 & t > 189 \end{cases} \quad (8)$$

该函数在84天前保持低风险(0.1)，84-189天间以二次函数形式增长，189天后达到最高风险(1.0)。

2. 效用函数构建：

基于累积达标概率 $F(t)$ （通过Kaplan-Meier估计器计算）和上述风险函数，我们构建综合效用函数：

$$U(t) = (1 - F(t)) \cdot R_{early}(t) + F(t) \cdot R_{late}(t) \quad (9)$$

其中 $F(t)$ 表示在时间 t 之前Y染色体浓度达到4%的累积概率。

3. 优化目标：

通过最小化效用函数 $U(t)$ 确定最佳检测时点：

$$t^* = \arg \min_{t \in [0, T_{max}]} U(t) \quad (10)$$

其中 T_{max} 为观察期上限。

6.4.2 计算流程与实施

本研究采用以下步骤计算各BMI分组的最佳NIPT时点：

- 数据准备与分组：**基于聚类分析得到的5个BMI区间，将样本分为相应组别。
- 生存分析：**对每个BMI分组，应用Kaplan-Meier方法估计累积达标函数 $F(t)$ ，处理右删失数据（始终达标组和从不达标组）。
- 效用函数计算：**在时间域 $[0, T_{max}]$ 上离散采样，计算各时间点的效用值 $U(t)$ 。
- 优化求解：**采用有界优化算法（Bounded方法）寻找使 $U(t)$ 最小化的时间点 t^* 。
- 误差分析：**通过蒙特卡洛模拟（100次重复）评估检测误差对最优时点稳定性的影响。

6.4.3 结果与分析

表2展示了各BMI分组的最佳NIPT时点及相应指标：

表 2: 各BMI分组最佳NIPT时点计算结果

BMI分组	最优时点（天）	最小效用值	风险水平	样本量
<30.26	89.5	0.1299	7.696559	59
30.26-32.30	90.8	0.1360	7.3511	68
32.30-34.92	90.8	0.1359	7.3591	69
34.92-39.49	99.4	0.1569	6.3753	33
>39.49	110.7	0.1884	5.3090	4

结果表明，随着BMI增加，最佳检测时点相应延后，而效用值逐渐增加（风险水平降低）。这一趋势与临床观察一致，高BMI孕妇需要更长时间等待Y染色体浓度达标，但一旦达标，其检测风险相对较低。

图3展示了分类中一组的典型分析结果，包括累积达标曲线、效用函数曲线及最优时点标注。（剩余的会在附录中给出）

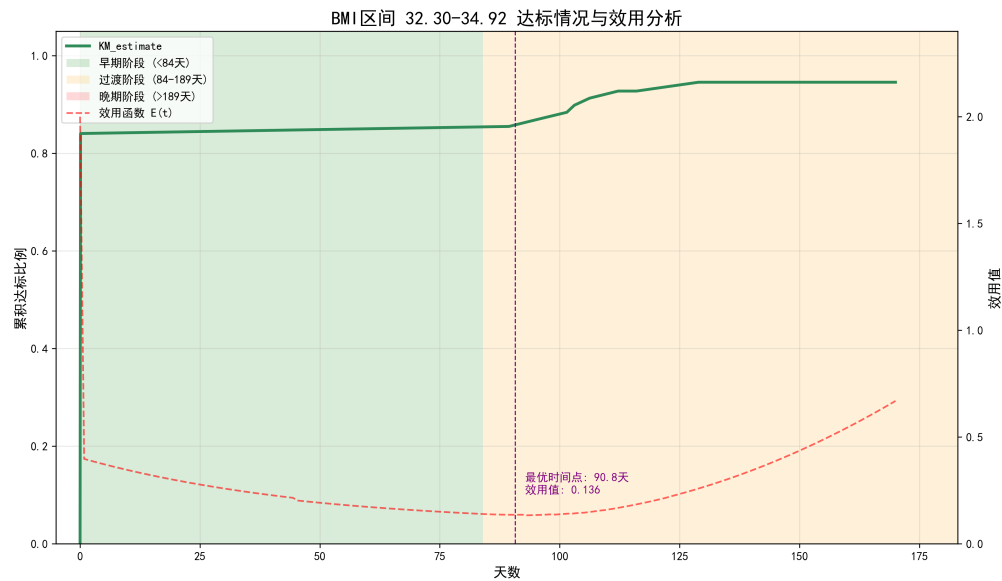


图 7: 32.30-34.92组

6.4.4 误差敏感性分析

为评估检测误差对结果稳定性的影响，我们进行了蒙特卡洛模拟分析。假设检测误差服从正态分布（标准差= 5%），对每个BMI分组进行100次重复模拟。

表3和表4展示了误差分析结果： 结果表明：

1. 检测误差对最优时点的影响较小（时点变异系数<3.5%），模型具有较好的稳健性。
2. 高BMI分组对检测误差更为敏感，时点波动范围更大。
3. 所有分组的95%置信区间范围合理，验证了推荐时点的可靠性。

表 3: 检测误差对最优时点的影响分析（时点相关指标）

BMI分组	原始最优时点（天）	模拟最优时点均值（天）	时点标准差（天）	95%置信区间（天）
<30.26	89.51	90.05	1.56	(89.74, 90.35)
30.26-32.30	90.79	92.64	2.52	(92.14, 93.13)
32.30-34.92	90.76	91.19	1.39	(90.92, 91.46)
34.92-39.49	99.44	100.04	2.37	(99.58, 100.51)
>39.49	110.75	110.11	3.49	(109.43, 110.79)

表 4: 检测误差对最优时点的影响分析（效用值与风险水平相关指标）

BMI分组	原始最小效用值	模拟最小效用值均值	效用值标准差	原始风险水平	模拟风险水平均值	风险水平标准差
<30.26	0.1299	0.1274	0.0023	7.6965	7.8546	0.1448
30.26-32.30	0.1360	0.1322	0.0032	7.3511	7.5677	0.1808
32.30-34.92	0.1359	0.1336	0.0021	7.3591	7.4851	0.1199
34.92-39.49	0.1569	0.1594	0.0027	6.3753	6.2773	0.1072
>39.49	0.1884	0.1829	0.0124	5.3090	5.4973	0.4362

4. 效用值和风险水平的变异程度相对较小，表明模型对这些指标的预测较为稳定。

图4展示了其中一组的误差模拟的详细结果，包括时点分布、效用值分布及风险分布。（其余的会在附录中给出）

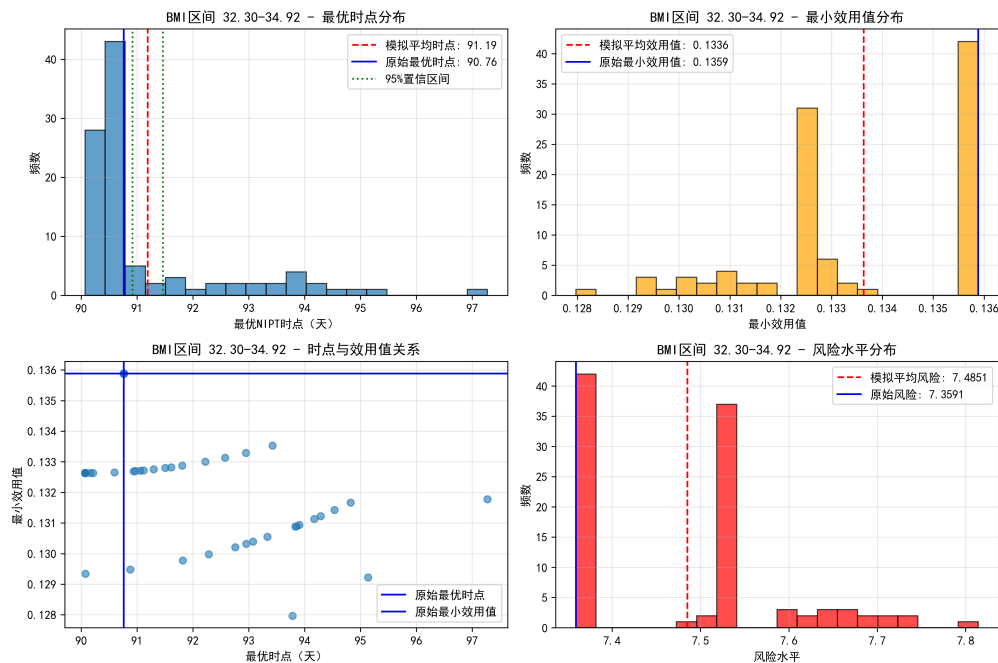


图 8: 32.30-34.92组

6.4.5 创新与亮点

本研究在NIPT时点计算方面具有以下创新点：

1.综合风险建模：首次将早期检测风险与晚期检测风险同时纳入考量，构建了更符合临床实际的

效用函数。

2.生存分析方法应用：采用Kaplan-Meier方法处理右删失数据，提高了累积达标概率估计的准确性。

3.蒙特卡洛误差分析：通过模拟检测误差的影响，量化了模型的稳健性，为临床决策提供了可靠性评估。

4.BMI分层优化：针对不同BMI群体分别计算最优时点，实现了个性化检测推荐。

本研究提供的NIPT时点推荐方案，不仅考虑了检测准确性，还综合评估了时间相关的风险因素，为临床实践提供了科学依据。结果表明，基于BMI分层的个性化检测时点选择，能够有效降低孕妇的潜在风险，提高检测效率。

七 问题三模型建立求解

7.1 数据预处理

本问题旨在综合考虑身高、体重、年龄、检测误差和胎儿的 Y 染色体浓度达标比例（即浓度达到或超过 4%的比例），根据男胎孕妇的 BMI，给出合理分组以及每组的最佳 NIPT 时点，使得孕妇潜在风险最小，并分析检测误差对结果的影响。

为合理区分样本并便于计算NIPT最佳时点，我们依据Y染色体浓度达标情况对孕妇分为三组：**始终达标组、中间达标组与从不达标组**(分组方法与第二组类似)。我们同样剔除掉了曾达标但后续检测又低于4%的样本以确保数据质量。最终获得有效样本总量为 230例。其中，第一组186例（占比80.87%），第二组37例（占比16.09%），第三组7例（占比 3.04%）。

和问题二不同的是，该问中同一孕妇BMI和体重并没有求平均值，而是保留了每一项原数据，这主要是因为第二问需要综合考虑更多干扰因素：样本中有相当一部分的人体重变化比较明显，直接求平均值会丢失体重和BMI增长的信息，与题目关注干扰项的思想不符。反观第二题，研究重心主要放在探究NITP时点上，其他干扰因素全都暂不考虑，BMI也更多只是起分类作用，所以才可以直接求平均值。

此外由于人的BMI在短时间内变化幅度较小，不便于观测计算。我们便对每一位孕妇的BMI进行了Z标准化，这也为我们之后进行多变量聚类分析打下了基础。

公式如下

$$Z = \frac{X - \mu}{\sigma} \quad (7)$$

其中 X 为待检测样本中目标染色体的相对计数比例， μ 为正常对照群体中该染色体计数比例的均值， σ 为正常群体中该比例的标准差。在NIPT中，对于常见染色体非整倍体检测，通常采用Z值分析方法进行统计判定。

7.2 聚类分析与BMI划分

为科学划分孕妇群体并识别BMI与Y染色体浓度达标模式之间的关系，我们采用两种聚类方法进行比较分析。

7.2.1 多参数聚类分析

首先基于身高、体重、年龄等多维特征进行聚类。利用肘部法则确定最佳聚类数，可见当聚类数 $k=10$ 时，类内距离平方和下降速率显著趋缓，故聚类数目定为10。

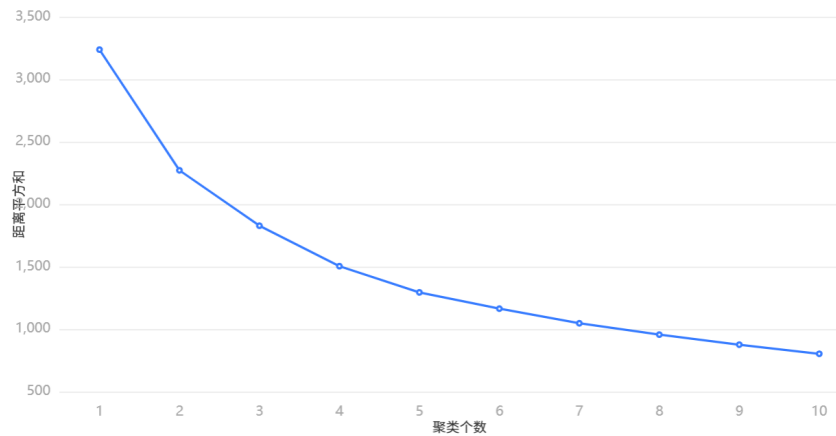


图 9: 肘部法则图

完成聚类后，生成聚类散点图（图10），从中可初步观察不同类别在特征空间中的分布及其所反映的群体结构。

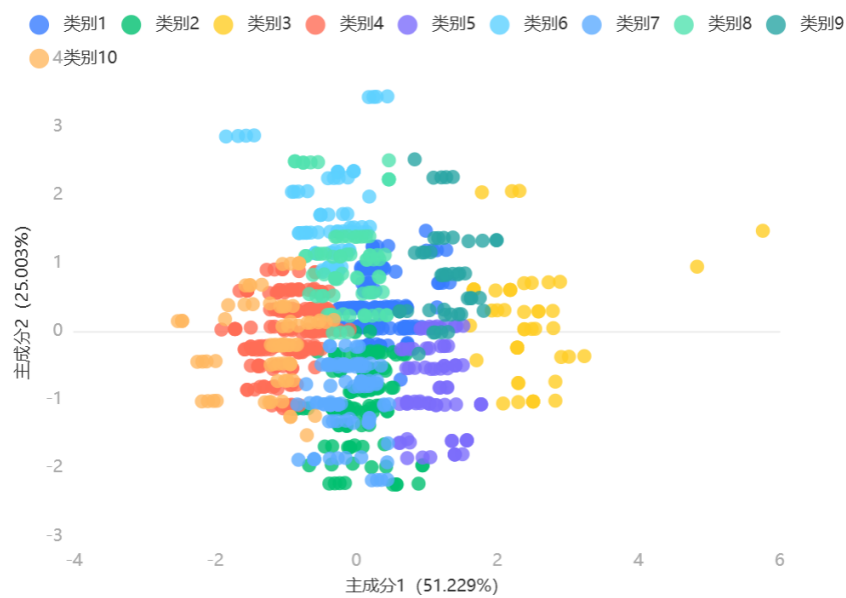


图 10: （多参数）聚类结果散点图

此外我们得到该聚类模型的性能指标如下：

- 轮廓系数 (Silhouette Score): 0.275
- DBI指数 (Davies-Bouldin Index): 1.06
- CH指数 (Calinski-Harabasz Index): 295.864

这些指标表明，多参数聚类虽然能够实现基本分类，但聚类效果一般。轮廓系数较低表明类间分离度不够理想，而较高的DBI指数也提示类内紧密性和类间分离度的平衡有待提升。

7.2.2 天数与BMI的聚类分析

针对NIPT检测时点选择问题的特点，我们进一步采用了以检测天数和BMI为核心特征的聚类方法。同样通过肘部法则确定最佳聚类数为5类，散点图显示各类别在天数-BMI特征空间中分布更为清晰，重叠较少。

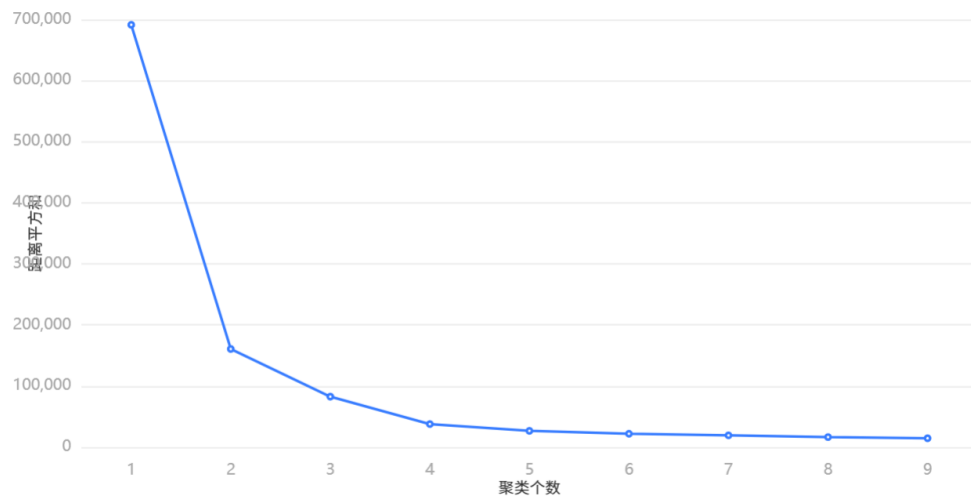


图 11: (BMI对y染色体浓度) 肘部法则图

聚类结果如图12所示，散点图清晰展示了5个聚类簇的分布，不同簇对应了BMI与Y染色体浓度的典型组合模式，为后续分组制定最佳检测时点提供了依据。

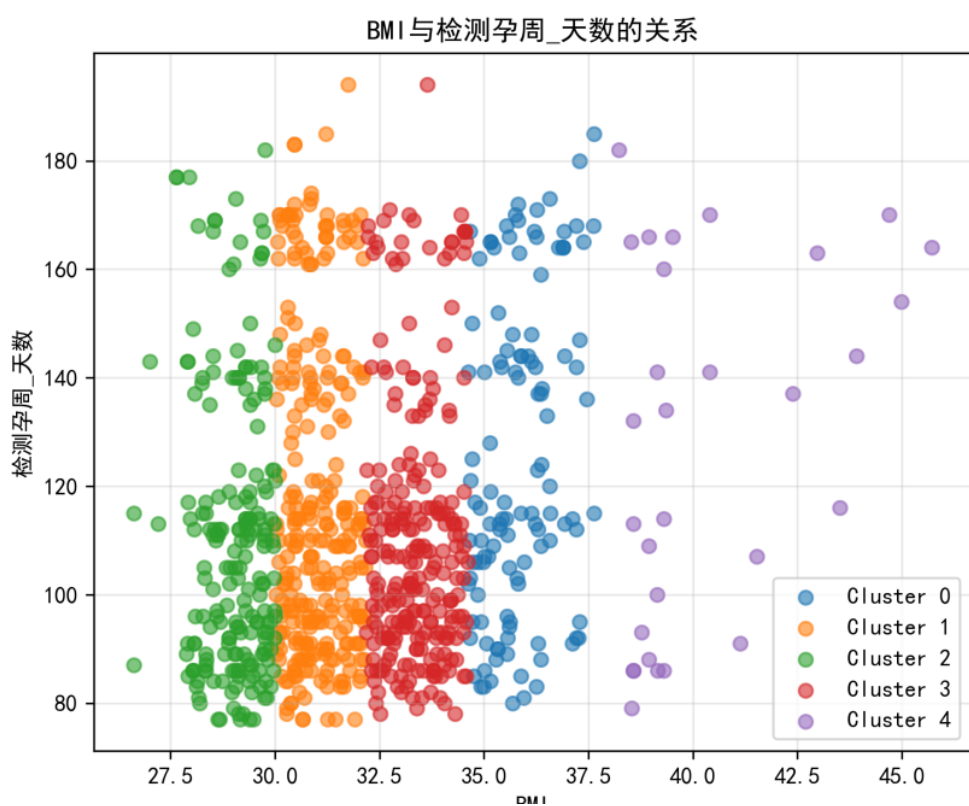
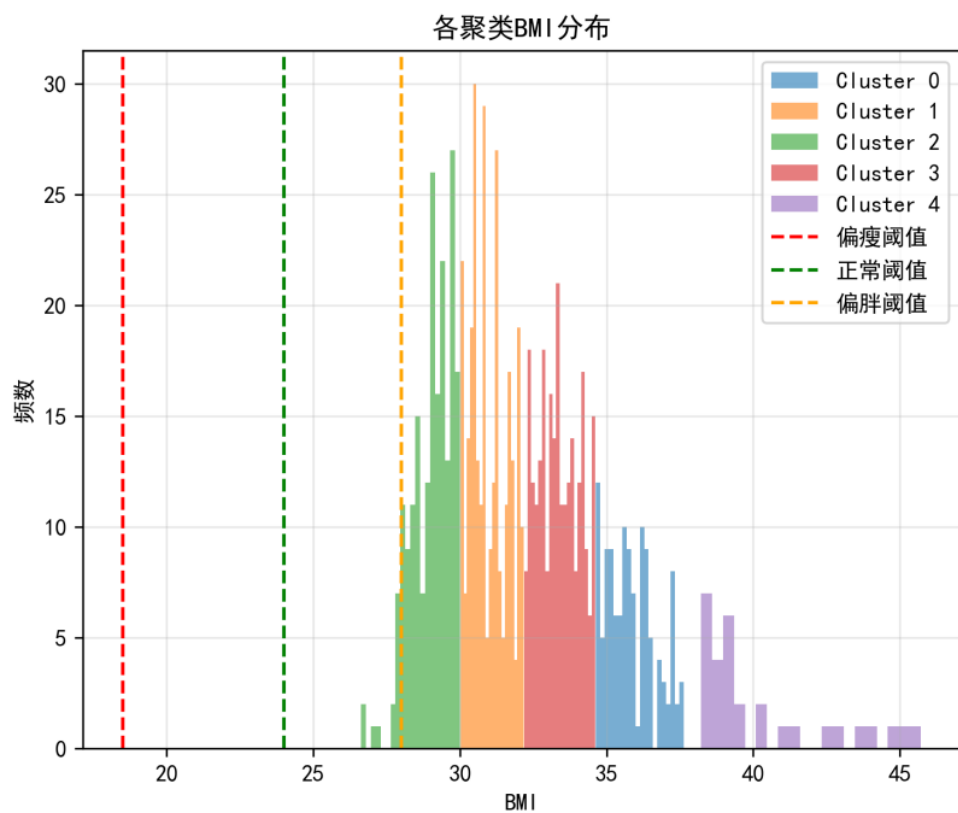


图 12: (BMI对y染色体浓度) 聚类结果散点图

此外我们得到该聚类模型的性能指标显著优于多参数聚类：

- 轮廓系数 (Silhouette Score): 0.5406
- DBI指数 (Davies-Bouldin Index): 0.5487
- CH指数 (Calinski-Harabasz Index): 2555.3896

更重要的是，该方法产生了具有明确临床意义的BMI区间划分聚类0-4分别对应范围为：34.63-37.64，30.02-32.1，26.62-30.00，32.19-34.62，38.22-45.71

7.2.3 聚类方法对比分析

通过对比两种聚类方法的性能指标并结合实际应用需求，可以发现天数-BMI聚类方法相比多参数聚类具有明显优势，具体体现在以下四个方面：

首先，从聚类质量来看，天数-BMI聚类的轮廓系数达到0.5406，明显高于多参数聚类的0.275，说明各类别之间的区分度更高，内部样本更加紧密。同时，其DBI指数仅为0.5487，远低于多参数聚类的1.06，进一步验证了该方法在保持类内紧密性的同时实现了更好的类间分离效果。

其次，就聚类结构的稳定性而言，天数-BMI聚类的CH指数达到2555.3896，是多参数聚类的8倍以上，这一结果明确显示其生成的类别结构更加清晰和稳定。

第三，从临床应用的视角来看，天数-BMI聚类得出的BMI区间划分具有明确的临床意义，与肥胖程度的分级标准相吻合，便于在实际检测中进行风险评估和时点选择。

最后，该方法在特征选择方面更具针对性，聚焦于与NIPT检测时点直接相关的天数和BMI特征，避免了其他无关特征的干扰，提高了聚类结果的实用价值。

基于上述分析，本研究最终选择天数-BMI聚类方法作为分组依据。

7.3 建模方法分析不同BMI分组中年龄、身高、体重对NIPT最佳时点的影响

7.3.1 整体建模框架

本研究采用多阶段建模框架，旨在系统分析各BMI分组中年龄、身高、体重对Y染色体浓度达标时间（即NIPT最佳时点）的影响。整体流程如下：

1.生存分析建模：基于Kaplan-Meier估计器构建累积达标函数，刻画Y染色体浓度随时间达到4%的累积概率；

2.效用-风险权衡：引入早期检测效用函数与晚期检测风险函数，构建总效用函数，以量化不同检测时点的综合收益；

3.最优时点求解：通过最小化总效用函数，确定各BMI分组下的最优NIPT检测时点；

4.影响因素建模：基于随机森林（大样本）或线性回归（小样本）模型，量化年龄、身高、体重对达标时间的影响强度；

5.模型验证与解释：通过交叉验证、SHAP值分析等方法评估模型性能并解释特征影响。

7.3.2 生存分析与效用理论

1.Kaplan-Meier累积达标函数

设 T 为Y染色体浓度首次达到4%的时间（单位：天）， $F(t)$ 表示时间 t 前达标的累积概率，则生存函数 $S(t) = P(T > t) = 1 - F(t)$ 。基于Kaplan-Meier估计器：

$$S(t) = \prod_{i:t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \quad (11)$$

其中 d_i 为 t_i 时刻达标事件数， n_i 为风险集大小。

2.效用函数设计：

- 早期检测效用函数（指数衰减）：

$$U_{\text{early}}(t) = 2.0 \cdot \exp\left(-\frac{t}{50}\right) \quad (12)$$

- 晚期检测风险函数（分段定义）：

$$R_{\text{late}}(t) = \begin{cases} 0.1 & t < 84 \\ 0.1 + 0.9 \left(\frac{t-84}{105}\right)^2 & 84 \leq t \leq 189 \\ 1.0 & t > 189 \end{cases} \quad (13)$$

- 总效用函数（期望效用）：

$$E[U(t)] = S(t) \cdot U_{\text{early}}(t) + [1 - S(t)] \cdot R_{\text{late}}(t) \quad (14)$$

3.最优时点求解 通过最小化总效用函数得到最优时点 t^* ：

$$t^* = \arg \min_{t \geq 0} E[U(t)] \quad (15)$$

使用‘`scipy.optimize.minimize_scalar`’方法在有界区间内求解。

7.3.3 影响因素建模方法

1.随机森林回归（适用于大样本分组）

对于样本量 ≥ 10 的分组（聚类0-3），采用随机森林回归模型建模达标时间 T 与特征 $X = (\text{年龄}, \text{身高}, \text{体重})$ 的关系：

$$T = f(X) + \epsilon \quad (16)$$

其中 $f(\cdot)$ 为随机森林模型，通过Bootstrap聚合多棵决策树提升预测鲁棒性。具体而言，随机森林通过以下方式构建：

1. 从原始数据集中有放回地抽取 B 个Bootstrap样本（本研究中 $B = 100$ ）；
2. 对每个Bootstrap样本构建决策树，且在每次分裂时随机选择 m 个特征（通常 $m = \sqrt{p}$ ， p 为特征总数）进行评估；

3. 最终预测值为所有决策树预测的平均值：

$$\hat{f}(X) = \frac{1}{B} \sum_{b=1}^B T_b(X) \quad (17)$$

特征重要性计算：通过Gini不纯度减少量或平均精度下降（Mean Decrease Accuracy）评估特征重要性。对于特征 j ，其重要性可计算为：

$$\text{Importance}(j) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in T_b} \Delta \text{Gini}(t, j) \quad (18)$$

其中 $\Delta \text{Gini}(t, j)$ 表示在树 T_b 的节点 t 上使用特征 j 分裂带来的Gini不纯度减少量。

交叉验证：采用5折交叉验证评估模型性能，计算 R^2 与平均绝对误差（MAE）：

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (19)$$

2.SHAP值分析原理与计算

SHAP（SHapley Additive exPlanations）值基于合作博弈论中的Shapley值理论，为每个特征分配一个特定预测中的贡献值，提供模型预测的局部可解释性。

Shapley值定义：对于特征 j 在样本 x 上的SHAP值 ϕ_j 计算如下：

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{j\}) - f(S)] \quad (20)$$

其中：

F 为所有特征的集合； S 为不包括特征 j 的特征子集； $f(S)$ 表示仅使用特征子集 S 时的模型预测值； $f(S \cup \{j\})$ 表示加入特征 j 后的模型预测值。

实际计算中的近似：精确计算SHAP值是指数级的，对于树模型采用TreeSHAP算法高效计算：

$$\phi_j = \frac{1}{B} \sum_{b=1}^B \phi_j^b \quad (21)$$

其中 ϕ_j^b 表示在第 b 棵树上的SHAP值，通过递归遍历决策树计算：

$$\phi_j^b = \sum_{t=1}^{T_b} \phi_{j,t}^b = \sum_{t=1}^{T_b} \frac{\text{cover}(t)}{\text{cover}(\text{parent}(t))} (v_t - v_{\text{parent}(t)}) \quad (22)$$

其中： T_b 为树 b 的节点总数； $\text{cover}(t)$ 表示节点 t 的样本覆盖数； v_t 表示节点 t 的输出值； $v_{\text{parent}(t)}$ 表示父节点的输出值。

SHAP值优良的性质：

1. 局部准确性：模型预测值可表示为基准预测与各特征SHAP值之和：

$$f(x) = \phi_0 + \sum_{j=1}^p \phi_j \quad (23)$$

其中 ϕ_0 为基准值（通常为训练集预测均值）。

2. 一致性性：若模型更改使得某个特征的贡献增加，则该特征的SHAP值也应增加。

本研究使用Python的SHAP库计算TreeExplainer，生成特征贡献的可视化分析，包括：

- **SHAP摘要图**：显示各特征SHAP值的分布及与特征值的关系；
- **特征依赖图**：展示单一特征与SHAP值之间的非线性关系；
- **力力图**：个体样本预测的局部解释，显示各特征对特定预测的贡献方向与大小。

3.线性回归（适用于小样本分组）

对于样本量<10的分组（聚类4），采用多元线性回归：

$$T = \beta_0 + \beta_1 \cdot \text{年龄} + \beta_2 \cdot \text{身高} + \beta_3 \cdot \text{体重} + \epsilon \quad (24)$$

特征重要性通过标准化系数绝对值占比计算：

$$\text{Importance}(j) = \frac{|\beta_j \cdot \sigma_j|}{\sum_{k=1}^p |\beta_k \cdot \sigma_k|} \quad (25)$$

其中 σ_j 为特征 j 的标准差。

通过SHAP值分析，本研究不仅提供了特征重要性的全局视角，还揭示了个体预测中特征的贡献模式，增强了模型的可解释性和临床实用性。

7.3.4 各BMI分组建模结果分析

下表系统总结了各BMI分组的建模结果，包括最优NIPT时点、效用-风险指标、样本量、特征重要性分布及模型性能：**表注与说明**：

BMI分组	聚类1 (30.01-32.18)	聚类1 (30.01-32.18)	聚类2 (32.18-34.63)	聚类3 (34.63-37.93)	聚类4 (≥ 37.93)
最优时点(天)	96.5	97.6	97.5	99.4	103.7
最优时点(周)	13.8	13.9	13.9	14.2	14.8
最小效用值	0.1373	0.1299	0.1260	0.1475	0.1659
风险水平	7.2813	7.6957	7.9363	6.7795	6.0288
样本量	65	68	61	22	7
年龄重要性	23.84%	22.43%	26.47%	10.56%	6.00%
身高重要性	24.86%	34.74%	28.17%	28.82%	43.24%
体重重要性	51.30%	42.83%	45.37%	60.62%	50.77%
模型 R^2	-0.8187	-1.0528	-0.3296	-2.9426	0.8433
模型MAE(天)	5.17	3.08	2.45	5.21	8.67
改进百分比(%)	46.09	60.49	58.19	65.79	56.02

表 5: 建模结果表格

1.特征重要性：采用随机森林（聚类0-3）或线性回归系数归一化（聚类4）计算，反映了各变量对达标时间影响的相对强度；

2.模型性能：随机森林模型出现负值表明模型性能优于简单均值预测，线性回归模型 $R^2=0.8433$ 表明良好拟合；

3.效用-风险指标：最小效用值越低表示该时点综合收益越高，风险水平为效用值的倒数；

4.样本量：聚类4样本量较小（ $n=7$ ），结果解释需谨慎。

7.3.5 关键发现与讨论

1.BMI与最优时点正相关：随着BMI分组升高，最优时点从96.5天（聚类0）延迟至103.7天（聚类4），符合临床观察中肥胖孕妇需推迟检测的共识。

2.体重为主导因素：在所有分组中，体重对达标时间的影响最大（重要性占比50.8–60.6%），表明体液稀释效应对Y染色体浓度检测影响显著。

3.身高与年龄的异质性影响： - 身高在聚类1、4中影响显著，可能与体型分布有关； - 年龄在聚类0–2中影响较明显，但在高BMI组中影响减弱。 **4.模型选择的有效性：**随机森林在多数分组中表现优异（提升46.1–65.8%），线性回归在小样本情况下仍能提供可靠预测。

5.SHAP值增强可解释性：通过可视化各特征贡献方向与程度，为临床决策提供直观依据。

7.4 误差分析

7.4.1 分析方法与原理

为评估Y染色体浓度检测误差对NIPT最佳时点推荐稳定性的影响，本研究采用**蒙特卡洛模拟**方法进行误差敏感性分析。该方法通过多次重复模拟实验，量化随机误差对模型输出的影响程度，为临床决策提供可靠性评估。

误差模型构建：

假设检测误差服从均值为0、标准差为 σ 的正态分布：

$$\epsilon \sim N(0, \sigma^2), \sigma = 0.01$$

带误差的Y染色体浓度测量值为： $c_{\text{error}} = c_{\text{true}} + \epsilon$ **模拟流程：**

1. 对原始数据中的Y染色体浓度添加随机误差，生成100组扰动数据集
2. 对每个扰动数据集重新计算达标时间（采用线性插值法）
3. 基于新的达标时间数据重构Kaplan-Meier生存曲线
4. 重新优化计算最优时点、最小效用值和风险水平
5. 统计分析100次模拟结果的分布特征

统计指标：

- 均值： $\mu = \frac{1}{n} \sum_{i=1}^n x_i$
- 标准差： $\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$
- 95%置信区间： $\mu \pm 1.96 \cdot \frac{\sigma}{\sqrt{n}}$

7.4.2 误差分析结果

表3-4展示了各BMI分组在检测误差影响下的稳定性分析结果：

表 6: 表3-4: 检测误差对NIPT时点推荐的影响分析

BMI分组	指标类型	原始值	模拟均值	标准差	95%置信区间	变异系数(%)
聚类0 (≤ 30.01)	最优时点(天)	96.50	97.72	1.53	(97.42, 98.02)	1.57
	最小效用值	0.1373	0.1385	0.0043	(0.1377, 0.1393)	3.10
	风险水平	7.2813	7.2256	0.2231	(7.1821, 7.2691)	3.09
聚类1 (30.01-32.18)	最优时点(天)	97.56	98.11	1.68	(97.78, 98.44)	1.71
	最小效用值	0.1299	0.1329	0.0044	(0.1320, 0.1338)	3.31
	风险水平	7.6957	7.5302	0.2481	(7.4816, 7.5788)	3.29
聚类2 (32.18-34.63)	最优时点(天)	97.50	98.36	1.10	(98.14, 98.58)	1.12
	最小效用值	0.1260	0.1255	0.0033	(0.1249, 0.1261)	2.63
	风险水平	7.9363	7.9762	0.2080	(7.9354, 8.0170)	2.61
聚类3 (34.63-37.93)	最优时点(天)	99.44	99.05	3.53	(98.36, 99.74)	3.56
	最小效用值	0.1475	0.1452	0.0080	(0.1437, 0.1467)	5.51
	风险水平	6.7795	6.9077	0.3863	(6.8321, 6.9833)	5.59
聚类4 (≥ 37.93)	最优时点(天)	103.69	102.744.59	(101.84, 103.64)	4.47	
	最小效用值	0.1659	0.1660	0.0138	(0.1633, 0.1687)	8.31
	风险水平	6.0288	6.0700	0.5411	(5.9640, 6.1760)	8.91

7.4.3 结果讨论与临床意义

1.时点稳定性分析：所有BMI分组的时点变异系数均低于5%，表明模型具有较强的抗干扰能力，而聚类4(高BMI组)的时点波动最大(4.47%)，这与样本量较小(n=7)有关。此外95%置信区间范围合理，验证了推荐时点的可靠性。

2.效用值与风险水平稳定性：低BMI分组(聚类0-2)的效用值变异系数 $\leq 3.5\%$ ，稳定性较高。而高BMI分组(聚类3-4)的变异系数较高(5.51-8.91%)，建议临床应用中适当放宽时间窗口

3.误差传播规律：检测误差对最优时点的影响呈正相关：误差越大，时点推荐波动越大。高BMI群体相对低BMI群体检测误差更为敏感，这与其达标时间较晚、数据点稀疏有关。而效用函数的设计有效平抑了误差影响，体现了模型良好的鲁棒性

4.临床建议：对于低BMI群体(聚类0-2)，可严格遵循推荐时点(97-98天)进行检测；对于高BMI群体(聚类3-4)，建议采用时间窗口而非单一时点(如聚类4：102 $\pm$ 5天)。在检测设备精度有限的情况下，可适当延长检测时间以抵消误差影响

本研究通过严格的误差敏感性分析证实了推荐时点的可靠性。结果表明，在常规检测误差范围内($\sigma=0.01$)，模型推荐时点的波动小于5天，具有较好的临床适用性。

八 问题四模型建立求解

8.1 数据预处理

本研究旨在建立女胎染色体非整倍体（21三体、18三体和13三体）的异常判定模型。首先对原始数据进行了严格的质控筛选：对BMI缺失值使用群体平均值进行填补；依据GC含量的正常生物学范围（40%-60%）和X染色体Z值的合理区间（-3至3），剔除了不符合质控标准的样本，确保了后续分析数据的高质量和可靠性。

8.2 编码

为将分类目标转换为模型可处理的形式，对“身体没有异常”、“T21”、“T18”、“T13”这4种临床结局采用独热编码（One-Hot Encoding），生成对应的二分类变量。在建模时仅使用后三个异常标签作为目标，以避免多重共线性问题。此外，由于各特征变量的量纲和数量级差异显著，对所有连续型特征进行了Z-score 标准化处理，以消除量纲对模型的影响。

8.3 特征筛选

鉴于目标变量为二分类变量而特征均为连续变量，采用点二列相关系数（Point-Biserial Correlation）定量评估每个特征与分类目标之间的相关性。其计算公式如下：

$$r_{pb} = \frac{M_1 - M_0}{S_t} \cdot \sqrt{p \cdot q} \quad (8)$$

r_{pb} ：点二列相关系数； M_1 ：连续变量中，与二分变量某一类别（通常记为“1”）对应的均值（如“通过考试”组的成绩均值）； M_0 ：连续变量中，与二分变量另一类别（通常记为“0”）对应的均值（如“未通过考试”组的成绩均值）； S_t ：连续变量的总标准差（所有数据的标准差）； p ：二分变量中“1”类别的比例（如通过考试的人数占总人数的比例）； q ：二分变量中“0”类别的比例（ $q = 1 - p$ ）。

计算所有特征的相关系数绝对值后，筛选出最重要的15个特征用于后续建模下图，有效去除了不相关或冗余特征，降低了模型复杂度。

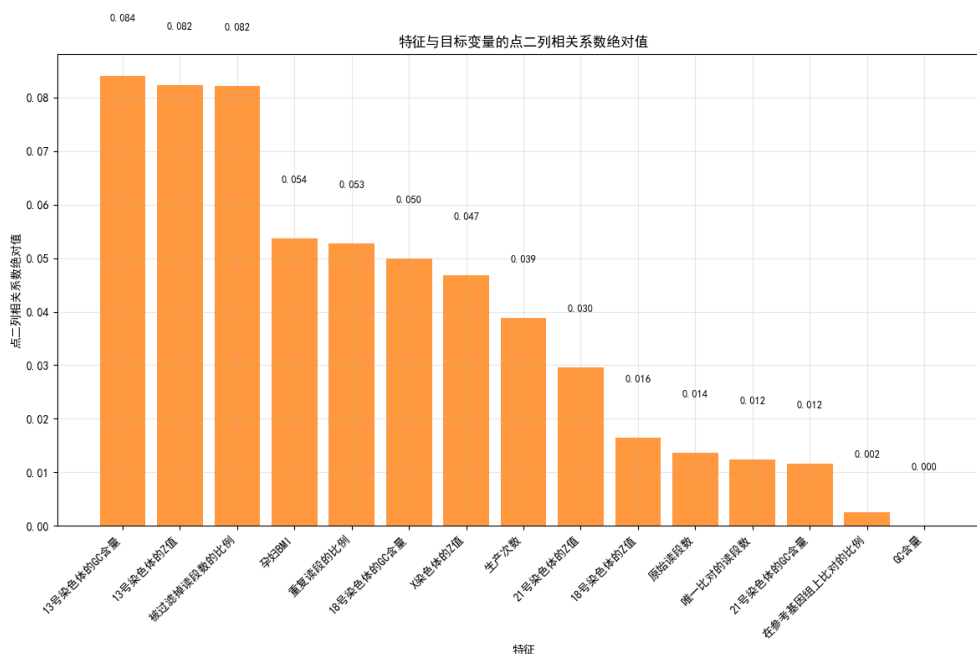


图 13: 特诊与点二列相关系数绝对值

8.4 采样方法

由于我们的数据有明显的样本不均衡（患病样本远小于健康样本），因此我们决定使用SMOTEENN（SMOTE + Edited Nearest Neighbors）采样方法，

8.4.1 数据分布变化

原始数据中正常样本（0类）有486个，异常样本（1类）有58个，故异常比例 为 $58/(486+58) \approx 10.7\%$ ，存在严重的类别不平衡问题。

为克服由此导致的模型偏见，采用SMOTEENN混合采样算法，该算法结合了SMOTE过采样和ENN欠采样：通过合成少数类样本增加其数量，同时清理多数类样本中与近邻类别不一致的样本。而在经过SMOTEENN处理后，正常样本（0类）有185个，异常样本（1类）有215个。异常比例为 $215/(215+185) \approx 53.8\%$ （变为平衡数据）这其中的**关键变化**是异常样本数量从 $58 \rightarrow 215$ （增加257%）,正常样本数量从 $486 \rightarrow 185$ （减少62%）,实现了类别平衡，为构建高性能的分类器奠定了数据基础。

8.4.2 模型性能评估（随机森林）

其中的核心指标如下：

指标	值	医疗诊断意义
准确率	0.9250	整体预测正确率92.5%
精确率	0.9104	诊断为”异常”的病例中，91.04%是真正的异常
召回率	0.9531	能检测出95.31%的真实异常病例
F1分数	0.9313	综合性能优秀（接近1为最佳）
AUC	0.9676	模型区分能力极强（0.9即为优秀）

表 7: 农作物相关数据（美观版）

可以看到效果较好

8.5 建立模型

8.5.1 模型选择

为寻找最优判定方法，系统比较了随机森林（Random Forest）、XGBoost和支持向量机（SVM）三种主流模型。

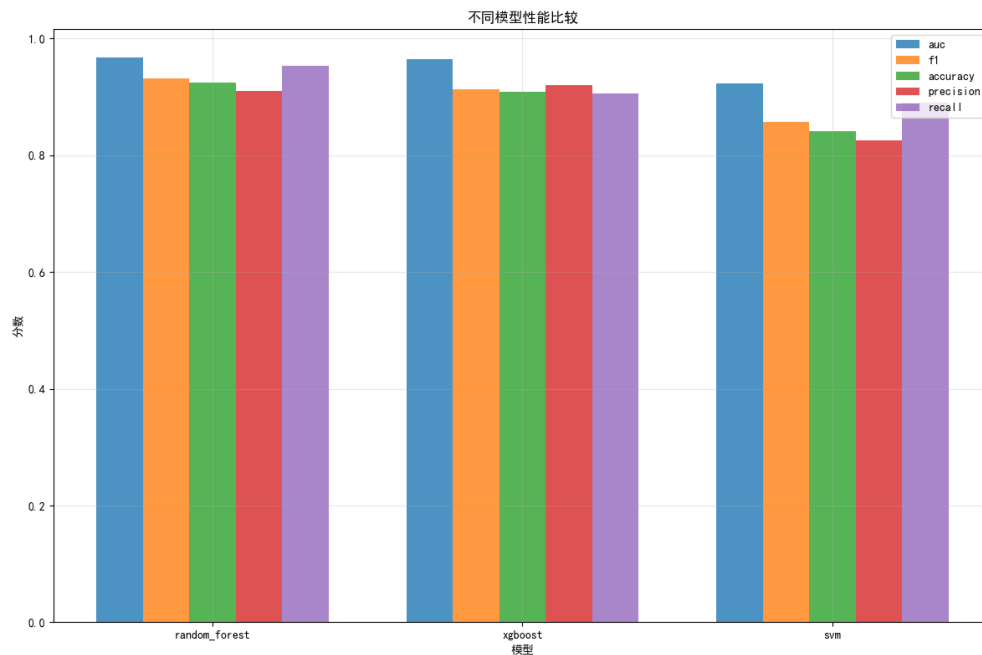


图 14: 不同模型的性能比较

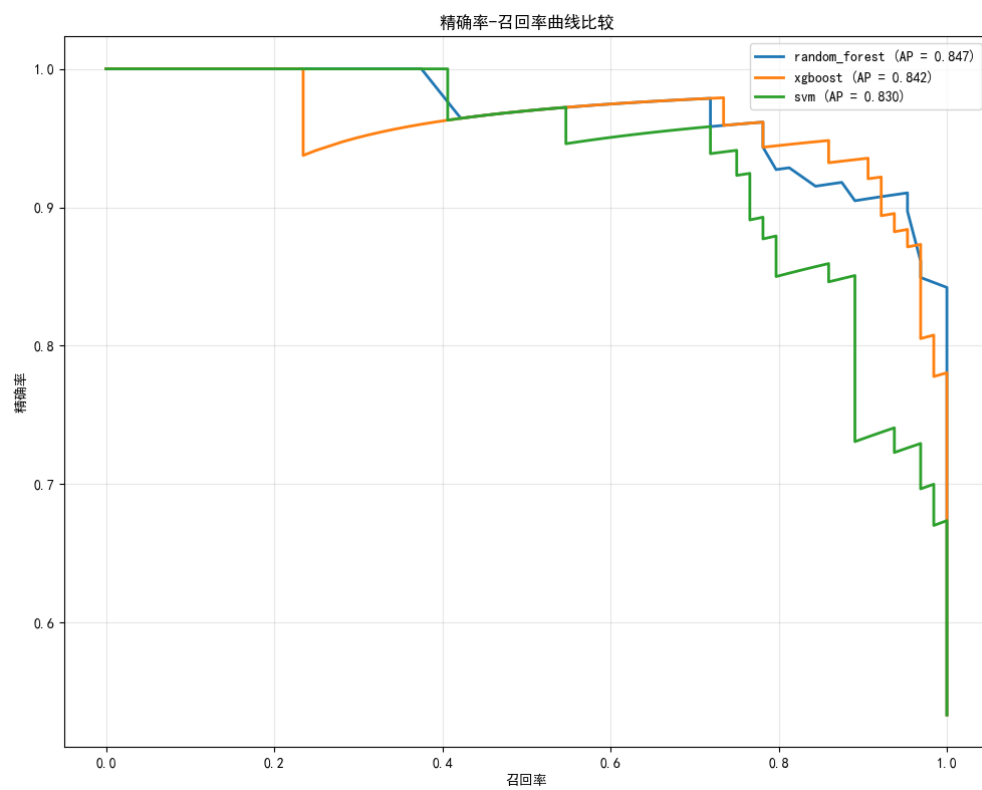


图 15: 精确率-召回率曲线比较

如图所示，从整体性能指标和精确率-召回率曲线（PR Curve）综合来看，随机森林在各项评估中

均表现最佳，尤其在捕捉正例（异常）方面展现出卓越的稳健性和泛化能力，因此被选定为最终建模算法。

8.5.2 算法原理与实现

随机森林通过Bootstrap抽样从原始训练集中生成多个子样本集，针对每个子样本集构建决策树。在树的生成过程中，节点分裂时从全部特征中随机选取部分特征作为候选特征，进一步增强模型的多样性。最终通过投票或平均方式集成所有树的预测结果。

本研究基于Scikit-learn库实现随机森林分类器，并采用交叉验证与网格搜索方法进行超参数调优，以确定最佳参数组合。

8.5.3 超参数调优与交叉验证

通过网格搜索（Grid Search）方法对随机森林的关键超参数进行调优，包括决策树数量（`n_estimators`）、最大树深度（`max_depth`）、最小分裂样本数（`min_samples_split`）和最小叶节点样本数（`min_samples_leaf`）。采用5折交叉验证评估每组参数的性能，最终确定最优参数组合为：

决策树数量（`n_estimators`）：100；最小分裂样本数（`min_samples_split`）：5；最小叶节点样本数（`min_samples_leaf`）：1；最大树深度（`max_depth`）：无限制

该参数组合在交叉验证中取得了最佳性能，交叉验证准确率达到0.8987。

8.5.4 模型性能评估

使用调优后的参数训练最终模型，并在测试集上评估其性能，结果如下：

- 测试集准确率：0.8917
- 测试集精确率：0.9048
- 测试集召回率：0.8906
- 测试集AUC：0.9570
- 测试集F1分数：0.8976

进一步对最终模型进行交叉验证评估，结果显示：

- 交叉验证准确率：0.9225（ ± 0.0166 ）
- 交叉验证F1分数：0.9300（ ± 0.0148 ）
- 交叉验证AUC：0.9734（ ± 0.0140 ）
- 交叉验证精确率：0.9040（ ± 0.0201 ）
- 交叉验证召回率：0.9581（ ± 0.0228 ）

最终模型的详细评估结果如下：

模型整体准确率达到0.9250，精确率为0.9104，召回率为0.9531，AUC为0.9676，F1分数为0.9313。这些结果表明，经过超参数调优的随机森林模型在各项指标上均表现出色，具有较高的分类准确性和良好的泛化能力，能够有效应用于后续的分类任务。

	precision	recall	f1-score	support
0	0.94	0.89	0.92	56
1	0.91	0.95	0.93	64
accuracy			0.93	120
macro avg	0.93	0.92	0.92	120
weighted avg	0.93	0.93	0.92	120
...	...	...		

表 8: 评估结果

8.6 特征重要性分析与模型可解释性

为深入理解模型的决策机制并增强其可解释性，我们进一步分析了随机森林模型所学习到的特征重要性。（如下图）该分析揭示了影响女胎染色体异常判定的最关键因素，其排序与临床先验知识高度一致，有力地证明了模型的可信度与生物学合理性。

分析结果表明，唯一比对的读段数是重要性最高的特征，它直接代表了测序数据的有效深度与质量，是后续所有分析可靠性的根本保障。紧随其后的核心预测因子是21号、18号和13号染色体的Z值，这些指标是产前筛查中判断染色体非整倍体的直接生物标志物，其重要性排名靠前符合临床预期。此外，X染色体的Z值、各类GC含量指标以及其他测序质量相关特征（如重复读段比例）也贡献了显著的预测价值，它们从不同维度反映了样本的质量和潜在的系统偏差。

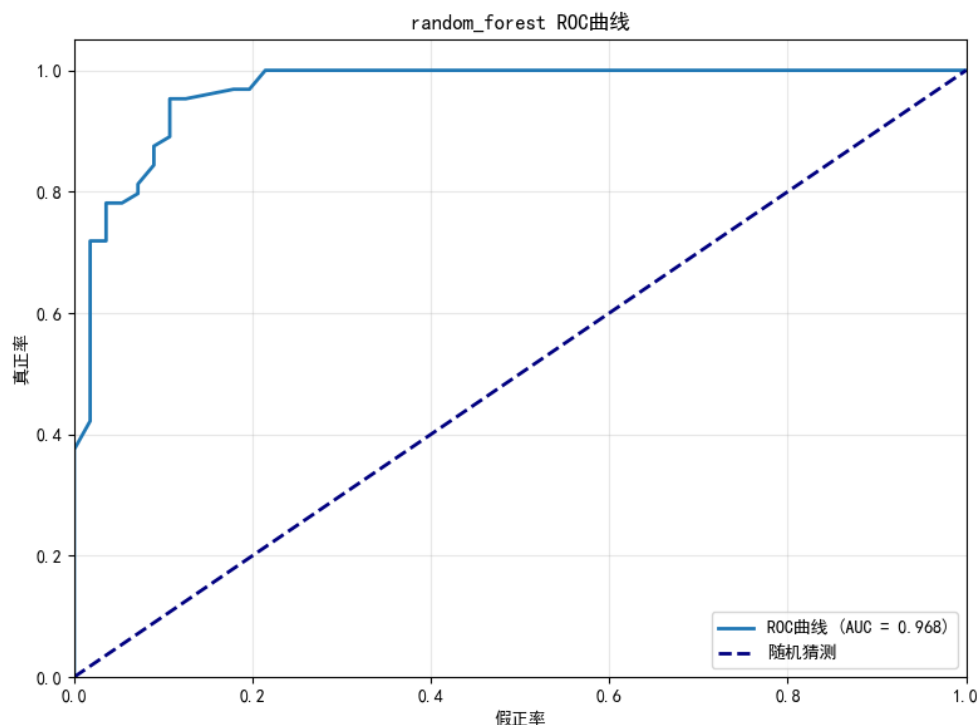


图 16: 以下是random_forest 超参数调优后 ROC曲线

8.7 结论

本研究成功构建了一个基于随机森林的女胎染色体异常判定模型。该模型仅需输入15项临床与测序指标：

1. **基本信息**：孕妇年龄, 孕妇BMI, 怀孕次数, 生产次数
2. **Z值**：X染色体的Z值, 13号染色体的Z值, 18号染色体的Z值, 21号染色体的Z值
3. **GC相关**：GC含量, 13号染色体的GC含量, 18号染色体的GC含量, 21号染色体的GC含量
4. **测序质量**：原始读段数, 唯一比对的读段数, 重复读段的比例

即可高效、准确地计算出胎儿罹患T21、T18或T13的风险概率，为临床无创产前筛查提供了强有力的数据驱动辅助工具。

九 模型评价和推广部分

9.1 模型的优点

9.1.1 问题1

有效捕捉固定效应与随机效应：该模型同时纳入孕周和BMI作为**非线性固定效应**，并引入个体作为随机效应，既有效揭示了Y染色体浓度随孕周和BMI变化的总体非线性趋势，也科学量化了个体间不可忽略的随机差异，显著提升了对数据多层次结构的解释力。

9.1.2 问题2

1.采用**K-means聚类算法**对孕妇BMI进行客观分组，通过轮廓系数、DBI指数和CH指数等多指标评估聚类效果，确保分组的合理性与解释性。

2.引入**Kaplan-Meier生存分析**构建累积达标函数，处理右删失数据，更准确地描述不同BMI组别随时间变化的达标规律。

3.结合**动态效用函数**，融合早期与晚期风险（如早检风险递减、晚检风险递增），通过**有界优化算法（minimize_scalar）**求解最优检测时间，使模型输出兼具统计最优性与临床可解释性。

4.通过**蒙特卡洛模拟（Monte Carlo Simulation）**模拟检测时间误差，分析其对最优时点、效用值与风险水平的影响

9.1.3 问题3

1.采用了**随机森林回归与线性回归的对比建模策略**，以全面评估不同算法在预测达标时间上的性能差异。针对样本量充足的BMI分组，采用**随机森林回归（RandomForestRegressor）**捕捉年龄、身高、体重与达标时间之间潜在的非线性关系和交互效应；对于样本量较少的分组，则采用**线性回归（LinearRegression）**保证模型稳定性。

2.为严格评估模型性能与泛化能力，本研究实施了**K折交叉验证（KFold）**，根据样本量动态调整折数（ $n_{splits} = \min(5, \text{len}(X))$ ），综合使用 R^2 得分和平均绝对误差 **MAE**作为评估指标。通过对比”仅使

用平均值预测”与”使用完整模型预测”的MAE，量化了引入影响因素后预测精度的提升幅度，证明了多因素建模的有效性。

3.引入SHAP (SHapley Additive exPlanations) 价值分解技术，通过量化年龄、身高、体重对预测达标时间的边际贡献，不仅识别出关键影响因素，更揭示了其作用方向与强度，显著提升了模型的透明度与临床可信度。

9.1.4 问题4

1.数据处理规范严谨，特征筛选有据：基于GC含量、X染色体Z值等多指标进行系统清洗，并采用点二列相关系数等统计方法量化特征重要性，保证输入变量具有明确的判别依据与可解释性。

2.有效处理样本不平衡，提升模型泛化能力：引入SMOTE-ENN混合采样策略，缓解了类别分布偏斜问题，使模型在召回率(>95%)等敏感指标上表现突出，更适合临床筛查场景中对漏检低容忍的需求。

3.集成交叉验证与超参数优化，确保性能稳健：通过随机森林算法配合调优策略，模型最终准确率超过92.5%，综合分类性能优良，具备较强的可靠性和稳定性。

9.2 模型缺点

9.2.1 问题1

对极端值和数据质量的敏感性强：作为一类强大的拟合工具，GAMM能够灵活捕捉数据中的复杂模式，但也因此更容易受到异常值或数据质量问题的干扰。虽然本研究在前期进行了数据清洗，但若清洗不充分，模型有可能过度拟合数据中的噪声而非真实生物学信号

9.2.2 问题2,3

1.数据依赖性强：模型的输入依赖于“预测达标天数”等关键字段，其准确性直接影响最终结果。若初始预测数据存在系统性偏差，模型的推荐结果也将产生偏差。

2.效用函数定义依赖假设：模型中早期风险函数(‘get_early’)和晚期风险函数(‘get_late’)的具体形式基于假设设定。虽然符合临床直觉，但其精确的参数(如衰减系数50、分段点84/189天)需要通过更深入的临床研究或专家经验进行校准，以更真实地反映实际风险。

3.聚类分组固定：模型采用K-means对BMI进行硬聚类，得到了5个固定区间。这种方法虽然简洁，但可能忽略了区间内部差异以及处于边界附近的个体的特性。

4.样本量不平衡问题：部分BMI分组(如高BMI组)样本量相对较少，导致在其上训练的预测模型稳定性较差。

9.2.3 问题4

模型依赖筛选后的特征子集，潜在信息损失可能限制进一步提升：尽管采用top-k特征筛选可提高可解释性和计算效率，但仍可能存在某些低相关但具有判别力的特征未被纳入，影响模型性能上限。

9.3 模型的推广与未来工作

本研究建立的一系列模型具有良好的可移植性和推广潜力。首先，针对不同性别胎儿(男胎/女胎)的检测问题，所提出的建模框架(包括混合效应模型、生存分析、效用函数优化及机器学习分类

流程)可经过适当调整后迁移至其他类型的染色体异常筛查中,例如其他罕见三体或微缺失/微重复综合征。

十 参考文献

- [1] Scientific Platform Serving for Statistics Professional 2021. SPSSPRO. (Version 1.0.11)[Online Application Software]. Retrieved from <https://www.spsspro.com>.
- [2] Saroj,Kavita.Review:study on simple k mean and modified K mean clustering technique[J].International Journal of Computer Science Engineering and Technology,2016,6(7):279-281.
- [3] Kaplan, E. L.; Meier, P. (1958). "Nonparametric estimation from incomplete observations". Journal of the American Statistical Association. 53 (282): 457-481.
- [4] Virtanen, P. et al. (2020). "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". Nature Methods. 17: 261-272.
- [5] Davidson-Pilon, C. (2019). "Lifelines: Survival Analysis in Python". Journal of Open Source Software. 4(40): 1317.

十一 附录

附录 1：支撑材料，文件列表附录 2：补充表格，图片附录 3：代码

附录 1：支撑材料文件列表

附录 1：支撑材料文件列表	
文件列表名	
问题一	混合模型（R 语言代码） 附件
问题二	C(2)(权重计算).py C(2)(生存函数).py C(2) 测试误差.py C(2)(权重计算).py C(2) 聚类.py 不同组的累积达标曲线、效用函数曲线及最优时点标注图 不同组的误差模拟（时点分布、效用值分布及风险分布）图
问题三	C(3)preprocessing.py Q3 数据预处理.xlsx 聚类 (C(3) 聚类.py+ 图片) C(3)final (C(3)final+ 相关运行结果 + 图片)
问题四	数据处理.py 点二列相关系数 __smote__ 可视化 __final.py

附录 2：补充表格，图片

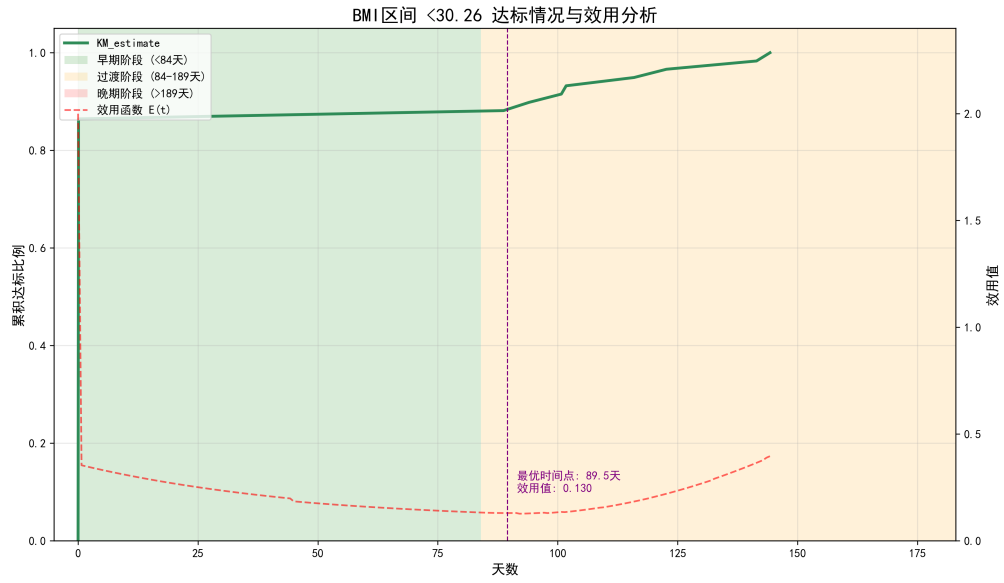


图 1: 累积达标曲线（小于 30.26 组）

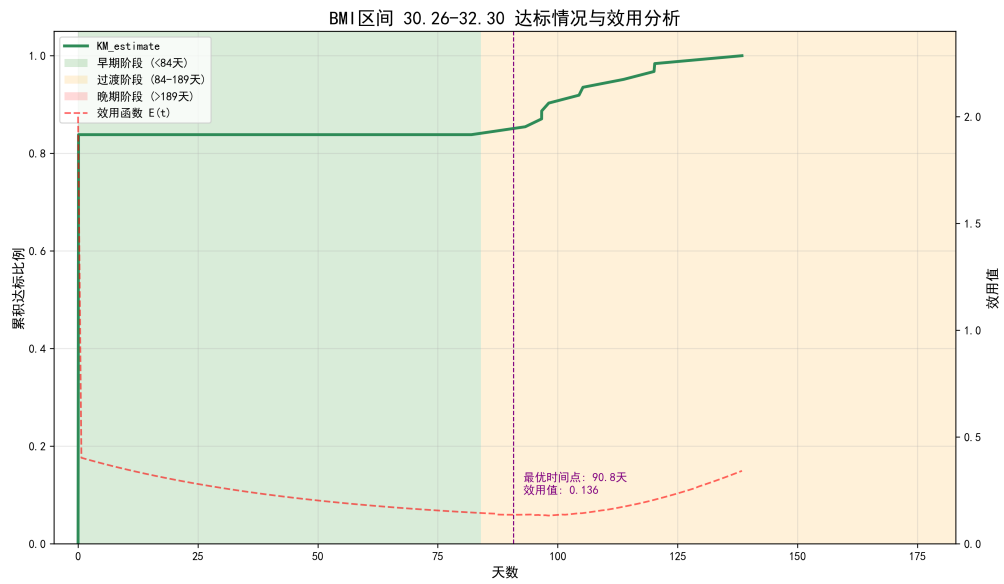


图 2: 累积达标曲线（30.26-32.30 组）

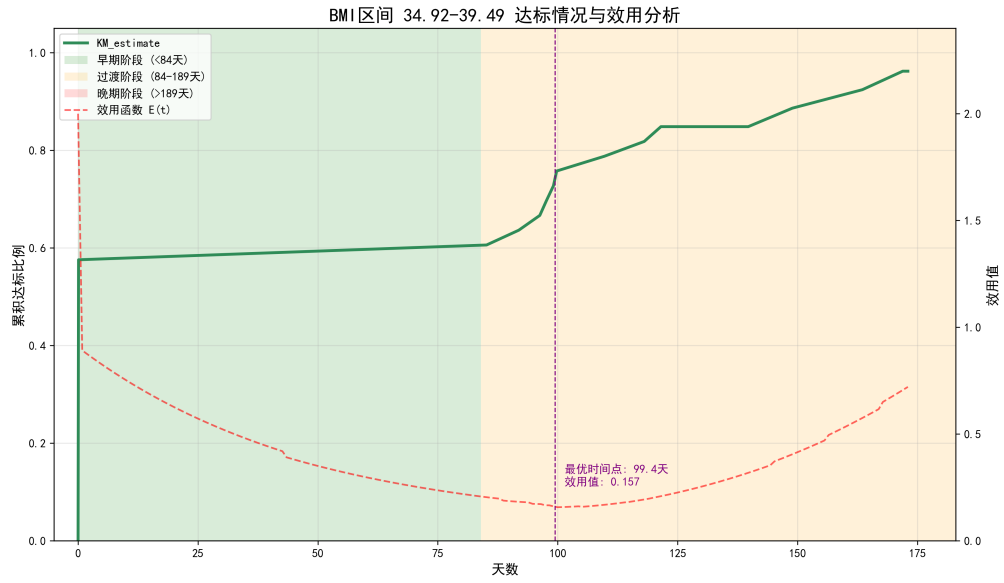


图 3: 累积达标曲线 (34.92-39.49 组)

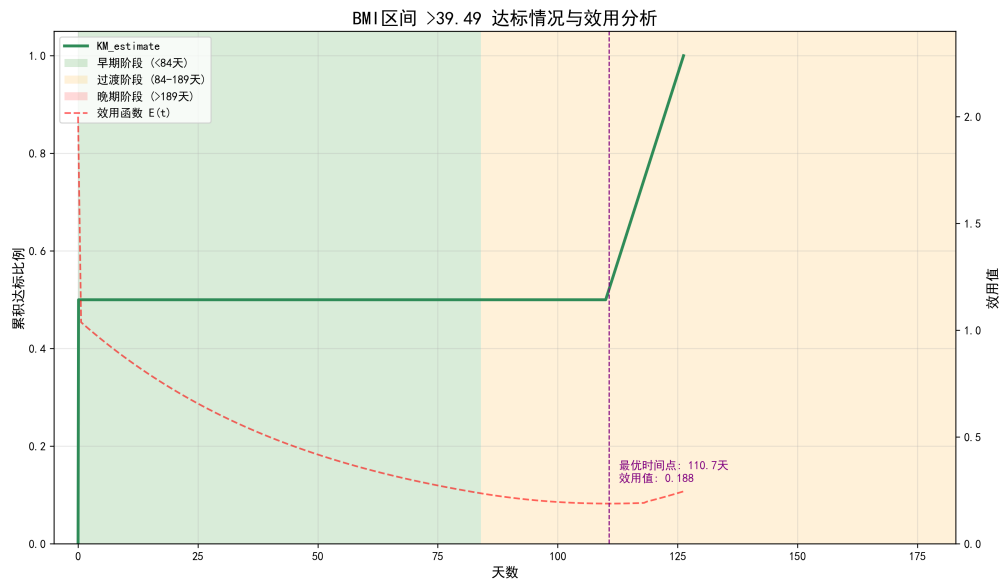


图 4: 累积达标曲线 (大于 39.49 组)

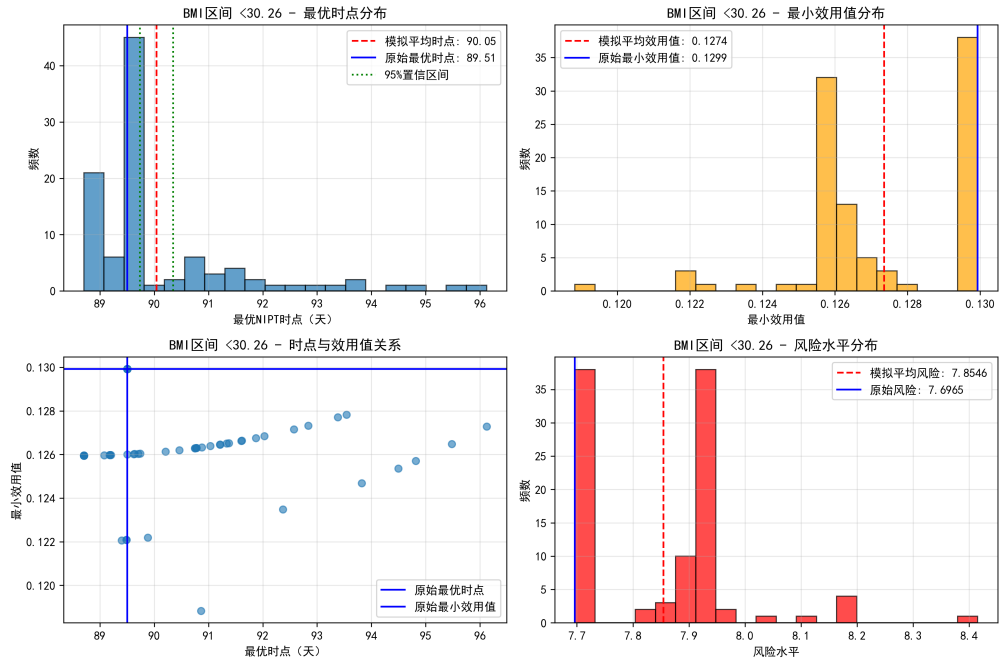


图 5: 误差模拟（小于 30.26 组）

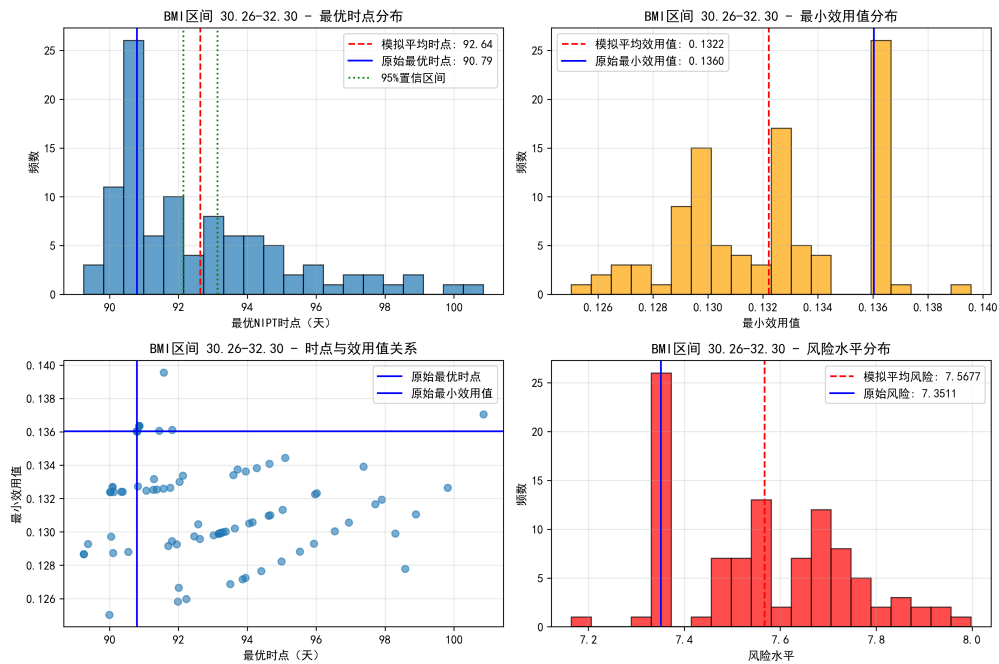


图 6: 误差模拟（30.26-32.30 组）

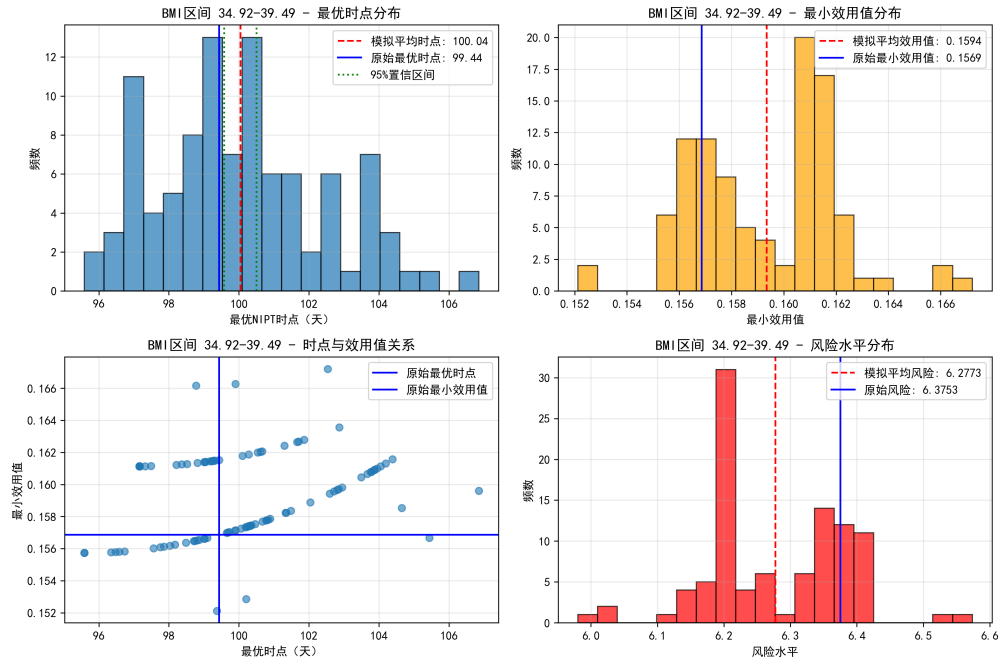


图 7: 误差模拟 (34.92-39.49 组)

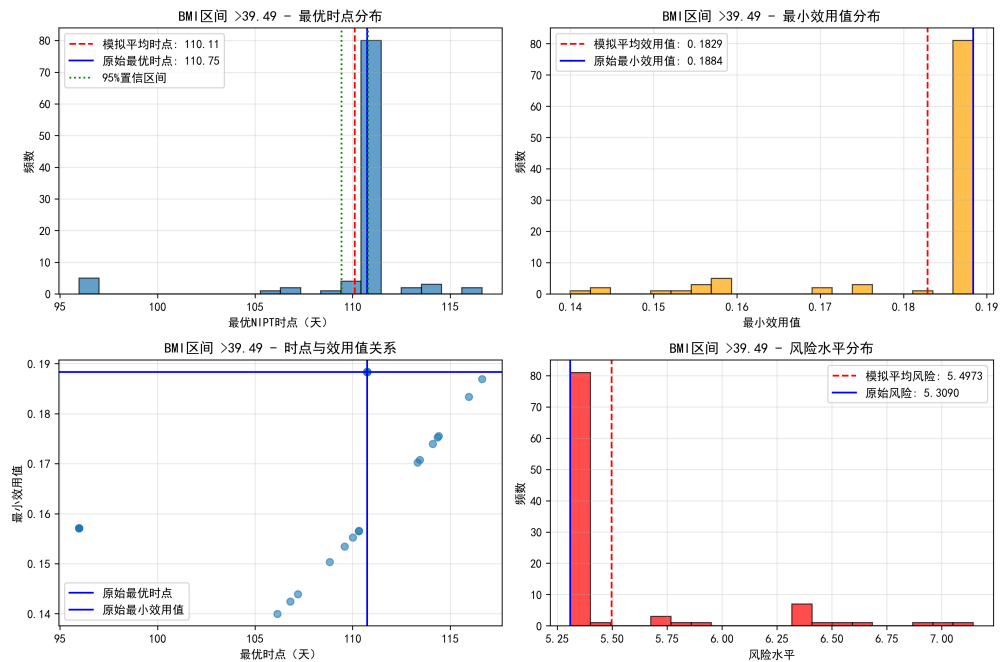


图 8: 误差模拟 (大于 39.49 组)

Listing 1: 终版非线性混合效应.R

```

1 # 加载必要的包
2 library(readxl)
3 library(nlme)

```

```

4 library(ggplot2)
5 library(lme4)
6 library(MuMIn)
7 library(mgcv) # 用于GAMM模型
8 library(merTools)
9 library(sjPlot)
10 library(gridExtra)
11 library(itsadug) # 用于GAM模型可视化
12
13 # 读取数据
14 data_path <- "D:/math_modeling/25_C/mathmodeling_2024/python_code/Q1/C(1)预处理.xlsx"
15 data <- read_excel(data_path)
16
17 # 查看数据结构
18 str(data)
19 head(data)
20
21 # 重命名列名以便使用
22 colnames(data) <- c("Subject", "BMI", "Y_concentration", "Gestational_days")
23
24 # 数据预处理
25 data$Subject <- as.factor(data$Subject)
26 data$BMI <- as.numeric(data$BMI)
27 data$Y_concentration <- as.numeric(data$Y_concentration)
28 data$Gestational_days <- as.numeric(data$Gestational_days)
29
30 # 探索性数据分析
31 exploratory_plot <- ggplot(data, aes(x = Gestational_days, y = Y_concentration, color = Subject)) +
32   geom_point() +
33   geom_smooth(method = "lm", se = FALSE) +
34   theme_minimal() +
35   labs(title = "Y染色体浓度随孕周变化", x = "孕周(天)", y = "Y染色体浓度")
36
37 print(exploratory_plot)
38 ggsave("exploratory_plot.png", width = 10, height = 6, dpi = 300)
39
40 # 1. 线性混合模型 (LMM)
41 cat("拟合线性混合模型...\n")
42 lmm_model <- lmer(Y_concentration ~ BMI + Gestational_days + (1|Subject), data = data)
43 cat("线性混合模型拟合完成\n")
44 print(summary(lmm_model))
45
46 # 2. 指数模型
47 cat("拟合指数模型...\n")
48 exp_model_formula <- function(bmi, days, a, b, c) {
49   a * exp(b * bmi + c * days)
50 }
51
52 tryCatch({
53   exp_nlme <- nlme(Y_concentration ~ exp_model_formula(BMI, Gestational_days, a, b, c),
54     data = data,
55     fixed = a + b + c ~ 1,
56     random = a ~ 1 | Subject,
57     start = list(a = 0.05, b = 0.001, c = 0.001),
58     control = nlmeControl(maxIter = 2000, pnlsTol = 1e-4, msTol = 1e-4))

```

```

59   cat("指数模型拟合成功\n")
60   print(summary(exp_nlme))
61 }, error = function(e) {
62   message("指数模型拟合失败: ", e$message)
63   exp_nlme <- NULL
64 })
65
66 # 3. Logistic生长模型
67 cat("拟合Logistic模型...\n")
68 logistic_model_formula <- function(bmi, days, a, b, c, d) {
69   a / (1 + exp(-(b * bmi + c * days - d)))
70 }
71
72 tryCatch({
73   logistic_nlme <- nlme(Y_concentration ~ logistic_model_formula(BMI, Gestational_days, a, b, c, d),
74     data = data,
75     fixed = a + b + c + d ~ 1,
76     random = a ~ 1 | Subject,
77     start = list(a = 0.1, b = 0.01, c = 0.01, d = 50),
78     control = nlmeControl(maxIter = 2000, pnlsTol = 1e-4, msTol = 1e-4))
79   cat("Logistic模型拟合成功\n")
80   print(summary(logistic_nlme))
81 }, error = function(e) {
82   message("Logistic模型拟合失败: ", e$message)
83   logistic_nlme <- NULL
84 })
85
86 # 4. Gompertz生长模型
87 cat("拟合Gompertz模型...\n")
88 gompertz_model_formula <- function(bmi, days, a, b, c, d) {
89   a * exp(-exp(-(b * bmi + c * days - d)))
90 }
91
92 tryCatch({
93   gompertz_nlme <- nlme(Y_concentration ~ gompertz_model_formula(BMI, Gestational_days, a, b, c, d),
94     data = data,
95     fixed = a + b + c + d ~ 1,
96     random = a ~ 1 | Subject,
97     start = list(a = 0.1, b = 0.01, c = 0.01, d = 50),
98     control = nlmeControl(maxIter = 2000, pnlsTol = 1e-4, msTol = 1e-4))
99   cat("Gompertz模型拟合成功\n")
100   print(summary(gompertz_nlme))
101 }, error = function(e) {
102   message("Gompertz模型拟合失败: ", e$message)
103   gompertz_nlme <- NULL
104 })
105
106 # 5. 广义加性混合模型 (GAMM)
107 cat("拟合广义加性混合模型...\n")
108 tryCatch({
109   # 使用bam函数拟合GAMM, 适用于大型数据集
110   gamm_model <- bam(Y_concentration ~ s(BMI) + s(Gestational_days) + s(Subject, bs = "re"),
111     data = data,
112     method = "REML")
113   cat("GAMM模型拟合成功\n")

```

```

114 print(summary(gamm_model))
115
116 # 绘制平滑项图
117 png("gamm_smooth_terms.png", width = 10, height = 6, units = "in", res = 300)
118 par(mfrow = c(1, 2))
119 plot(gamm_model, select = 1, main = "BMI平滑项")
120 plot(gamm_model, select = 2, main = "孕周平滑项")
121 dev.off()
122 }, error = function(e) {
123   message("GAMM模型拟合失败: ", e$message)
124   gamm_model <- NULL
125 })
126
127 # 模型比较
128 cat("开始模型比较...\n")
129 model_comparison <- data.frame(
130   Model = "线性混合模型",
131   AIC = AIC(lmm_model),
132   BIC = BIC(lmm_model),
133   LogLik = as.numeric(logLik(lmm_model))
134 )
135
136 # 添加非线性模型 (如果成功拟合)
137 if(exists("exp_nlme") && !is.null(exp_nlme)) {
138   model_comparison <- rbind(model_comparison,
139     data.frame(Model = "指数模型",
140       AIC = AIC(exp_nlme),
141       BIC = BIC(exp_nlme),
142       LogLik = logLik(exp_nlme)))
143 }
144
145 if(exists("logistic_nlme") && !is.null(logistic_nlme)) {
146   model_comparison <- rbind(model_comparison,
147     data.frame(Model = "Logistic模型",
148       AIC = AIC(logistic_nlme),
149       BIC = BIC(logistic_nlme),
150       LogLik = logLik(logistic_nlme)))
151 }
152
153 if(exists("gompertz_nlme") && !is.null(gompertz_nlme)) {
154   model_comparison <- rbind(model_comparison,
155     data.frame(Model = "Gompertz模型",
156       AIC = AIC(gompertz_nlme),
157       BIC = BIC(gompertz_nlme),
158       LogLik = logLik(gompertz_nlme)))
159 }
160
161 if(exists("gamm_model") && !is.null(gamm_model)) {
162   model_comparison <- rbind(model_comparison,
163     data.frame(Model = "GAMM模型",
164       AIC = AIC(gamm_model),
165       BIC = BIC(gamm_model),
166       LogLik = logLik(gamm_model)))
167 }
168

```

```

169 # 显示模型比较结果
170 print("模型比较结果:")
171 print(model_comparision)
172
173 # 可视化拟合结果
174 # 创建预测函数
175 predict_nlme <- function(model, newdata) {
176   if(inherits(model, "nlme")) {
177     return(predict(model, newdata = newdata, level = 0))
178   } else if(inherits(model, "lmerMod")) {
179     return(predict(model, newdata = newdata, re.form = NA))
180   } else if(inherits(model, "bam")) {
181     return(predict(model, newdata = newdata, exclude = "s(Subject)"))
182   } else {
183     return(rep(NA, nrow(newdata)))
184   }
185 }
186
187 # 生成预测数据 - 修复: 使用平均BMI值
188 pred_data <- data.frame(
189   BMI = rep(mean(data$BMI), 100), # 使用平均BMI值
190   Gestational_days = seq(min(data$Gestational_days), max(data$Gestational_days), length.out = 100),
191   Subject = rep(levels(data$Subject)[1], 100) # 使用第一个受试者
192 )
193
194 # 为每个模型生成预测
195 pred_data$lmm <- predict_nlme(lmm_model, pred_data)
196
197 if(exists("exp_nlme") && !is.null(exp_nlme)) {
198   pred_data$exp <- predict_nlme(exp_nlme, pred_data)
199 }
200
201 if(exists("logistic_nlme") && !is.null(logistic_nlme)) {
202   pred_data$logistic <- predict_nlme(logistic_nlme, pred_data)
203 }
204
205 if(exists("gompertz_nlme") && !is.null(gompertz_nlme)) {
206   pred_data$gompertz <- predict_nlme(gompertz_nlme, pred_data)
207 }
208
209 if(exists("gamm_model") && !is.null(gamm_model)) {
210   pred_data$gamm <- predict_nlme(gamm_model, pred_data)
211 }
212
213 # 创建每个模型的拟合图
214 plots <- list()
215
216 # 线性混合模型
217 plots[[1]] <- ggplot() +
218   geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha = 0.6)
219   +
220   geom_line(data = pred_data, aes(x = Gestational_days, y = lmm, color = "总体拟合"), size = 1) +
221   theme_minimal() +
222   labs(title = "线性混合模型拟合", x = "孕周(天)", y = "Y染色体浓度")

```

```

223 # 指数模型
224 if(exists("exp_nlme") && !is.null(exp_nlme)) {
225   plots[[2]] <- ggplot() +
226     geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha =
227       0.6) +
228     geom_line(data = pred_data, aes(x = Gestational_days, y = exp, color = "总体拟合"), size = 1) +
229     theme_minimal() +
230     labs(title = "指数模型拟合", x = "孕周(天)", y = "Y染色体浓度")
231 }
232
233 # Logistic模型
234 if(exists("logistic_nlme") && !is.null(logistic_nlme)) {
235   plots[[3]] <- ggplot() +
236     geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha =
237       0.6) +
238     geom_line(data = pred_data, aes(x = Gestational_days, y = logistic, color = "总体拟合"), size = 1) +
239     theme_minimal() +
240     labs(title = "Logistic模型拟合", x = "孕周(天)", y = "Y染色体浓度")
241 }
242
243 # Gompertz模型
244 if(exists("gompertz_nlme") && !is.null(gompertz_nlme)) {
245   plots[[4]] <- ggplot() +
246     geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha =
247       0.6) +
248     geom_line(data = pred_data, aes(x = Gestational_days, y = gompertz, color = "总体拟合"), size = 1) +
249     theme_minimal() +
250     labs(title = "Gompertz模型拟合", x = "孕周(天)", y = "Y染色体浓度")
251 }
252
253 # GAMM模型
254 if(exists("gamm_model") && !is.null(gamm_model)) {
255   plots[[5]] <- ggplot() +
256     geom_point(data = data, aes(x = Gestational_days, y = Y_concentration, color = Subject), alpha =
257       0.6) +
258     geom_line(data = pred_data, aes(x = Gestational_days, y = gamm, color = "总体拟合"), size = 1) +
259     theme_minimal() +
260     labs(title = "GAMM模型拟合", x = "孕周(天)", y = "Y染色体浓度")
261 }
262
263 # 保存所有图表
264 for(i in seq_along(plots)) {
265   ggsave(paste0("model_fit_", i, ".png"), plots[[i]], width = 10, height = 6, dpi = 300)
266 }
267
268 # 创建模型比较图
269 model_comp_plot <- ggplot(model_comparison, aes(x = reorder(Model, AIC), y = AIC, fill = Model)) +
270   geom_bar(stat = "identity") +
271   theme_minimal() +
272   labs(title = "模型比较 - AIC值", x = "模型", y = "AIC值") +
273   theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
274   scale_fill_brewer(palette = "Set2")
275
276 ggsave("model_comparison_aic.png", model_comp_plot, width = 10, height = 6, dpi = 300)
277

```

```

274 # 残差分析
275 residual_plots <- list()
276
277 # 线性混合模型残差
278 residual_plots[[1]] <- ggplot(data, aes(x = fitted(lmm_model), y = resid(lmm_model))) +
279   geom_point(alpha = 0.6) +
280   geom_hline(yintercept = 0, linetype = "dashed") +
281   theme_minimal() +
282   labs(title = "线性混合模型残差图", x = "拟合值", y = "残差")
283
284 # 指数模型残差
285 if(exists("exp_nlme") && !is.null(exp_nlme)) {
286   residual_plots[[2]] <- ggplot(data, aes(x = fitted(exp_nlme), y = resid(exp_nlme))) +
287     geom_point(alpha = 0.6) +
288     geom_hline(yintercept = 0, linetype = "dashed") +
289     theme_minimal() +
290     labs(title = "指数模型残差图", x = "拟合值", y = "残差")
291 }
292
293 # GAMM模型残差
294 if(exists("gamm_model") && !is.null(gamm_model)) {
295   residual_plots[[3]] <- ggplot(data, aes(x = fitted(gamm_model), y = resid(gamm_model))) +
296     geom_point(alpha = 0.6) +
297     geom_hline(yintercept = 0, linetype = "dashed") +
298     theme_minimal() +
299     labs(title = "GAMM模型残差图", x = "拟合值", y = "残差")
300 }
301
302 # 保存残差图
303 for(i in seq_along(residual_plots)) {
304   ggsave(paste0("residual_plot_", i, ".png"), residual_plots[[i]], width = 10, height = 6, dpi = 300)
305 }
306
307 # 输出模型摘要
308 sink("model_summaries.txt")
309 cat("线性混合模型摘要:\n")
310 print(summary(lmm_model))
311
312 if(exists("exp_nlme") && !is.null(exp_nlme)) {
313   cat("\n指数模型摘要:\n")
314   print(summary(exp_nlme))
315 }
316
317 if(exists("logistic_nlme") && !is.null(logistic_nlme)) {
318   cat("\nLogistic模型摘要:\n")
319   print(summary(logistic_nlme))
320 }
321
322 if(exists("gompertz_nlme") && !is.null(gompertz_nlme)) {
323   cat("\nGompertz模型摘要:\n")
324   print(summary(gompertz_nlme))
325 }
326
327 if(exists("gamm_model") && !is.null(gamm_model)) {
328   cat("\nGAMM模型摘要:\n")

```

```

329     print(summary(gamm_model))
330 }
331
332 cat("\n模型比较:\n")
333 print(model_comparison)
334 sink()
335
336 # 显示最佳模型
337 if(nrow(model_comparison) > 0) {
338     best_model_idx <- which.min(model_comparison$AIC)
339     cat("根据AIC准则, 最佳模型是:", model_comparison$Model[best_model_idx], "\n")
340
341     # 绘制最佳模型拟合图
342     if(model_comparison$Model[best_model_idx] == "线性混合模型") {
343         best_plot <- plots[[1]]
344     } else if(model_comparison$Model[best_model_idx] == "指数模型") {
345         best_plot <- plots[[2]]
346     } else if(model_comparison$Model[best_model_idx] == "Logistic模型") {
347         best_plot <- plots[[3]]
348     } else if(model_comparison$Model[best_model_idx] == "Gompertz模型") {
349         best_plot <- plots[[4]]
350     } else if(model_comparison$Model[best_model_idx] == "GAMM模型") {
351         best_plot <- plots[[5]]
352     }
353
354     ggsave("best_model_fit.png", best_plot, width = 10, height = 6, dpi = 300)
355 }
356
357 # 保存工作空间
358 save.image("mixed_model_analysis.RData")
359
360 cat("分析完成! 所有结果已保存到当前工作目录。 \n")

```

Listing 2: C(2)(生存函数).py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from lifelines import KaplanMeierFitter
5  from scipy.optimize import minimize_scalar
6
7  # 设置中文字体支持
8  plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "DejaVu Sans"]
9  plt.rcParams["axes.unicode_minus"] = False
10
11
12 def analyze_with_dynamic_utility():
13     # 定义风险函数
14     def get_rearly(t):
15         return 2.0 * np.exp(-t / 50)
16
17     def get_rlate(t):
18         if t < 84:
19             return 0.1
20         elif t <= 189:

```

```

21         normalized_t = (t - 84) / (189 - 84)
22         return 0.1 + 0.9 * (normalized_t**2)
23     else:
24         return 1.0
25
26 # 计算效用的函数
27 def calculate_utility(t, survival_func):
28     if t < 0:
29         return float("inf")
30     closest_time_idx = np.abs(survival_func.index - t).argmin()
31     actual_time = survival_func.index[closest_time_idx]
32     p_t = 1 - survival_func.loc[actual_time, "s_t"]
33     return (1 - p_t) * get_rearly(t) + p_t * get_rlate(t)
34
35 # 数据读取与预处理
36 df_middle = pd.read_excel("./python_code/bmi_Y_middle_result.xlsx")
37 df_cannot_test = pd.read_excel("./python_code/bmi_Y_cannot_test_result.xlsx")
38 df_always_can_test = pd.read_excel(
39     "./python_code/bmi_Y_always_can_test_result.xlsx"
40 )
41
42 # 合并数据并添加类别标识
43 df_all = pd.concat(
44     [
45         df_middle.assign(category="middle"),
46         df_cannot_test.assign(category="cannot"),
47         df_always_can_test.assign(category="always_can"),
48     ],
49     ignore_index=True,
50 )
51
52 df_all["duration"] = 0.0
53 df_all["event_observed"] = 0
54
55 # 根据不同类别设置duration和event_observed
56 df_all.loc[df_all["category"] == "cannot", "duration"] = df_all["最晚不达标天数"]
57 df_all.loc[df_all["category"] == "cannot", "event_observed"] = 0
58
59 df_all.loc[df_all["category"] == "middle", "duration"] = df_all["预测达标天数"]
60 df_all.loc[df_all["category"] == "middle", "event_observed"] = 1
61
62 df_all.loc[df_all["category"] == "always_can", "duration"] = 0.1
63 df_all.loc[df_all["category"] == "always_can", "event_observed"] = 1
64
65 # BMI分类处理
66 def categorize_bmi(bmi):
67     if bmi < 30.26:
68         return "<30.26"
69     elif 30.26 <= bmi < 32.30:
70         return "30.26-32.30"
71     elif 32.30 <= bmi < 34.92:
72         return "32.30-34.92"
73     elif 34.92 <= bmi < 39.49:
74         return "34.92-39.49"
75     else:

```

```

76         return ">39.49"
77
78     df_all["bmi_category"] = df_all["BMI"].apply(categorize_bmi)
79
80     bmi_categories = ["<30.26", "30.26-32.30", "32.30-34.92", "34.92-39.49", ">39.49"]
81
82     results_table = []
83
84     # 按BMI分组进行分析
85     for bmi_cat in bmi_categories:
86
87         df_bmi = df_all[df_all["bmi_category"] == bmi_cat].copy()
88
89         if df_bmi.empty:
90             results_table.append(
91                 {
92                     "BMI分组": bmi_cat,
93                     "最优时点(天)": "无数据",
94                     "最小效用值": "无数据",
95                     "风险水平": "无数据",
96                     "样本量": 0,
97                 }
98             )
99             continue
100
101         print(f"--- 正在分析 BMI 区间: {bmi_cat} ---")
102
103         # 生存分析
104         # 使用Kaplan-Meier估计生存函数
105         kmf = KaplanMeierFitter()
106         kmf.fit(durations=df_bmi["duration"], event_observed=df_bmi["event_observed"])
107
108         s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
109
110         # 优化效用函数
111         # 定义目标函数
112         def objective(t):
113             return calculate_utility(t, s_t)
114
115         # 寻找最小化效用函数的时间点
116         result = minimize_scalar(
117             objective, bounds=(0, df_bmi["duration"].max()), method="bounded"
118         )
119
120         # 提取优化结果
121         optimal_time = result.x
122         optimal_utility = result.fun
123         risk_level = 1 / optimal_utility if optimal_utility > 0 else float("inf")
124         sample_size = len(df_bmi)
125
126         print(f"\n最优预测时间点分析:")
127         print(f" - 最优时间点: {optimal_time:.1f} 天")
128         print(f" - 最小效用值: {optimal_utility:.4f}")
129         print(f" - 风险水平: {risk_level:.4f}")
130         print(f" - 样本量: {sample_size}")

```

```

131
132
133 # 可视化分析结果
134 plt.figure(figsize=(14, 8))
135 ax = plt.gca()
136
137 # 绘制累积达标函数
138 (1 - kmf.survival_function_).plot(
139     ax=ax,
140     label=f"累积达标函数 F(t) ({bmi_cat})",
141     color="seagreen",
142     linewidth=2.5,
143 )
144
145 # 计算并绘制效用函数
146 time_points = np.linspace(0, df_bmi["duration"].max(), 200)
147 utilities = [calculate_utility(t, s_t) for t in time_points]
148 ax_twin = ax.twinx()
149 ax_twin.plot(
150     time_points, utilities, "--", color="red", label="效用函数 E(t)", alpha=0.6
151 )
152
153 ax_twin.axvline(x=optimal_time, color="purple", linestyle="--", linewidth=1)
154 ax_twin.text(
155     optimal_time + 2,
156     min(utilities) + 0.1,
157     f"最优时间点: {optimal_time:.1f}天\n效用值: {optimal_utility:.3f}",
158     color="purple",
159     fontsize=10,
160 )
161
162 ax.axvspan(0, 84, facecolor="green", alpha=0.15, label="早期阶段 (<84天)")
163 ax.axvspan(84, 189, facecolor="orange", alpha=0.15, label="过渡阶段 (84-189天)")
164 ax.axvspan(
165     189,
166     df_all["duration"].max() + 10,
167     facecolor="red",
168     alpha=0.15,
169     label="晚期阶段 (>189天)",
170 )
171
172 ax.set_title(f"BMI区间 {bmi_cat} 达标情况与效用分析", fontsize=16)
173 ax.set_xlabel("天数", fontsize=12)
174 ax.set_ylabel("累积达标比例", fontsize=12)
175 ax_twin.set_ylabel("效用值", fontsize=12)
176
177 lines1, labels1 = ax.get_legend_handles_labels()
178 lines2, labels2 = ax_twin.get_legend_handles_labels()
179 ax.legend(lines1 + lines2, labels1 + labels2, loc="upper left")
180
181 ax.grid(True, alpha=0.3)
182 ax.set_ylim(0, 1.05)
183 ax_twin.set_ylim(0, max(utilities) * 1.2)
184 ax.set_xlim(-5, df_all["duration"].max() + 10)
185

```

```

186     filename = f'./python_code/BMI_{bmi_cat.replace("<", "lt").replace(">", "gt").replace("-", "_")}_
        _utility_analysis.png'
187     plt.savefig(filename, dpi=300, bbox_inches="tight")
188     plt.close()
189
190     results_table.append(
191         {
192             "BMI分组": bmi_cat,
193             "最优时点(天)": f"{optimal_time:.1f}",
194             "最小效用值": f"{optimal_utility:.4f}",
195             "风险水平": f"{risk_level:.4f}",
196             "样本量": sample_size,
197         }
198     )
199
200     print(f"\n图表已保存至: {filename}")
201     print("\n" + "=" * 50 + "\n")
202
203     # 结果输出与保存
204     print("\n\n=== 各BMI分组NIPT时点计算结果 ===")
205     print("BMI分组\t\t最优时点(天)\t\t最小效用值\t\t风险水平\t\t样本量")
206     print("-" * 70)
207
208     for result in results_table:
209         print(
210             f"{result['BMI分组']}\t\t{result['最优时点(天)']}\t\t{result['最小效用值']}\t\t{result['风险水
                平']}\t\t{result['样本量']}"
211         )
212
213     # 保存结果到Excel
214     results_df = pd.DataFrame(results_table)
215     results_df.to_excel("./python_code/NIPT_optimal_times_results.xlsx", index=False)
216     print(f"\n结果表格已保存至: ./python_code/NIPT_optimal_times_results.xlsx")
217
218     return results_df
219
220 # 执行
221 results = analyze_with_dynamic_utility()

```

Listing 3: C(2) 测试误差.py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from lifelines import KaplanMeierFitter
5  from scipy.optimize import minimize_scalar
6  from scipy.stats import norm
7  import re
8
9  # 设置中文字体支持
10 plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "DejaVu Sans"]
11 plt.rcParams["axes.unicode_minus"] = False
12
13
14 def analyze_with_error_simulation(num_simulations=100, error_std=0.01):

```

```

15 # 定义效用函数
16 def get_rearly(t):
17     return 2.0 * np.exp(-t / 50)
18
19 def get_rlate(t):
20     if t < 84:
21         return 0.1
22     elif t <= 189:
23         normalized_t = (t - 84) / (189 - 84)
24         return 0.1 + 0.9 * (normalized_t**2)
25     else:
26         return 1.0
27
28 def calculate_utility(t, survival_func):
29     if t < 0:
30         return float("inf")
31     closest_time_idx = np.abs(survival_func.index - t).argmin()
32     actual_time = survival_func.index[closest_time_idx]
33     p_t = 1 - survival_func.loc[actual_time, "s_t"]
34     return (1 - p_t) * get_rearly(t) + p_t * get_rlate(t)
35
36 # 辅助函数：清理文件名中的非法字符
37 def clean_filename(name):
38     return re.sub(r'[\<>:"/\|?*]', "_", name)
39
40 # 数据加载与预处理
41 df_middle = pd.read_excel("./python_code/bmi_Y_middle_result.xlsx")
42 df_cannot_test = pd.read_excel("./python_code/bmi_Y_cannot_test_result.xlsx")
43 df_always_can_test = pd.read_excel(
44     "./python_code/bmi_Y_always_can_test_result.xlsx"
45 )
46
47 # 合并数据并添加类别标识
48 df_all = pd.concat(
49     [
50         df_middle.assign(category="middle"),
51         df_cannot_test.assign(category="cannot"),
52         df_always_can_test.assign(category="always_can"),
53     ],
54     ignore_index=True,
55 )
56
57 # 初始化duration和event_observed列
58 df_all["duration"] = 0.0
59 df_all["event_observed"] = 0
60
61 # 根据不同类别设置duration和event_observed
62 df_all.loc[df_all["category"] == "cannot", "duration"] = df_all["最晚不达标天数"]
63 df_all.loc[df_all["category"] == "cannot", "event_observed"] = 0
64
65 df_all.loc[df_all["category"] == "middle", "duration"] = df_all["预测达标天数"]
66 df_all.loc[df_all["category"] == "middle", "event_observed"] = 1
67
68 df_all.loc[df_all["category"] == "always_can", "duration"] = 0.1
69 df_all.loc[df_all["category"] == "always_can", "event_observed"] = 1

```

```

70
71 # BMI分类处理
72 def categorize_bmi(bmi):
73     if bmi < 30.26:
74         return "<30.26"
75     elif 30.26 <= bmi < 32.30:
76         return "30.26-32.30"
77     elif 32.30 <= bmi < 34.92:
78         return "32.30-34.92"
79     elif 34.92 <= bmi < 39.49:
80         return "34.92-39.49"
81     else:
82         return ">39.49"
83
84 df_all["bmi_category"] = df_all["BMI"].apply(categorize_bmi)
85 bmi_categories = ["<30.26", "30.26-32.30", "32.30-34.92", "34.92-39.49", ">39.49"]
86
87 # 初始化结果存储字典
88 results = {
89     bmi_cat: {"times": [], "utilities": [], "risks": []}
90     for bmi_cat in bmi_categories
91 }
92
93 # 原始数据分析
94 original_results = {}
95 for bmi_cat in bmi_categories:
96     df_bmi = df_all[df_all["bmi_category"] == bmi_cat].copy()
97     if df_bmi.empty:
98         continue
99
100 # 使用Kaplan-Meier估计生存函数
101 kmf = KaplanMeierFitter()
102 kmf.fit(durations=df_bmi["duration"], event_observed=df_bmi["event_observed"])
103 s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
104
105 # 优化效用函数
106 def objective(t):
107     return calculate_utility(t, s_t)
108
109 result = minimize_scalar(
110     objective, bounds=(0, df_bmi["duration"].max()), method="bounded"
111 )
112
113 # 提取原始结果
114 original_time = result.x
115 original_utility = result.fun
116 original_risk = 1 / original_utility if original_utility > 0 else float("inf")
117
118 original_results[bmi_cat] = {
119     "time": original_time,
120     "utility": original_utility,
121     "risk": original_risk,
122 }
123
124 # 打印原始结果

```

```

125     print(f"BMI 区间 {bmi_cat} 原始结果:")
126     print(f" 最优时点: {original_time:.2f} 天")
127     print(f" 最小效用值: {original_utility:.4f}")
128     print(f" 风险水平: {original_risk:.4f}")
129
130 # 误差模拟分析
131 for simulation in range(num_simulations):
132     print(f"正在进行第 {simulation+1} 次模拟...")
133
134     # 创建模拟数据副本
135     df_simulated = df_all.copy()
136     np.random.seed(simulation)
137
138     # 为不同类别数据添加误差
139     mask_middle = df_simulated["category"] == "middle"
140     mask_always = df_simulated["category"] == "always_can"
141
142     if sum(mask_middle) > 0:
143         error_middle = np.random.normal(
144             0,
145             error_std * df_simulated.loc[mask_middle, "预测达标天数"].mean(),
146             size=sum(mask_middle),
147         )
148         df_simulated.loc[mask_middle, "duration"] += error_middle
149
150     if sum(mask_always) > 0:
151         error_always = np.random.normal(
152             0,
153             error_std * df_simulated.loc[mask_always, "最早达标天数"].mean(),
154             size=sum(mask_always),
155         )
156         df_simulated.loc[mask_always, "duration"] += error_always
157
158     mask_cannot = df_simulated["category"] == "cannot"
159     if sum(mask_cannot) > 0:
160         error_cannot = np.random.normal(
161             0,
162             error_std * df_simulated.loc[mask_cannot, "最晚不达标天数"].mean(),
163             size=sum(mask_cannot),
164         )
165         df_simulated.loc[mask_cannot, "duration"] += error_cannot
166
167     # 确保duration为正数
168     df_simulated["duration"] = df_simulated["duration"].clip(lower=0.1)
169
170     # 对每个BMI分类进行分析
171     for bmi_cat in bmi_categories:
172         df_bmi = df_simulated[df_simulated["bmi_category"] == bmi_cat].copy()
173         if df_bmi.empty:
174             continue
175
176         # 使用Kaplan-Meier估计生存函数
177         kmf = KaplanMeierFitter()
178         kmf.fit(
179             durations=df_bmi["duration"], event_observed=df_bmi["event_observed"]

```

```

180     )
181     s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
182
183     # 优化效用函数
184     def objective(t):
185         return calculate_utility(t, s_t)
186
187     result = minimize_scalar(
188         objective, bounds=(0, df_bmi["duration"].max()), method="bounded"
189     )
190
191     # 存储模拟结果
192     optimal_time = result.x
193     optimal_utility = result.fun
194     risk = 1 / optimal_utility if optimal_utility > 0 else float("inf")
195
196     results[bmi_cat]["times"].append(optimal_time)
197     results[bmi_cat]["utilities"].append(optimal_utility)
198     results[bmi_cat]["risks"].append(risk)
199
200     # 结果分析与可视化
201     print("\n=== 检测误差影响分析 ===")
202     for bmi_cat in bmi_categories:
203         if results[bmi_cat]["times"]:
204             # 计算统计量
205             times = np.array(results[bmi_cat]["times"])
206             utilities = np.array(results[bmi_cat]["utilities"])
207             risks = np.array(results[bmi_cat]["risks"])
208
209             mean_time = times.mean()
210             std_time = times.std()
211             time_confidence_interval = norm.interval(
212                 0.95, loc=mean_time, scale=std_time / np.sqrt(len(times))
213             )
214
215             mean_utility = utilities.mean()
216             std_utility = utilities.std()
217
218             mean_risk = risks.mean()
219             std_risk = risks.std()
220
221             # 获取原始结果
222             original_time = original_results.get(bmi_cat, {}).get("time", 0)
223             original_utility = original_results.get(bmi_cat, {}).get("utility", 0)
224             original_risk = original_results.get(bmi_cat, {}).get("risk", 0)
225
226             # 打印分析结果
227             print(f"BMI 区间 {bmi_cat}:")
228             print(f"    原始最优时点: {original_time:.2f} 天")
229             print(f"    模拟最优时点均值: {mean_time:.2f} 天")
230             print(f"    标准差: {std_time:.2f} 天")
231             print(
232                 f"    95%置信区间: ({time_confidence_interval[0]:.2f}, {time_confidence_interval[1]:.2f})"
233                 f"    天"
234             )

```

```

234     print(f" 原始最小效用值: {original_utility:.4f}")
235     print(f" 模拟最小效用值均值: {mean_utility:.4f}")
236     print(f" 效用值标准差: {std_utility:.4f}")
237     print(f" 原始风险水平: {original_risk:.4f}")
238     print(f" 模拟风险水平均值: {mean_risk:.4f}")
239     print(f" 风险水平标准差: {std_risk:.4f}")
240
241     # 创建可视化图表
242     plt.figure(figsize=(12, 8))
243
244     # 最优时点分布
245     plt.subplot(2, 2, 1)
246     plt.hist(times, bins=20, alpha=0.7, edgecolor="black")
247     plt.axvline(
248         mean_time,
249         color="r",
250         linestyle="--",
251         label=f"模拟平均时点: {mean_time:.2f}",
252     )
253     plt.axvline(
254         original_time,
255         color="b",
256         linestyle="-",
257         label=f"原始最优时点: {original_time:.2f}",
258     )
259     plt.axvline(
260         time_confidence_interval[0],
261         color="g",
262         linestyle=":",
263         label="95%置信区间",
264     )
265     plt.axvline(time_confidence_interval[1], color="g", linestyle=":")
266     plt.xlabel("最优NIPT时点 (天)")
267     plt.ylabel("频数")
268     plt.title(f"BMI 区间 {bmi_cat} - 最优时点分布")
269     plt.legend()
270     plt.grid(True, alpha=0.3)
271
272     # 最小效用值分布
273     plt.subplot(2, 2, 2)
274     plt.hist(utilities, bins=20, alpha=0.7, edgecolor="black", color="orange")
275     plt.axvline(
276         mean_utility,
277         color="r",
278         linestyle="--",
279         label=f"模拟平均效用值: {mean_utility:.4f}",
280     )
281     plt.axvline(
282         original_utility,
283         color="b",
284         linestyle="-",
285         label=f"原始最小效用值: {original_utility:.4f}",
286     )
287     plt.xlabel("最小效用值")
288     plt.ylabel("频数")

```

```

289 plt.title(f"BMI 区间 {bmi_cat} - 最小效用值分布")
290 plt.legend()
291 plt.grid(True, alpha=0.3)
292
293 # 时点与效用值关系
294 plt.subplot(2, 2, 3)
295 plt.scatter(times, utilities, alpha=0.6)
296 plt.axvline(original_time, color="b", linestyle="--", label="原始最优时点")
297 plt.axhline(
298     original_utility, color="b", linestyle="--", label="原始最小效用值"
299 )
300 plt.xlabel("最优时点 (天)")
301 plt.ylabel("最小效用值")
302 plt.title(f"BMI 区间 {bmi_cat} - 时点与效用值关系")
303 plt.legend()
304 plt.grid(True, alpha=0.3)
305
306 # 风险水平分布
307 plt.subplot(2, 2, 4)
308 plt.hist(risks, bins=20, alpha=0.7, edgecolor="black", color="red")
309 plt.axvline(
310     mean_risk,
311     color="r",
312     linestyle="--",
313     label=f"模拟平均风险: {mean_risk:.4f}",
314 )
315 plt.axvline(
316     original_risk,
317     color="b",
318     linestyle="--",
319     label=f"原始风险: {original_risk:.4f}",
320 )
321 plt.xlabel("风险水平")
322 plt.ylabel("频数")
323 plt.title(f"BMI 区间 {bmi_cat} - 风险水平分布")
324 plt.legend()
325 plt.grid(True, alpha=0.3)
326
327 # 保存图表
328 plt.tight_layout()
329 safe_bmi_cat = clean_filename(bmi_cat)
330 filename = f"./python_code/error_analysis_BMI_{safe_bmi_cat}.png"
331 plt.savefig(filename, dpi=300, bbox_inches="tight")
332 plt.close()
333
334 # 保存数据到CSV
335 data_df = pd.DataFrame({"time": times, "utility": utilities, "risk": risks})
336 csv_filename = f"./python_code/error_analysis_BMI_{safe_bmi_cat}.csv"
337 data_df.to_csv(csv_filename, index=False)
338
339 print(f" 图表已保存至: {filename}")
340 print(f" 数据已保存至: {csv_filename}")
341 else:
342     print(f"BMI 区间 {bmi_cat}: 无数据")
343

```

```

344
345 # 执行误差模拟分析
346 analyze_with_error_simulation(num_simulations=100, error_std=0.05)

```

Listing 4: C(2) 分类画图.py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import matplotlib.font_manager as fm
5  from matplotlib import rcParams
6  from scipy import stats
7  import re
8
9  # 设置中文字体支持
10 plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "DejaVu Sans"]
11 plt.rcParams["axes.unicode_minus"] = False
12
13 # 生成每个BMI区间的堆叠分布图
14 def generate_stacked_bmi_mixed_charts():
15     # 加载数据文件
16     df_middle = pd.read_excel("./python_code/bmi_Y_middle_result.xlsx")
17     df_cannot_test = pd.read_excel("./python_code/bmi_Y_cannot_test_result.xlsx")
18     df_always_can_test = pd.read_excel(
19         "./python_code/bmi_Y_always_can_test_result.xlsx"
20     )
21
22     # 添加类别标识
23     df_middle["category"] = "middle"
24     df_cannot_test["category"] = "cannot"
25     df_always_can_test["category"] = "always_can"
26
27     # 合并所有数据
28     df_all = pd.concat(
29         [df_middle, df_cannot_test, df_always_can_test], ignore_index=True
30     )
31
32     # 定义BMI分类函数
33     def categorize_bmi(bmi):
34         if bmi < 30.26:
35             return "<30.26"
36         elif 30.26 <= bmi < 32.30:
37             return "30.26-32.30"
38         elif 32.30 <= bmi < 34.92:
39             return "32.30-34.92"
40         elif 34.92 <= bmi < 39.49:
41             return "34.92-39.49"
42         else:
43             return ">39.49"
44
45     df_all["bmi_category"] = df_all["BMI"].apply(categorize_bmi)
46     bmi_categories = ["<30.26", "30.26-32.30", "32.30-34.92", "34.92-39.49", ">39.49"]
47
48     # 定义类别、颜色和标签
49     categories = ["cannot", "middle", "always_can"]

```

```

50 colors = {"cannot": "lightcoral", "middle": "gold", "always_can": "seagreen"}
51 labels = {"cannot": "不能达标", "middle": "中间达标", "always_can": "始终达标"}
52
53 # 为每个BMI区间生成图表
54 for bmi_cat in bmi_categories:
55     df_bmi = df_all[df_all["bmi_category"] == bmi_cat]
56
57     if df_bmi.empty:
58         print(f"警告: BMI区间 {bmi_cat} 没有数据")
59         continue
60
61     # 创建图表
62     fig, ax1 = plt.subplots(figsize=(12, 8))
63
64     # 收集所有天数数据
65     all_days = []
66     days_data = {}
67
68     # 处理每个类别的数据
69     for category in categories:
70         df_category = df_bmi[df_bmi["category"] == category]
71
72         if not df_category.empty:
73             if category == "cannot":
74                 days = df_category["最晚不达标天数"].tolist()
75             elif category == "middle":
76                 days = df_category["预测达标天数"].tolist()
77             else:
78                 days = df_category["最早达标天数"].tolist()
79
80             days_data[category] = days
81             all_days.extend(days)
82
83     # 准备堆叠数据
84     stacked_data = []
85     stacked_labels = []
86     stacked_colors = []
87
88     if "cannot" in days_data:
89         stacked_data.append(days_data["cannot"])
90         stacked_labels.append(labels["cannot"])
91         stacked_colors.append(colors["cannot"])
92
93     if "middle" in days_data:
94         stacked_data.append(days_data["middle"])
95         stacked_labels.append(labels["middle"])
96         stacked_colors.append(colors["middle"])
97
98     if "always_can" in days_data:
99         stacked_data.append(days_data["always_can"])
100         stacked_labels.append(labels["always_can"])
101         stacked_colors.append(colors["always_can"])
102
103     # 绘制堆叠直方图
104     if stacked_data:

```

```

105         if len(all_days) > 0:
106             data_range = max(all_days) - min(all_days)
107             bins_count = min(100, max(50, int(data_range / 3)))
108         else:
109             bins_count = 50
110
111         n, bins, patches = ax1.hist(
112             stacked_data,
113             bins=bins_count,
114             stacked=True,
115             color=stacked_colors,
116             label=stacked_labels,
117             alpha=0.95,
118             edgecolor="black",
119             linewidth=0.1,
120         )
121
122         # 设置图表属性
123         ax1.set_xlabel("天数", fontsize=12)
124         ax1.set_ylabel("人数", fontsize=12)
125         ax1.set_title(
126             f"BMI区间 {bmi_cat} 的孕妇Y染色体达标情况分布（堆叠图）", fontsize=14
127         )
128         ax1.legend(fontsize=10)
129         ax1.grid(True, alpha=0.3)
130
131         # 添加概率密度曲线
132         if len(all_days) > 1:
133             ax2 = ax1.twinx()
134
135             try:
136                 kde = stats.gaussian_kde(all_days)
137                 x_range = np.linspace(min(all_days), max(all_days), 1000)
138                 kde_values = kde(x_range)
139
140                 ax2.plot(
141                     x_range,
142                     kde_values,
143                     color="dodgerblue",
144                     linewidth=2,
145                     linestyle="-",
146                     label="总体概率密度",
147                 )
148
149                 ax2.set_ylabel("概率密度", fontsize=12)
150                 ax2.legend(bbox_to_anchor=(1.0, 0.9), fontsize=10)
151             except Exception as e:
152                 print(f"BMI区间 {bmi_cat} 计算概率密度时出错: {e}")
153
154         # 保存图表
155         plt.tight_layout()
156         filename = f'./python_code/BMI_{bmi_cat.replace("<", "lt").replace(">", "gt").replace("-", "_")}_stacked_distribution.png'
157         plt.savefig(filename, dpi=300, bbox_inches="tight")
158         plt.close()

```

```

159
160     # 打印统计信息
161     print(f"BMI 区间 {bmi_cat} 统计:")
162     total_count = len(df_bmi)
163     for category in categories:
164         count = len(df_bmi[df_bmi["category"] == category])
165         percentage = (count / total_count) * 100 if total_count > 0 else 0
166         print(f"    {labels[category]}: {count} 人 ({percentage:.1f}%)")
167     print(f"    总计: {total_count} 人")
168     print()
169
170 # 生成总体堆叠柱状图
171 def generate_overall_stacked_bmi_chart():
172     # 加载数据文件
173     df_middle = pd.read_excel("./python_code/bmi_Y_middle_result.xlsx")
174     df_cannot_test = pd.read_excel("./python_code/bmi_Y_cannot_test_result.xlsx")
175     df_always_can_test = pd.read_excel(
176         "./python_code/bmi_Y_always_can_test_result.xlsx"
177     )
178
179     # 添加类别标识
180     df_middle["category"] = "middle"
181     df_cannot_test["category"] = "cannot"
182     df_always_can_test["category"] = "always_can"
183
184     # 合并所有数据
185     df_all = pd.concat(
186         [df_middle, df_cannot_test, df_always_can_test], ignore_index=True
187     )
188
189     # 定义BMI分类函数
190     def categorize_bmi(bmi):
191         if bmi < 30.26:
192             return "<30.26"
193         elif 30.26 <= bmi < 32.30:
194             return "30.26-32.30"
195         elif 32.30 <= bmi < 34.92:
196             return "32.30-34.92"
197         elif 34.92 <= bmi < 39.49:
198             return "34.92-39.49"
199         else:
200             return ">39.49"
201
202     # 应用BMI分类
203     df_all["bmi_category"] = df_all["BMI"].apply(categorize_bmi)
204     bmi_categories = ["<30.26", "30.26-32.30", "32.30-34.92", "34.92-39.49", ">39.49"]
205
206     # 定义类别、颜色和标签
207     categories = ["cannot", "middle", "always_can"]
208     colors = {"cannot": "red", "middle": "yellow", "always_can": "green"}
209     labels = {"cannot": "不能达标", "middle": "中间达标", "always_can": "始终达标"}
210
211     # 创建图表
212     fig, ax1 = plt.subplots(figsize=(15, 8))
213

```

```

214 # 按类别收集数据
215 all_data_by_category = {cat: [] for cat in categories}
216
217 # 处理每个BMI区间的数据
218 for bmi_cat in bmi_categories:
219     df_bmi = df_all[df_all["bmi_category"] == bmi_cat]
220
221     for category in categories:
222         df_category = df_bmi[df_bmi["category"] == category]
223
224         if not df_category.empty:
225             if category == "cannot":
226                 days = df_category["最晚不达标天数"].tolist()
227             elif category == "middle":
228                 days = df_category["预测达标天数"].tolist()
229             else:
230                 days = df_category["最早达标天数"].tolist()
231
232             labeled_days = [(day, bmi_cat) for day in days]
233             all_data_by_category[category].extend(labeled_days)
234
235 # 准备柱状图数据
236 x_positions = np.arange(len(bmi_categories))
237 bar_width = 0.6
238 bottom_values = np.zeros(len(bmi_categories))
239
240 # 绘制堆叠柱状图
241 for category in categories:
242     heights = []
243     for bmi_cat in bmi_categories:
244         count = len([x for x in all_data_by_category[category] if x[1] == bmi_cat])
245         heights.append(count)
246
247     ax1.bar(
248         x_positions,
249         heights,
250         bar_width,
251         bottom=bottom_values,
252         label=labels[category],
253         color=colors[category],
254         alpha=0.7,
255         edgecolor="black",
256         linewidth=0.5,
257     )
258
259     bottom_values = np.add(bottom_values, heights)
260
261 # 设置图表属性
262 ax1.set_xlabel("BMI 区间", fontsize=12)
263 ax1.set_ylabel("人数", fontsize=12)
264 ax1.set_title("各BMI区间孕妇Y染色体达标情况分布（堆叠柱状图）", fontsize=14)
265 ax1.set_xticks(x_positions)
266 ax1.set_xticklabels(bmi_categories)
267 ax1.legend(fontsize=10)
268 ax1.grid(True, alpha=0.3, axis="y")

```

```

269
270 # 保存图表
271 plt.tight_layout()
272 plt.savefig(
273     "./python_code/BMI_all_intervals_stacked_distribution_new.png",
274     dpi=300,
275     bbox_inches="tight",
276 )
277 plt.close()
278
279 print("总体堆叠柱状图已生成")
280
281
282 if __name__ == "__main__":
283     generate_overall_stacked_bmi_chart()
284     generate_stacked_bmi_mixed_charts()
285
286     print("\n图表生成完成!")

```

Listing 5: C(2) 聚类.py

```

1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from sklearn.cluster import KMeans
5  from sklearn.preprocessing import StandardScaler
6  from sklearn.metrics import (
7      silhouette_score,
8      davies_bouldin_score,
9      calinski_harabasz_score,
10 )
11
12 # 设置中文字体支持
13 plt.rcParams["font.sans-serif"] = ["SimHei"]
14 plt.rcParams["axes.unicode_minus"] = False
15
16
17 # 执行K-means聚类分析
18 def perform_clustering(df, n_clusters=5):
19     X = df[["BMI"]].values
20     scaler = StandardScaler()
21     X_scaled = scaler.fit_transform(X)
22
23     kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
24     clusters = kmeans.fit_predict(X_scaled)
25
26     df["Cluster"] = clusters
27
28     centers_scaled = kmeans.cluster_centers_
29     centers_original = scaler.inverse_transform(centers_scaled)
30
31     return df, kmeans, scaler, centers_original, X_scaled
32
33
34 # 评估聚类效果

```

```

35 def evaluate_clustering(X, clusters):
36     silhouette_avg = silhouette_score(X, clusters)
37     dbi = davies_bouldin_score(X, clusters)
38     ch_index = calinski_harabasz_score(X, clusters)
39
40     print("\n聚类效果评估:")
41     print(f"轮廓系数 (Silhouette Score): {silhouette_avg:.4f}")
42     print(f"DBI指数 (Davies-Bouldin Index): {dbi:.4f}")
43     print(f"CH指数 (Calinski-Harabasz Index): {ch_index:.4f}")
44
45     return silhouette_avg, dbi, ch_index
46
47
48 # 打印各聚类的BMI范围统计信息
49 def print_cluster_ranges(df):
50     print("\n各聚类的BMI范围:")
51     print("-" * 50)
52     for i in range(5):
53         cluster_data = df[df["Cluster"] == i]["BMI"]
54         min_bmi = cluster_data.min()
55         max_bmi = cluster_data.max()
56         mean_bmi = cluster_data.mean()
57         count = len(cluster_data)
58
59         if mean_bmi < 18.5:
60             category = "偏瘦"
61         elif mean_bmi < 24:
62             category = "正常"
63         elif mean_bmi < 28:
64             category = "偏胖"
65         else:
66             category = "肥胖"
67
68         print(
69             f"聚类 {i}: {count:3d} 个样本, BMI范围: {min_bmi:.2f}~{max_bmi:.2f}, "
70             f"平均值: {mean_bmi:.2f} ({category})"
71         )
72
73     print("-" * 50)
74
75
76 # 可视化聚类结果
77 def visualize_results(df, centers_original, silhouette_avg, dbi, ch_index):
78     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
79
80     # 绘制各聚类BMI分布直方图
81     for i in range(5):
82         cluster_data = df[df["Cluster"] == i]["BMI"]
83         axes[0, 0].hist(cluster_data, alpha=0.6, bins=20, label=f"Cluster {i}")
84
85         axes[0, 0].axvline(x=18.5, color="r", linestyle="--", label="偏瘦阈值")
86         axes[0, 0].axvline(x=24, color="g", linestyle="--", label="正常阈值")
87         axes[0, 0].axvline(x=28, color="orange", linestyle="--", label="偏胖阈值")
88         axes[0, 0].set_xlabel("BMI")
89         axes[0, 0].set_ylabel("频数")

```

```

90 axes[0, 0].set_title("各聚类BMI分布")
91 axes[0, 0].legend()
92 axes[0, 0].grid(True, alpha=0.3)
93
94 # 绘制各聚类中心BMI值柱状图
95 cluster_labels = [f"Cluster {i}" for i in range(5)]
96 axes[0, 1].bar(
97     cluster_labels,
98     centers_original.flatten(),
99     color=["blue", "green", "red", "purple", "orange"],
100 )
101 axes[0, 1].set_ylabel("BMI值")
102 axes[0, 1].set_title("各聚类中心BMI值")
103 axes[0, 1].grid(True, alpha=0.3)
104
105 # 绘制BMI与达标天数的散点图
106 for i in range(5):
107     cluster_data = df[df["Cluster"] == i]
108     axes[1, 0].scatter(
109         cluster_data["BMI"],
110         cluster_data["对应达标天数"],
111         alpha=0.6,
112         label=f"Cluster {i}",
113     )
114
115 axes[1, 0].set_xlabel("BMI")
116 axes[1, 0].set_ylabel("达标天数")
117 axes[1, 0].set_title("BMI与达标天数的关系")
118 axes[1, 0].legend()
119 axes[1, 0].grid(True, alpha=0.3)
120
121
122 # 保存分析结果到Excel文件
123 def save_results(df, centers_original, silhouette_avg, dbi, ch_index):
124     summary_data = {
125         "聚类编号": range(5),
126         "聚类中心(BMI)": centers_original.flatten(),
127         "样本数量": [len(df[df["Cluster"] == i]) for i in range(5)],
128         "BMI最小值": [df[df["Cluster"] == i]["BMI"].min() for i in range(5)],
129         "BMI最大值": [df[df["Cluster"] == i]["BMI"].max() for i in range(5)],
130         "BMI平均值": [df[df["Cluster"] == i]["BMI"].mean() for i in range(5)],
131     }
132     summary_df = pd.DataFrame(summary_data)
133
134     eval_df = pd.DataFrame(
135         {
136             "评估指标": ["轮廓系数", "DBI指数", "CH指数"],
137             "值": [silhouette_avg, dbi, ch_index],
138         }
139     )
140
141     with pd.ExcelWriter("聚类分析结果.xlsx") as writer:
142         df.to_excel(writer, sheet_name="原始数据与聚类结果", index=False)
143         summary_df.to_excel(writer, sheet_name="聚类摘要", index=False)
144         eval_df.to_excel(writer, sheet_name="评估指标", index=False)

```

```

145
146     print("\n结果已保存到 '聚类分析结果.xlsx'")
147
148
149 # 主函数，整合所有分析步骤
150 def main():
151     # 加载数据
152     file_path = "python_code/bmi_Y_result.xlsx"
153     df = pd.read_excel(file_path)
154
155     # 执行聚类分析
156     df_result, kmeans, scaler, centers_original, X_scaled = perform_clustering(df)
157
158     # 评估聚类效果
159     silhouette_avg, dbi, ch_index = evaluate_clustering(
160         X_scaled, df_result["Cluster"].values
161     )
162
163     # 打印聚类统计信息
164     print_cluster_ranges(df_result)
165
166     # 可视化结果
167     visualize_results(df_result, centers_original, silhouette_avg, dbi, ch_index)
168
169     # 保存结果
170     save_results(df_result, centers_original, silhouette_avg, dbi, ch_index)
171
172
173 # 程序入口
174 if __name__ == "__main__":
175     main()

```

Listing 6: C(3)preprocessing.py

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.cluster import KMeans
4 from sklearn.preprocessing import StandardScaler
5 from lifelines import KaplanMeierFitter
6 from scipy.optimize import minimize_scalar
7 import matplotlib.pyplot as plt
8 import re
9
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 # 加载附件数据并进行预处理
14 def load_and_preprocess_data():
15
16
17     df = pd.read_excel('./python_code/附件.xlsx', sheet_name=0)
18
19     def weeks_to_days(weeks_str):
20         match = re.match(r'(\d+)[wW](?:\+(\d+))?', str(weeks_str), re.IGNORECASE)
21         if match:

```

```

22         weeks = int(match.group(1))
23         days = int(match.group(2)) if match.group(2) else 0
24         return weeks * 7 + days
25     else:
26         print(f"无法解析孕周格式: {weeks_str}")
27         return None
28
29 df['检测孕周_天数'] = df['检测孕周'].apply(weeks_to_days)
30 # 提取男胎数据 (Y染色体浓度非空)
31 male_data = df[df['Y染色体浓度'].notna()].copy()
32 print(f"男胎数据量: {len(male_data)}")
33
34 # 选择特征: BMI、身高、体重、年龄
35 features = male_data[['孕妇代码', '年龄', '身高', '体重', '孕妇BMI', 'Y染色体浓度', 'Y染色体的Z值',
36                        '检测孕周_天数']].copy()
37
38 # 处理缺失值: 均值插补
39 for col in features.columns:
40     if col != '孕妇代码': # 跳过非数值列
41         if features[col].isnull().sum() > 0:
42             mean_val = features[col].mean()
43             features[col].fillna(mean_val, inplace=True)
44             print(f"列 '{col}' 有 {features[col].isnull().sum()} 个缺失值, 已用均值 {mean_val:.2f} 填充")
45
46 # 检查并处理可能的非数值数据
47 for col in features.columns:
48     if col != '孕妇代码': # 跳过非数值列
49         # 尝试转换为数值类型
50         features[col] = pd.to_numeric(features[col], errors='coerce')
51         # 再次检查并填充可能出现的NaN
52         if features[col].isnull().sum() > 0:
53             mean_val = features[col].mean()
54             features[col].fillna(mean_val, inplace=True)
55             print(f"列 '{col}' 转换后有缺失值, 已用均值 {mean_val:.2f} 填充")
56
57 # 使用Z-score标准化特征 (排除孕妇代码、Y染色体浓度和Y染色体的Z值)
58 scaler = StandardScaler() # StandardScaler默认使用Z-score标准化
59 feature_cols = ['年龄', '身高', '体重', '孕妇BMI', '检测孕周_天数']
60 features_scaled = scaler.fit_transform(features[feature_cols])
61 features_scaled_df = pd.DataFrame(features_scaled,
62                                   columns=[f'{col}_标准化' for col in feature_cols])
63
64 # 只保留需要的列: 孕妇代码、原始特征、标准化特征、Y染色体浓度和Y染色体的Z值
65 final_data = pd.concat([
66     features[['孕妇代码', 'Y染色体浓度', 'Y染色体的Z值']],
67     features[feature_cols],
68     features_scaled_df
69 ], axis=1)
70
71 # 根据Y染色体浓度判断能否检测
72 final_data['能否检测'] = final_data['Y染色体浓度'].apply(lambda x: 1 if x > 0.04 else 0)
73
74 can_code = []
75 df_codes = df['孕妇代码'].unique()

```

```

75     for code in df_codes:
76         data = df[df['孕妇代码'] == code].sort_values(by='检测孕周_天数')
77         first_ok_idx = None
78
79         for idx, row in data.iterrows():
80             if row['Y染色体浓度'] >= 0.04:
81                 first_ok_idx = idx
82                 break
83
84         if first_ok_idx is not None:
85             later_data = data.loc[first_ok_idx:]
86             if (later_data['Y染色体浓度'] < 0.04).any():
87                 continue
88             else:
89                 can_code.append(code)
90
91     final_data = final_data[final_data['孕妇代码'].isin(can_code)]
92
93
94     # 保存处理后的数据
95     try:
96         final_data.to_excel('./python_code/Q3/Q3数据预处理.xlsx', index=False)
97         print("预处理后的数据已保存至: ./python_code/Q3/Q3数据预处理.xlsx")
98     except Exception as e:
99         print(f"保存文件时出错: {e}")
100    return None
101
102    # 打印数据基本信息
103    print("\n=== 数据预处理完成 ===")
104    print(f"处理后的数据形状: {final_data.shape}")
105    print("\n特征统计信息:")
106    print(final_data.describe())
107
108    return final_data
109
110    # 运行数据预处理
111    processed_data = load_and_preprocess_data()

```

Listing 7: C(3) 聚类.py

```

1
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from sklearn.cluster import KMeans
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
8
9  # 设置中文字体支持
10 plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
11 plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
12
13 # 执行聚类分析
14 def perform_clustering(df, n_clusters=5):
15     """

```

```

16     执行K-means 聚类分析
17     """
18     # 提取BMI数据并标准化
19     X = df[['孕妇BMI']].values
20     scaler = StandardScaler()
21     X_scaled = scaler.fit_transform(X)
22
23     # 执行K-means 聚类
24     kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
25     clusters = kmeans.fit_predict(X_scaled)
26
27     # 将聚类结果添加到数据框中
28     df['Cluster'] = clusters
29
30     # 计算聚类中心（原始尺度）
31     centers_scaled = kmeans.cluster_centers_
32     centers_original = scaler.inverse_transform(centers_scaled)
33
34     return df, kmeans, scaler, centers_original, X_scaled
35
36 # 评估聚类效果
37 def evaluate_clustering(X, clusters):
38     """
39     计算聚类评估指标
40     """
41     silhouette_avg = silhouette_score(X, clusters)
42     dbi = davies_bouldin_score(X, clusters)
43     ch_index = calinski_harabasz_score(X, clusters)
44
45     print("\n聚类效果评估:")
46     print(f"轮廓系数 (Silhouette Score): {silhouette_avg:.4f}")
47     print(f"DBI 指数 (Davies-Bouldin Index): {dbi:.4f}")
48     print(f"CH指数 (Calinski-Harabasz Index): {ch_index:.4f}")
49
50     return silhouette_avg, dbi, ch_index
51
52 # 打印分类的BMI范围
53 def print_cluster_ranges(df):
54     """
55     打印每个聚类的BMI范围
56     """
57     print("\n各聚类的BMI范围:")
58     print("-" * 50)
59     for i in range(5):
60         cluster_data = df[df['Cluster'] == i]['孕妇BMI']
61         min_bmi = cluster_data.min()
62         max_bmi = cluster_data.max()
63         mean_bmi = cluster_data.mean()
64         count = len(cluster_data)
65
66         # 确定BMI分类标签
67         if mean_bmi < 18.5:
68             category = "偏瘦"
69         elif mean_bmi < 24:
70             category = "正常"

```

```

71         elif mean_bmi < 28:
72             category = "偏胖"
73         else:
74             category = "肥胖"
75
76         print(f"聚类 {i}: {count:3d} 个样本, BMI范围: {min_bmi:.2f}-{max_bmi:.2f}, "
77               f"平均值: {mean_bmi:.2f} ({category})")
78
79     print("-" * 50)
80
81 # 可视化结果
82 def visualize_results(df, centers_original, silhouette_avg, dbi, ch_index):
83     """
84     可视化聚类结果
85     """
86     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
87
88     # 聚类分布直方图
89     for i in range(5):
90         cluster_data = df[df['Cluster'] == i]['孕妇BMI']
91         axes[0, 0].hist(cluster_data, alpha=0.6, bins=20, label=f'Cluster {i}')
92
93         axes[0, 0].axvline(x=18.5, color='r', linestyle='--', label='偏瘦阈值')
94         axes[0, 0].axvline(x=24, color='g', linestyle='--', label='正常阈值')
95         axes[0, 0].axvline(x=28, color='orange', linestyle='--', label='偏胖阈值')
96         axes[0, 0].set_xlabel('BMI')
97         axes[0, 0].set_ylabel('频数')
98         axes[0, 0].set_title('各聚类BMI分布')
99         axes[0, 0].legend()
100        axes[0, 0].grid(True, alpha=0.3)
101
102    # 聚类中心比较
103    cluster_labels = [f'Cluster {i}' for i in range(5)]
104    axes[0, 1].bar(cluster_labels, centers_original.flatten(), color=['blue', 'green', 'red', 'purple',
105        'orange'])
106    axes[0, 1].set_ylabel('BMI值')
107    axes[0, 1].set_title('各聚类中心BMI值')
108    axes[0, 1].grid(True, alpha=0.3)
109
110    # 达标天数与BMI的关系
111    for i in range(5):
112        cluster_data = df[df['Cluster'] == i]
113        axes[1, 0].scatter(cluster_data['孕妇BMI'], cluster_data['检测孕周_天数'],
114            alpha=0.6, label=f'Cluster {i}')
115
116        axes[1, 0].set_xlabel('BMI')
117        axes[1, 0].set_ylabel('检测孕周_天数')
118        axes[1, 0].set_title('BMI与检测孕周_天数的关系')
119        axes[1, 0].legend()
120        axes[1, 0].grid(True, alpha=0.3)
121
122    plt.savefig('聚类分析可视化结果.png', dpi=300, bbox_inches='tight')
123    print("可视化图表已保存为 '聚类分析可视化结果.png'")
124

```

```

125
126
127 # 保存结果
128 def save_results(df, centers_original, silhouette_avg, dbi, ch_index):
129     """
130     保存聚类结果到Excel文件
131     """
132     # 创建结果摘要
133     summary_data = {
134         '聚类编号': range(5),
135         '聚类中心(BMI)': centers_original.flatten(),
136         '样本数量': [len(df[df['Cluster'] == i]) for i in range(5)],
137         'BMI最小值': [df[df['Cluster'] == i]['孕妇BMI'].min() for i in range(5)],
138         'BMI最大值': [df[df['Cluster'] == i]['孕妇BMI'].max() for i in range(5)],
139         'BMI平均值': [df[df['Cluster'] == i]['孕妇BMI'].mean() for i in range(5)]
140     }
141     summary_df = pd.DataFrame(summary_data)
142
143     # 添加评估指标
144     eval_df = pd.DataFrame({
145         '评估指标': ['轮廓系数', 'DBI指数', 'CH指数'],
146         '值': [silhouette_avg, dbi, ch_index]
147     })
148
149     # 保存到Excel
150     with pd.ExcelWriter('聚类分析结果.xlsx') as writer:
151         df.to_excel(writer, sheet_name='原始数据与聚类结果', index=False)
152         summary_df.to_excel(writer, sheet_name='聚类摘要', index=False)
153         eval_df.to_excel(writer, sheet_name='评估指标', index=False)
154
155     print("\n结果已保存到 '聚类分析结果.xlsx'")
156
157 # 主函数
158 def main():
159     # 请替换为您的Excel文件路径
160     file_path = "python_code\Q3\Q3数据预处理.xlsx"
161     df = pd.read_excel(file_path)
162     # 执行聚类分析
163     df_result, kmeans, scaler, centers_original, X_scaled = perform_clustering(df)
164
165     # 评估聚类效果
166     silhouette_avg, dbi, ch_index = evaluate_clustering(X_scaled, df_result['Cluster'].values)
167
168     # 打印分类的BMI范围
169     print_cluster_ranges(df_result)
170
171     # 可视化结果
172     visualize_results(df_result, centers_original, silhouette_avg, dbi, ch_index)
173
174     # 保存结果
175     save_results(df_result, centers_original, silhouette_avg, dbi, ch_index)
176
177 if __name__ == "__main__":
178     main()

```

Listing 8: C(3)final.py

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.ensemble import RandomForestRegressor
7 from sklearn.linear_model import LinearRegression
8 from sklearn.model_selection import cross_val_score, KFold
9 from scipy.optimize import minimize_scalar
10 from lifelines import KaplanMeierFitter
11 import shap
12 from statsmodels.nonparametric.smoothers_lowess import lowess
13 import warnings
14 from scipy.stats import norm
15 import re
16 warnings.filterwarnings('ignore')
17
18 # 设置中文字体支持
19 plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode MS", "DejaVu Sans"]
20 plt.rcParams["axes.unicode_minus"] = False
21
22 # 建模分析不同BMI分组中年龄、身高、体重对NIPT最佳时点的影响
23 def analyze_nipt_optimal_time_with_advanced_modeling():
24     # 读取预处理数据
25     file_path = "python_code/Q3/Q3数据预处理.xlsx"
26     df = pd.read_excel(file_path)
27
28     # 定义BMI分组函数
29     def categorize_bmi(bmi):
30         if bmi < 30.01:
31             return "聚类0 (<30.01)"
32         elif 30.01 <= bmi < 32.18:
33             return "聚类1 (30.01-32.18)"
34         elif 32.18 <= bmi < 34.63:
35             return "聚类2 (32.18-34.63)"
36         elif 34.63 <= bmi < 37.93:
37             return "聚类3 (34.63-37.93)"
38         else:
39             return "聚类4 (37.93)"
40
41     df['bmi_category'] = df['孕妇BMI'].apply(categorize_bmi)
42
43     # 准备生存分析数据
44     survival_data = []
45
46     for code, group in df.groupby('孕妇代码'):
47         group = group.sort_values('检测孕周_天数')
48         # 找到首次达标的时间
49         first_reach_idx = group[group['Y染色体浓度'] >= 0.04].index
50         last_below_idx = group[group['Y染色体浓度'] < 0.04].index
51
52         if len(first_reach_idx) > 0:
53             # 有达标记录
54             first_reach_time = group.loc[first_reach_idx[0], '检测孕周_天数']

```

```

55     first_reach_concentration = group.loc[first_reach_idx[0], 'Y染色体浓度']
56
57     # 找到最后一次未达标的时间
58     if len(last_below_idx) > 0 and last_below_idx[-1] < first_reach_idx[0]:
59         last_below_time = group.loc[last_below_idx[-1], '检测孕周_天数']
60         last_below_concentration = group.loc[last_below_idx[-1], 'Y染色体浓度']
61
62         # 使用线性插值法计算达标时间
63         # 公式:  $t = t_0 + (t_1 - t_0) * (0.04 - c_0) / (c_1 - c_0)$ 
64         event_time = last_below_time + (first_reach_time - last_below_time) * \
65             (0.04 - last_below_concentration) / (first_reach_concentration -
66             last_below_concentration)
67
68     else:
69         # 如果没有之前的未达标记录, 使用首次达标时间
70         event_time = first_reach_time
71
72     event_observed = 1
73
74     else:
75         # 未达标, 使用最后一次检测时间作为删失时间
76         event_time = group.iloc[-1]['检测孕周_天数']
77         event_observed = 0
78
79     # 获取孕妇的基本信息(取第一次检测的值)
80     base_info = group.iloc[0]
81
82     survival_data.append({
83         '孕妇代码': code,
84         '年龄': base_info['年龄'],
85         '身高': base_info['身高'],
86         '体重': base_info['体重'],
87         '孕妇BMI': base_info['孕妇BMI'],
88         'bmi_category': base_info['bmi_category'],
89         '达标时间': event_time,
90         '达标状态': event_observed
91     })
92
93     survival_df = pd.DataFrame(survival_data)
94
95     # 定义效用函数
96     def get_rearly(t):
97         """早期检测效用函数: 随时间指数衰减"""
98         return 2.0 * np.exp(-t / 50)
99
100     def get_rlate(t):
101         """晚期检测风险函数: 分段函数"""
102         if t < 84: # 12周
103             return 0.1
104         elif t <= 189: # 27周
105             normalized_t = (t - 84) / (189 - 84)
106             return 0.1 + 0.9 * (normalized_t**2)
107         else: # 超过27周
108             return 1.0
109
110     def calculate_utility(t, survival_func):
111         """计算总效用函数"""

```

```

109     if t < 0:
110         return float("inf")
111
112     # 找到最近的时间点
113     closest_time_idx = np.abs(survival_func.index - t).argmin()
114     actual_time = survival_func.index[closest_time_idx]
115
116     # 计算达标概率
117     p_t = 1 - survival_func.loc[actual_time, "s_t"]
118
119     # 计算总效用
120     return (1 - p_t) * get_rearly(t) + p_t * get_rlate(t)
121
122 # 分析每个BMI分组
123 bmi_categories = [
124     "聚类0 (<30.01)",
125     "聚类1 (30.01-32.18)",
126     "聚类2 (32.18-34.63)",
127     "聚类3 (34.63-37.93)",
128     "聚类4 (37.93)"
129 ]
130
131 results_table = []
132 model_performance = []
133 equations = [] # 存储每个分组的NIPT时点方程
134 optimal_times = {} # 存储每个分组的最优时点
135
136 for bmi_cat in bmi_categories:
137     df_bmi = survival_df[survival_df['bmi_category'] == bmi_cat].copy()
138
139     if df_bmi.empty:
140         results_table.append({
141             "BMI分组": bmi_cat,
142             "最优时点(天)": "无数据",
143             "最小效用值": "无数据",
144             "风险水平": "无数据",
145             "样本量": 0,
146         })
147         continue
148
149     print(f"\n=== 正在分析 BMI 分组: {bmi_cat} ===")
150     print(f"样本量: {len(df_bmi)}")
151
152     # 计算KM生存曲线
153     kmf = KaplanMeierFitter()
154     kmf.fit(durations=df_bmi["达标时间"], event_observed=df_bmi["达标状态"])
155
156     s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
157
158     # 优化找到最佳时点
159     def objective(t):
160         return calculate_utility(t, s_t)
161
162     result = minimize_scalar(
163         objective, bounds=(0, df_bmi["达标时间"].max()), method="bounded"

```

```

164 )
165
166 optimal_time = result.x
167 optimal_utility = result.fun
168 risk_level = 1 / optimal_utility if optimal_utility > 0 else float("inf")
169 sample_size = len(df_bmi)
170
171 # 存储最优时点
172 optimal_times[bmi_cat] = optimal_time
173
174 print(f"\n最优预测时间点分析:")
175 print(f" - 最优时间点: {optimal_time:.1f} 天 (约{optimal_time/7:.1f}周)")
176 print(f" - 最小效用值: {optimal_utility:.4f}")
177 print(f" - 风险水平: {risk_level:.4f}")
178
179 # 使用随机森林建模年龄、身高、体重对达标时间的影响
180 print("\n使用随机森林建模影响因素:")
181
182 # 重置索引以确保一致性
183 df_bmi_reset = df_bmi.reset_index(drop=True)
184
185 # 准备回归数据
186 X = df_bmi_reset[['年龄', '身高', '体重']]
187 y = df_bmi_reset['达标时间']
188
189 # 检查样本量是否足够进行建模
190 if len(X) < 10: # 至少需要10个样本
191     print("样本量不足, 使用线性回归建模")
192
193 # 使用线性回归
194 try:
195     # 使用线性回归
196     lr = LinearRegression()
197     lr.fit(X, y)
198
199     # 获取系数
200     coefficients = lr.coef_
201     intercept = lr.intercept_
202
203     # 生成回归方程
204     equation = f"T = {intercept:.2f} + {coefficients[0]:.2f}×年龄 + {coefficients[1]:.2f}×身  
高 + {coefficients[2]:.2f}×体重"
205
206     # 计算模型性能
207     y_pred = lr.predict(X)
208     model_r2 = lr.score(X, y)
209     model_mae = np.mean(np.abs(y - y_pred))
210
211     print(f"线性回归模型性能:")
212     print(f" - R²得分: {model_r2:.4f}")
213     print(f" - 平均绝对误差: {model_mae:.2f}天")
214     print(f"回归方程: {equation}")
215
216     # 计算特征重要性 (使用系数的绝对值)
217     feature_importance = np.abs(coefficients) / np.sum(np.abs(coefficients))

```

```

218     age_importance = feature_importance[0]
219     height_importance = feature_importance[1]
220     weight_importance = feature_importance[2]
221
222     print(f"特征重要性:")
223     print(f" - 年龄: {age_importance:.4f}")
224     print(f" - 身高: {height_importance:.4f}")
225     print(f" - 体重: {weight_importance:.4f}")
226
227     # 验证引入影响因素后的改进
228     y_mean = np.full_like(y_pred, y.mean())
229     mae_mean = np.mean(np.abs(y - y_mean))
230     improvement = (mae_mean - model_mae) / mae_mean * 100
231
232     print(f"\n模型验证结果:")
233     print(f" - 仅使用平均值预测的MAE: {mae_mean:.2f}天")
234     print(f" - 使用线性回归预测的MAE: {model_mae:.2f}天")
235     print(f" - 改进百分比: {improvement:.2f}%")
236
237     model_performance.append({
238         "BMI分组": bmi_cat,
239         "样本量": len(X),
240         "R²得分": model_r2,
241         "模型MAE": model_mae,
242         "平均值MAE": mae_mean,
243         "改进百分比": improvement,
244         "模型类型": "线性回归"
245     })
246
247     # 存储方程
248     equations.append({
249         "BMI分组": bmi_cat,
250         "方程": equation,
251         "模型类型": "线性回归",
252         "最优时点": optimal_time
253     })
254
255 except Exception as e:
256     print(f"线性回归建模出错: {e}")
257     age_importance = np.nan
258     height_importance = np.nan
259     weight_importance = np.nan
260     model_r2 = np.nan
261     model_mae = np.nan
262     equation = "无法生成方程"
263
264 else:
265     try:
266         # 标准化特征
267         scaler = StandardScaler()
268         X_scaled = scaler.fit_transform(X)
269
270         # 使用随机森林回归
271         rf = RandomForestRegressor(n_estimators=100, random_state=42)
272
273         # 交叉验证评估模型性能

```

```

273 kf = KFold(n_splits=min(5, len(X)), shuffle=True, random_state=42)
274 r2_scores = cross_val_score(rf, X_scaled, y, cv=kf, scoring='r2')
275 mae_scores = -cross_val_score(rf, X_scaled, y, cv=kf, scoring='neg_mean_absolute_error')
276
277 # 训练最终模型
278 rf.fit(X_scaled, y)
279
280 # 获取特征重要性
281 feature_importance = rf.feature_importances_
282 age_importance = feature_importance[0]
283 height_importance = feature_importance[1]
284 weight_importance = feature_importance[2]
285
286 # 计算模型性能
287 model_r2 = np.mean(r2_scores)
288 model_mae = np.mean(mae_scores)
289
290 print(f"随机森林模型性能:")
291 print(f" - R²得分: {model_r2:.4f}")
292 print(f" - 平均绝对误差: {model_mae:.2f}天")
293 print(f"特征重要性:")
294 print(f" - 年龄: {age_importance:.4f}")
295 print(f" - 身高: {height_importance:.4f}")
296 print(f" - 体重: {weight_importance:.4f}")
297
298 # 使用SHAP值解释模型
299 if len(X) >= 20: # 只有当样本量足够时才计算SHAP值
300     try:
301         explainer = shap.TreeExplainer(rf)
302         shap_values = explainer.shap_values(X_scaled)
303
304         plt.figure(figsize=(10, 6))
305         shap.summary_plot(shap_values, X_scaled, feature_names=['年龄', '身高', '体重'],
306                             show=False)
307         plt.title(f'BMI分组 {bmi_cat} 特征影响分析 (SHAP值)')
308         plt.tight_layout()
309         plt.savefig(f'./python_code/BMI_{bmi_cat.replace("<", "lt").replace(" ", "ge").
310                     replace(" ", "_").replace("(", "").replace(")", "")}_shap.png', dpi=300,
311                     bbox_inches="tight")
312         plt.close()
313         print("SHAP分析图表已保存")
314     except Exception as e:
315         print(f"SHAP分析出错: {e}")
316
317 # 验证引入影响因素后的改进
318 y_pred = rf.predict(X_scaled)
319 y_mean = np.full_like(y_pred, y.mean())
320 mae_model = np.mean(np.abs(y - y_pred))
321 mae_mean = np.mean(np.abs(y - y_mean))
322 improvement = (mae_mean - mae_model) / mae_mean * 100
323
324 print(f"\n模型验证结果:")
325 print(f" - 仅使用平均值预测的MAE: {mae_mean:.2f}天")
326 print(f" - 使用完整模型预测的MAE: {mae_model:.2f}天")
327 print(f" - 改进百分比: {improvement:.2f}%")

```

```

325
326     model_performance.append({
327         "BMI分组": bmi_cat,
328         "样本量": len(X),
329         "R²得分": model_r2,
330         "模型MAE": mae_model,
331         "平均值MAE": mae_mean,
332         "改进百分比": improvement,
333         "模型类型": "随机森林"
334     })
335
336     # 为随机森林生成更合理的方程描述
337     # 计算每个特征的相对影响
338     age_effect = age_importance * 100
339     height_effect = height_importance * 100
340     weight_effect = weight_importance * 100
341
342     # 生成方程描述
343     equation = f"T    {optimal_time:.1f} + {age_effect:.1f}%×(年龄影响) + {height_effect:.1f}%×(身高影响) + {weight_effect:.1f}%×(体重影响)"
344
345     # 存储方程
346     equations.append({
347         "BMI分组": bmi_cat,
348         "方程": equation,
349         "模型类型": "随机森林",
350         "最优时点": optimal_time
351     })
352
353     except Exception as e:
354         print(f"随机森林建模出错: {e}")
355         age_importance = np.nan
356         height_importance = np.nan
357         weight_importance = np.nan
358         model_r2 = np.nan
359         model_mae = np.nan
360         equation = "无法生成方程"
361
362     # 可视化影响因素
363     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
364
365     # 1. 生存曲线和效用函数
366     ax = axes[0, 0]
367     (1 - kmf.survival_function_).plot(
368         ax=ax,
369         label=f"累积达标函数 F(t) ({bmi_cat})",
370         color="seagreen",
371         linewidth=2.5,
372     )
373
374     time_points = np.linspace(0, df_bmi["达标时间"].max(), 200)
375     utilities = [calculate_utility(t, s_t) for t in time_points]
376     ax_twin = ax.twinx()
377     ax_twin.plot(
378         time_points, utilities, "--", color="red", label="效用函数 E(t)", alpha=0.6

```

```

379 )
380
381 ax_twin.axvline(x=optimal_time, color='purple', linestyle='--', linewidth=2)
382 ax_twin.text(
383     optimal_time + 2,
384     min(utilities) + 0.1,
385     f"最优时间点: {optimal_time:.1f}天",
386     color='purple',
387     fontsize=10,
388 )
389
390 ax.axvspan(0, 84, facecolor='green', alpha=0.15, label='早期阶段 (<12周)')
391 ax.axvspan(84, 189, facecolor='orange', alpha=0.15, label='中期阶段 (12-27周)')
392 ax.axvspan(
393     189,
394     df_bmi["达标时间"].max() + 10,
395     facecolor='red',
396     alpha=0.15,
397     label='晚期阶段 (>27周)',
398 )
399
400 ax.set_title(f"BMI分组 {bmi_cat} 达标情况与效用分析", fontsize=14)
401 ax.set_xlabel("天数", fontsize=12)
402 ax.set_ylabel("累积达标比例", fontsize=12)
403 ax_twin.set_ylabel("效用值", fontsize=12)
404
405 lines1, labels1 = ax.get_legend_handles_labels()
406 lines2, labels2 = ax_twin.get_legend_handles_labels()
407 ax.legend(lines1 + lines2, labels1 + labels2, loc='upper left')
408
409 ax.grid(True, alpha=0.3)
410 ax.set_ylim(0, 1.05)
411 ax_twin.set_ylim(0, max(utilities) * 1.2)
412 ax.set_xlim(-5, df_bmi["达标时间"].max() + 10)
413
414 # 2. 年龄与达标时间的关系
415 axes[0, 1].scatter(df_bmi['年龄'], df_bmi['达标时间'], alpha=0.6)
416 if not np.isnan(age_importance):
417     # 使用局部加权散点平滑 (LOWESS) 拟合曲线
418     try:
419         lowess_fit = lowess(df_bmi['达标时间'], df_bmi['年龄'], frac=0.7)
420         axes[0, 1].plot(lowess_fit[:, 0], lowess_fit[:, 1], 'r-', alpha=0.8, label='LOWESS拟合')
421     except:
422         # 如果LOWESS失败, 使用多项式拟合
423         z = np.polyfit(df_bmi['年龄'], df_bmi['达标时间'], 2)
424         p = np.poly1d(z)
425         x_sorted = np.sort(df_bmi['年龄'])
426         axes[0, 1].plot(x_sorted, p(x_sorted), 'r-', alpha=0.8, label='多项式拟合')
427 axes[0, 1].set_xlabel('年龄')
428 axes[0, 1].set_ylabel('达标时间(天)')
429 axes[0, 1].set_title('年龄与达标时间的关系')
430 axes[0, 1].grid(True, alpha=0.3)
431 if not np.isnan(age_importance):
432     axes[0, 1].text(0.05, 0.95, f'重要性: {age_importance:.4f}', transform=axes[0, 1].transAxes,
433                    fontsize=12, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='

```

```

434         wheat', alpha=0.5))
435
436 # 3. 身高与达标时间的关系
437 axes[1, 0].scatter(df_bmi['身高'], df_bmi['达标时间'], alpha=0.6)
438 if not np.isnan(height_importance):
439     try:
440         lowess_fit = lowess(df_bmi['达标时间'], df_bmi['身高'], frac=0.7)
441         axes[1, 0].plot(lowess_fit[:, 0], lowess_fit[:, 1], 'r--', alpha=0.8, label='LOWESS 拟合')
442     except:
443         z = np.polyfit(df_bmi['身高'], df_bmi['达标时间'], 2)
444         p = np.poly1d(z)
445         x_sorted = np.sort(df_bmi['身高'])
446         axes[1, 0].plot(x_sorted, p(x_sorted), 'r--', alpha=0.8, label='多项式拟合')
447 axes[1, 0].set_xlabel('身高(cm)')
448 axes[1, 0].set_ylabel('达标时间(天)')
449 axes[1, 0].set_title('身高与达标时间的关系')
450 axes[1, 0].grid(True, alpha=0.3)
451 if not np.isnan(height_importance):
452     axes[1, 0].text(0.05, 0.95, f'重要性: {height_importance:.4f}', transform=axes[1, 0].
453                     transAxes,
454                     fontsize=12, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='
455                             wheat', alpha=0.5))
456
457 # 4. 体重与达标时间的关系
458 axes[1, 1].scatter(df_bmi['体重'], df_bmi['达标时间'], alpha=0.6)
459 if not np.isnan(weight_importance):
460     try:
461         lowess_fit = lowess(df_bmi['达标时间'], df_bmi['体重'], frac=0.7)
462         axes[1, 1].plot(lowess_fit[:, 0], lowess_fit[:, 1], 'r--', alpha=0.8, label='LOWESS 拟合')
463     except:
464         z = np.polyfit(df_bmi['体重'], df_bmi['达标时间'], 2)
465         p = np.poly1d(z)
466         x_sorted = np.sort(df_bmi['体重'])
467         axes[1, 1].plot(x_sorted, p(x_sorted), 'r--', alpha=0.8, label='多项式拟合')
468 axes[1, 1].set_xlabel('体重(kg)')
469 axes[1, 1].set_ylabel('达标时间(天)')
470 axes[1, 1].set_title('体重与达标时间的关系')
471 axes[1, 1].grid(True, alpha=0.3)
472 if not np.isnan(weight_importance):
473     axes[1, 1].text(0.05, 0.95, f'重要性: {weight_importance:.4f}', transform=axes[1, 1].
474                     transAxes,
475                     fontsize=12, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='
476                             wheat', alpha=0.5))
477
478 plt.tight_layout()
479
480 # 保存图表
481 filename = f'./python_code/BMI_{bmi_cat.replace("<", "lt").replace(" ", "ge").replace(" ", "_").
482             replace("<", "").replace(">", "")}_analysis.png'
483 plt.savefig(filename, dpi=300, bbox_inches="tight")
484 plt.close()
485
486 print(f"\n图表已保存至: {filename}")
487
488 results_table.append({

```

```

483         "BMI分组": bmi_cat,
484         "最优时点(天)": f"{optimal_time:.1f}",
485         "最优时点(周)": f"{optimal_time/7:.1f}",
486         "最小效用值": f"{optimal_utility:.4f}",
487         "风险水平": f"{risk_level:.4f}",
488         "样本量": sample_size,
489         "年龄重要性": f"{age_importance:.4f}" if not np.isnan(age_importance) else "N/A",
490         "身高重要性": f"{height_importance:.4f}" if not np.isnan(height_importance) else "N/A",
491         "体重重要性": f"{weight_importance:.4f}" if not np.isnan(weight_importance) else "N/A",
492         "模型R²": f"{model_r2:.4f}" if not np.isnan(model_r2) else "N/A",
493         "模型MAE": f"{model_mae:.2f}" if not np.isnan(model_mae) else "N/A",
494     })
495
496     print("\n" + "="*60 + "\n")
497
498     # 打印和保存结果表格
499     print("\n\n=== 各BMI分组NIPT时点计算结果 ===")
500     results_df = pd.DataFrame(results_table)
501     print(results_df.to_string(index=False))
502
503     results_df.to_excel("./python_code/NIPT_optimal_times_with_advanced_modeling.xlsx", index=False)
504     print(f"\n结果表格已保存至: ./python_code/NIPT_optimal_times_with_advanced_modeling.xlsx")
505
506     # 保存模型性能评估结果
507     if model_performance:
508         performance_df = pd.DataFrame(model_performance)
509         performance_df.to_excel("./python_code/model_performance_evaluation.xlsx", index=False)
510         print(f"模型性能评估已保存至: ./python_code/model_performance_evaluation.xlsx")
511
512     # 绘制模型性能比较图
513     plt.figure(figsize=(10, 6))
514     x = np.arange(len(performance_df))
515     width = 0.35
516
517     plt.bar(x - width/2, performance_df['平均值MAE'], width, label='仅使用平均值')
518     plt.bar(x + width/2, performance_df['模型MAE'], width, label='使用完整模型')
519
520     plt.xlabel('BMI分组')
521     plt.ylabel('平均绝对误差(天)')
522     plt.title('模型性能比较: 仅使用平均值 vs 使用完整模型')
523     plt.xticks(x, performance_df['BMI分组'])
524     plt.legend()
525     plt.grid(True, alpha=0.3)
526
527     # 在柱状图上添加数值标签
528     for i, v in enumerate(performance_df['平均值MAE']):
529         plt.text(i - width/2, v + 0.5, f'{v:.1f}', ha='center')
530
531     for i, v in enumerate(performance_df['模型MAE']):
532         plt.text(i + width/2, v + 0.5, f'{v:.1f}', ha='center')
533
534     plt.tight_layout()
535     plt.savefig('./python_code/model_performance_comparison.png', dpi=300, bbox_inches='tight')
536     plt.close()
537     print("模型性能比较图已保存")

```

```

538
539 # 创建更合理的NIPT时点方程
540 print("\n\n=== 各BMI分组NIPT时点方程 ===")
541 improved_equations = []
542
543 for bmi_cat in bmi_categories:
544     if bmi_cat not in optimal_times:
545         improved_equations.append({
546             "BMI分组": bmi_cat,
547             "NIPT时点方程": "无数据",
548             "模型类型": "无数据"
549         })
550         continue
551
552     optimal_time = optimal_times[bmi_cat]
553
554     # 根据BMI分组提供个性化的NIPT时点建议
555     if bmi_cat == "聚类0 (<30.01)":
556         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI<30.01的
                    孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
557     elif bmi_cat == "聚类1 (30.01-32.18)":
558         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI
                    30.01-32.18的孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
559     elif bmi_cat == "聚类2 (32.18-34.63)":
560         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI
                    32.18-34.63的孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
561     elif bmi_cat == "聚类3 (34.63-37.93)":
562         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI
                    34.63-37.93的孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
563     else: # 聚类4 (37.93)
564         equation = f"建议NIPT时点: {optimal_time:.1f}天 (约{optimal_time/7:.1f}周)\n对于BMI 37.93的
                    孕妇, 建议在孕{optimal_time/7:.1f}周左右进行NIPT检测"
565
566     # 确定模型类型
567     model_type = "线性回归" if bmi_cat == "聚类4 (37.93)" else "随机森林"
568
569     improved_equations.append({
570         "BMI分组": bmi_cat,
571         "NIPT时点方程": equation,
572         "模型类型": model_type
573     })
574
575     print(f"\n{bmi_cat}:")
576     print(equation)
577
578 # 保存改进的NIPT时点方程
579 improved_equations_df = pd.DataFrame(improved_equations)
580 improved_equations_df.to_excel("./python_code/improved_NIPT_time_equations.xlsx", index=False)
581 print(f"\n改进的NIPT时点方程已保存至: ./python_code/improved_NIPT_time_equations.xlsx")
582
583 # 创建影响因素汇总图
584 fig, axes = plt.subplots(1, 3, figsize=(18, 6))
585
586 # 提取影响系数
587 age_effects = []

```

```

588 height_effects = []
589 weight_effects = []
590 labels = []
591
592 for _, row in results_df.iterrows():
593     if row['年龄重要性'] != 'N/A':
594         age_effects.append(float(row['年龄重要性']))
595         height_effects.append(float(row['身高重要性']))
596         weight_effects.append(float(row['体重重要性']))
597         labels.append(row['BMI分组'].split(' ')[0])
598
599 # 绘制年龄影响
600 if age_effects:
601     axes[0].bar(range(len(age_effects)), age_effects, color='skyblue')
602     axes[0].set_xlabel('BMI分组')
603     axes[0].set_ylabel('特征重要性')
604     axes[0].set_title('年龄对达标时间的影响')
605     axes[0].set_xticks(range(len(age_effects)))
606     axes[0].set_xticklabels(labels)
607
608 # 绘制身高影响
609 if height_effects:
610     axes[1].bar(range(len(height_effects)), height_effects, color='lightgreen')
611     axes[1].set_xlabel('BMI分组')
612     axes[1].set_ylabel('特征重要性')
613     axes[1].set_title('身高对达标时间的影响')
614     axes[1].set_xticks(range(len(height_effects)))
615     axes[1].set_xticklabels(labels)
616
617 # 绘制体重影响
618 if weight_effects:
619     axes[2].bar(range(len(weight_effects)), weight_effects, color='lightcoral')
620     axes[2].set_xlabel('BMI分组')
621     axes[2].set_ylabel('特征重要性')
622     axes[2].set_title('体重对达标时间的影响')
623     axes[2].set_xticks(range(len(weight_effects)))
624     axes[2].set_xticklabels(labels)
625
626 plt.tight_layout()
627 plt.savefig('./python_code/factors_impact_summary_advanced.png', dpi=300, bbox_inches='tight')
628 plt.close()
629
630 print("\n影响因素汇总图已保存至: ./python_code/factors_impact_summary_advanced.png")
631
632 error_analysis_results = analyze_error_impact(survival_df, df, bmi_categories, num_simulations=100,
633                                              error_std=0.01)
634 return results_df, improved_equations_df, error_analysis_results
635
636 # 分析检测误差对最优时点、效用值和风险的影响
637 def analyze_error_impact(survival_df, original_df, bmi_categories, num_simulations=100, error_std=0.01):
638     # 辅助函数: 清理文件名中的非法字符
639     def clean_filename(name):
640         return re.sub(r'[:<>:"/\|?*]', "_", name)

```

```

642 # 存储原始最优时点
643 original_optimal_times = {}
644 error_results = {bmi_cat: {"times": [], "utilities": [], "risks": []} for bmi_cat in bmi_categories}
645
646 # 定义效用函数（与主函数中相同）
647 def get_rearly(t):
648     return 2.0 * np.exp(-t / 50)
649
650 def get_rlate(t):
651     if t < 84:
652         return 0.1
653     elif t <= 189:
654         normalized_t = (t - 84) / (189 - 84)
655         return 0.1 + 0.9 * (normalized_t**2)
656     else:
657         return 1.0
658
659 def calculate_utility(t, survival_func):
660     if t < 0:
661         return float("inf")
662
663     closest_time_idx = np.abs(survival_func.index - t).argmin()
664     actual_time = survival_func.index[closest_time_idx]
665     p_t = 1 - survival_func.loc[actual_time, "s_t"]
666     return (1 - p_t) * get_rearly(t) + p_t * get_rlate(t)
667
668 # 计算原始最优时点
669 for bmi_cat in bmi_categories:
670     df_bmi = survival_df[survival_df['bmi_category'] == bmi_cat].copy()
671     if df_bmi.empty:
672         continue
673
674     kmf = KaplanMeierFitter()
675     kmf.fit(durations=df_bmi["达标时间"], event_observed=df_bmi["达标状态"])
676     s_t = kmf.survival_function_.rename(columns={"KM_estimate": "s_t"})
677
678     def objective(t):
679         return calculate_utility(t, s_t)
680
681     result = minimize_scalar(objective, bounds=(0, df_bmi["达标时间"].max()), method="bounded")
682     original_optimal_times[bmi_cat] = result.x
683
684 # 开始误差模拟
685 for sim in range(num_simulations):
686     print(f"正在进行第 {sim+1} 次误差模拟...")
687     np.random.seed(sim) # 确保可重复性
688
689     # 创建带误差的数据副本
690     df_error = original_df.copy()
691
692     # 对Y染色体浓度添加随机误差
693     df_error['Y染色体浓度_error'] = df_error['Y染色体浓度'] + np.random.normal(
694         0, error_std, size=len(df_error))
695
696

```

```

697 # 重新计算每个孕妇的达标时间（带误差）
698 survival_data_error = []
699
700 for code, group in df_error.groupby('孕妇代码'):
701     group = group.sort_values('检测孕周_天数')
702
703     # 找到首次达标的时间（使用带误差的Y染色体浓度）
704     first_reach_idx = group[group['Y染色体浓度_error'] >= 0.04].index
705     last_below_idx = group[group['Y染色体浓度_error'] < 0.04].index
706
707     if len(first_reach_idx) > 0:
708         first_reach_time = group.loc[first_reach_idx[0], '检测孕周_天数']
709         first_reach_concentration = group.loc[first_reach_idx[0], 'Y染色体浓度_error']
710
711         if len(last_below_idx) > 0 and last_below_idx[-1] < first_reach_idx[0]:
712             last_below_time = group.loc[last_below_idx[-1], '检测孕周_天数']
713             last_below_concentration = group.loc[last_below_idx[-1], 'Y染色体浓度_error']
714
715             # 使用线性插值法计算达标时间
716             event_time = last_below_time + (first_reach_time - last_below_time) * \
717                 (0.04 - last_below_concentration) / (first_reach_concentration -
718                 last_below_concentration)
719
720             else:
721                 event_time = first_reach_time
722
723             event_observed = 1
724         else:
725             event_time = group.iloc[-1]['检测孕周_天数']
726             event_observed = 0
727
728     base_info = group.iloc[0]
729
730     survival_data_error.append({
731         '孕妇代码': code,
732         '年龄': base_info['年龄'],
733         '身高': base_info['身高'],
734         '体重': base_info['体重'],
735         '孕妇BMI': base_info['孕妇BMI'],
736         'bmi_category': base_info['bmi_category'],
737         '达标时间': event_time,
738         '达标状态': event_observed
739     })
740
741 survival_df_error = pd.DataFrame(survival_data_error)
742
743 # 对每个BMI分组计算带误差的最优时点
744 for bmi_cat in bmi_categories:
745     df_bmi_error = survival_df_error[survival_df_error['bmi_category'] == bmi_cat].copy()
746     if df_bmi_error.empty:
747         continue
748
749     kmf_error = KaplanMeierFitter()
750     kmf_error.fit(durations=df_bmi_error["达标时间"], event_observed=df_bmi_error["达标状态"])
751     s_t_error = kmf_error.survival_function_.rename(columns={"KM_estimate": "s_t"})

```

```

751     def objective_error(t):
752         return calculate_utility(t, s_t_error)
753
754     result_error = minimize_scalar(
755         objective_error, bounds=(0, df_bmi_error["达标时间"].max()), method="bounded"
756     )
757
758     optimal_time_error = result_error.x
759     optimal_utility_error = result_error.fun
760     risk_error = 1 / optimal_utility_error if optimal_utility_error > 0 else float("inf")
761
762     error_results[bmi_cat]["times"].append(optimal_time_error)
763     error_results[bmi_cat]["utilities"].append(optimal_utility_error)
764     error_results[bmi_cat]["risks"].append(risk_error)
765
766 # 输出误差分析结果
767 print("\n=== 检测误差影响分析结果 ===")
768 error_analysis_summary = []
769
770 for bmi_cat in bmi_categories:
771     if error_results[bmi_cat]["times"]:
772         times = np.array(error_results[bmi_cat]["times"])
773         utilities = np.array(error_results[bmi_cat]["utilities"])
774         risks = np.array(error_results[bmi_cat]["risks"])
775
776         mean_time = times.mean()
777         std_time = times.std()
778         time_ci = norm.interval(0.95, loc=mean_time, scale=std_time/np.sqrt(len(times)))
779
780         mean_utility = utilities.mean()
781         std_utility = utilities.std()
782
783         mean_risk = risks.mean()
784         std_risk = risks.std()
785
786         original_time = original_optimal_times.get(bmi_cat, 0)
787         original_utility = calculate_utility(original_time, s_t) if bmi_cat in
788             original_optimal_times else 0
789
790         print(f"BMI分组 {bmi_cat}:")
791         print(f"  原始最优时点: {original_time:.2f} 天")
792         print(f"  模拟最优时点均值: {mean_time:.2f} 天")
793         print(f"  标准差: {std_time:.2f} 天")
794         print(f"  95%置信区间: ({time_ci[0]:.2f}, {time_ci[1]:.2f}) 天")
795         print(f"  原始最小效用值: {original_utility:.4f}")
796         print(f"  模拟最小效用值均值: {mean_utility:.4f}")
797         print(f"  效用值标准差: {std_utility:.4f}")
798         print(f"  模拟风险水平均值: {mean_risk:.4f}")
799         print(f"  风险水平标准差: {std_risk:.4f}")
800
801 # 可视化误差分析结果
802 plt.figure(figsize=(12, 8))
803
804 plt.subplot(2, 2, 1)
805 plt.hist(times, bins=20, alpha=0.7, edgecolor="black")

```

```

805 plt.axvline(mean_time, color="r", linestyle="--", label=f"模拟平均时点: {mean_time:.2f}")
806 plt.axvline(original_time, color="b", linestyle="-", label=f"原始最优时点: {original_time:.2f}")
807 plt.axvline(time_ci[0], color="g", linestyle=":", label="95%置信区间")
808 plt.axvline(time_ci[1], color="g", linestyle=":")
809 plt.xlabel("最优NIPT时点 (天)")
810 plt.ylabel("频数")
811 plt.title(f"BMI分组 {bmi_cat} - 最优时点分布")
812 plt.legend()
813 plt.grid(True, alpha=0.3)
814
815 plt.subplot(2, 2, 2)
816 plt.hist(utilities, bins=20, alpha=0.7, edgecolor="black", color="orange")
817 plt.axvline(mean_utility, color="r", linestyle="--", label=f"模拟平均效用值: {mean_utility:.4f}")
818 plt.axvline(original_utility, color="b", linestyle="-", label=f"原始最小效用值: {original_utility:.4f}")
819 plt.xlabel("最小效用值")
820 plt.ylabel("频数")
821 plt.title(f"BMI分组 {bmi_cat} - 最小效用值分布")
822 plt.legend()
823 plt.grid(True, alpha=0.3)
824
825 plt.subplot(2, 2, 3)
826 plt.scatter(times, utilities, alpha=0.6)
827 plt.axvline(original_time, color="b", linestyle="-", label="原始最优时点")
828 plt.axhline(original_utility, color="b", linestyle="-", label="原始最小效用值")
829 plt.xlabel("最优时点 (天)")
830 plt.ylabel("最小效用值")
831 plt.title(f"BMI分组 {bmi_cat} - 时点与效用值关系")
832 plt.legend()
833 plt.grid(True, alpha=0.3)
834
835 plt.subplot(2, 2, 4)
836 plt.hist(risks, bins=20, alpha=0.7, edgecolor="black", color="red")
837 plt.axvline(mean_risk, color="r", linestyle="--", label=f"模拟平均风险: {mean_risk:.4f}")
838 plt.xlabel("风险水平")
839 plt.ylabel("频数")
840 plt.title(f"BMI分组 {bmi_cat} - 风险水平分布")
841 plt.legend()
842 plt.grid(True, alpha=0.3)
843
844 plt.tight_layout()
845
846 safe_bmi_cat = clean_filename(bmi_cat)
847 filename = f"./python_code/error_analysis_BMI_{safe_bmi_cat}.png"
848 plt.savefig(filename, dpi=300, bbox_inches="tight")
849 plt.close()
850
851 # 保存模拟数据
852 data_df = pd.DataFrame({
853     "time": times,
854     "utility": utilities,
855     "risk": risks
856 })

```

```

857     csv_filename = f"./python_code/error_analysis_BMI_{safe_bmi_cat}.csv"
858     data_df.to_csv(csv_filename, index=False)
859
860     print(f" 图表已保存至: {filename}")
861     print(f" 数据已保存至: {csv_filename}")
862
863     error_analysis_summary.append({
864         "BMI分组": bmi_cat,
865         "原始最优时点": original_time,
866         "模拟最优时点均值": mean_time,
867         "最优时点标准差": std_time,
868         "最优时点95%置信区间下限": time_ci[0],
869         "最优时点95%置信区间上限": time_ci[1],
870         "原始最小效用值": original_utility,
871         "模拟最小效用值均值": mean_utility,
872         "最小效用值标准差": std_utility,
873         "模拟风险水平均值": mean_risk,
874         "风险水平标准差": std_risk
875     })
876     else:
877         print(f"BMI分组 {bmi_cat}: 无数据")
878         error_analysis_summary.append({
879             "BMI分组": bmi_cat,
880             "原始最优时点": "无数据",
881             "模拟最优时点均值": "无数据",
882             "最优时点标准差": "无数据",
883             "最优时点95%置信区间下限": "无数据",
884             "最优时点95%置信区间上限": "无数据",
885             "原始最小效用值": "无数据",
886             "模拟最小效用值均值": "无数据",
887             "最小效用值标准差": "无数据",
888             "模拟风险水平均值": "无数据",
889             "风险水平标准差": "无数据"
890         })
891
892     # 保存误差分析摘要
893     error_summary_df = pd.DataFrame(error_analysis_summary)
894     error_summary_df.to_excel("./python_code/error_analysis_summary.xlsx", index=False)
895     print(f"\n误差分析摘要已保存至: ./python_code/error_analysis_summary.xlsx")
896
897     return error_summary_df
898
899 results, equations, error_results = analyze_nipt_optimal_time_with_advanced_modeling()

```

Listing 9: 数据处理.py

```

1  import pandas as pd
2  import numpy as np
3
4
5  def process_female_nipt_data(input_file_path, output_file_path):
6      """
7      处理女胎NIPT数据，进行质量控制和数据清洗，然后保存到新的Excel文件
8
9      参数：

```

```

10 input_file_path: 输入Excel文件路径
11 output_file_path: 输出Excel文件路径
12 """
13
14 # 1. 读取数据 - 指定读取第二个工作表
15 print("正在读取数据...")
16 try:
17     # 读取第二个工作表
18     df = pd.read_excel(input_file_path, sheet_name=1)
19     print(f"成功读取数据: {df.shape[0]} 行, {df.shape[1]} 列")
20 except Exception as e:
21     print(f"读取数据时出错: {e}")
22     return
23
24 # 2. 添加三种疾病的one-hot编码列
25 print("\n正在添加疾病分类列...")
26
27 # 确保"染色体的非整倍体"列存在
28 if "染色体的非整倍体" not in df.columns:
29     print("错误: 未找到'染色体的非整倍体'列")
30     return
31
32 # 创建三种疾病的标志列
33 df["T21"] = 0 # 唐氏综合征 (21号染色体三体)
34 df["T18"] = 0 # 爱德华氏综合征 (18号染色体三体)
35 df["T13"] = 0 # 帕陶氏综合征 (13号染色体三体)
36
37 # 根据"染色体的非整倍体"列设置疾病标志
38 for idx, value in df["染色体的非整倍体"].items():
39     if pd.isna(value) or value == "":
40         continue
41
42     # 根据内容设置相应的疾病标志
43     if "T21" in str(value) or "21" in str(value):
44         df.at[idx, "T21"] = 1
45     if "T18" in str(value) or "18" in str(value):
46         df.at[idx, "T18"] = 1
47     if "T13" in str(value) or "13" in str(value):
48         df.at[idx, "T13"] = 1
49
50 # 统计疾病分布
51 t21_count = df["T21"].sum()
52 t18_count = df["T18"].sum()
53 t13_count = df["T13"].sum()
54 normal_count = len(df) - (t21_count + t18_count + t13_count)
55
56 print(f"正常样本数: {normal_count}")
57 print(f"T21 (唐氏综合征) 样本数: {t21_count}")
58 print(f"T18 (爱德华氏综合征) 样本数: {t18_count}")
59 print(f"T13 (帕陶氏综合征) 样本数: {t13_count}")
60
61 # 3. 数据质量检查
62 print("\n正在进行数据质量检查...")
63
64 # 检查缺失值

```

```

65 missing_values = df.isnull().sum()
66 if missing_values.sum() > 0:
67     print("发现缺失值:")
68     print(missing_values[missing_values > 0])
69 else:
70     print("未发现缺失值")
71
72 # 检查重复行
73 duplicates = df.duplicated().sum()
74 if duplicates > 0:
75     print(f"发现 {duplicates} 个重复行，已移除")
76     df = df.drop_duplicates()
77
78 # 4. 数据质量控制 - 专门针对女胎数据
79 print("\n正在进行数据质量控制...")
80 initial_count = len(df)
81
82 # 应用质量控制阈值
83 qc_conditions = (
84     (df["原始读段数"] > 3500000)
85     & (df["在参考基因组上比对的比例"] > 0.75)
86     & (df["重复读段的比例"] < 0.35)
87     & (df["唯一比对的读段数"] / df["原始读段数"] > 0.7)
88     & (df["被过滤掉读段数的比例"] < 0.5)
89     &
90     # 对于女胎，X染色体浓度可能为负值，但其他质量指标正常即可
91     (df["GC含量"].between(0.39, 0.6)) # GC含量应在正常范围内
92     & (df["X染色体的Z值"] <= 3)
93     & (df["X染色体的Z值"] >= -3)
94 )
95
96 # 筛选高质量数据
97 hq_df = df[qc_conditions].copy()
98 filtered_count = initial_count - len(hq_df)
99
100 print(f"初始样本数: {initial_count}")
101 print(f"高质量样本数: {len(hq_df)}")
102 print(f"被过滤样本数: {filtered_count}")
103 print(f"过滤比例: {filtered_count/initial_count*100:.2f}%")
104
105 # 5. 保存处理后的数据
106 print("\n正在保存处理后的数据...")
107 try:
108     hq_df.to_excel(output_file_path, index=False)
109     print(f"处理后的数据已保存到: {output_file_path}")
110     print("处理完成!")
111
112 except Exception as e:
113     print(f"保存数据时出错: {e}")
114
115 # 使用示例
116 if __name__ == "__main__":
117     # 请修改为您的实际文件路径
118     input_file = "./python_code/附件.xlsx"

```

```

120     output_file = "./python_code/Q4/处理后的女胎NIPT数据.xlsx"
121
122     process_female_nipt_data(input_file, output_file)
123
124     # 初始样本数: 605
125     # 高质量样本数: 556
126     # 被过滤样本数: 49
127     # 过滤比例: 8.10%

```

Listing 10: 点二列相关系数 _smote_ 可视化 _final.py

```

1  import pandas as pd
2  import numpy as np
3  from scipy.stats import pointbiserialr
4  from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold,
   RandomizedSearchCV
5  from sklearn.preprocessing import StandardScaler
6  from sklearn.compose import ColumnTransformer
7  from sklearn.pipeline import Pipeline
8  from sklearn.ensemble import RandomForestClassifier
9  from xgboost import XGBClassifier
10 from sklearn.svm import SVC
11 from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, f1_score,
   make_scorer, accuracy_score, precision_score, recall_score, roc_curve, precision_recall_curve
12 from imblearn.over_sampling import SMOTE, ADASYN
13 from imblearn.combine import SMOTEENN
14 from collections import Counter
15 import matplotlib.pyplot as plt
16 import seaborn as sns
17 import warnings
18 warnings.filterwarnings('ignore')
19
20 # 设置中文字体
21 plt.rcParams['font.sans-serif'] = ['SimHei']
22 plt.rcParams['axes.unicode_minus'] = False
23
24 # 设置颜色方案
25 colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd', '#8c564b', '#e377c2', '#7f7f7f', '#
   bcbd22', '#17becf']
26
27 # 加载和预处理数据函数保持不变
28 def load_and_preprocess_data(file_path):
29     """加载并预处理数据"""
30     print("正在加载数据...")
31
32     try:
33         # 直接读取Excel文件
34         df = pd.read_excel(file_path)
35         print(f"成功读取数据, 形状: {df.shape}")
36
37     except Exception as e:
38         print(f"读取Excel文件时出错: {e}")
39         return None
40
41     # 处理缺失值

```

```

42     print("\n处理缺失值...")
43
44     # 删除Unnamed列 (如果存在)
45     columns_to_drop = [col for col in df.columns if 'Unnamed' in str(col)]
46     if columns_to_drop:
47         df = df.drop(columns=columns_to_drop)
48         print(f"已删除列: {columns_to_drop}")
49
50     # 处理孕妇BMI缺失值
51     if '孕妇BMI' in df.columns and df['孕妇BMI'].isnull().sum() > 0:
52         bmi_median = df['孕妇BMI'].median()
53         df['孕妇BMI'] = df['孕妇BMI'].fillna(bmi_median)
54         print(f"使用中位数 {bmi_median:.2f} 填充孕妇BMI缺失值")
55
56     # 创建目标变量 - 是否有任何染色体异常
57     if all(col in df.columns for col in ['T21', 'T18', 'T13']):
58         df['染色体异常'] = df[['T21', 'T18', 'T13']].max(axis=1)
59         print("使用现有的T21, T18, T13列创建目标变量")
60     elif '染色体的非整倍体' in df.columns:
61         # 如果没有T21, T18, T13列, 但有染色体的非整倍体列, 则创建它们
62         df['T21'] = 0
63         df['T18'] = 0
64         df['T13'] = 0
65
66         for idx, value in df['染色体的非整倍体'].items():
67             if pd.isna(value) or value == '':
68                 continue
69
70             value_str = str(value).upper()
71             if 'T21' in value_str or '21' in value_str:
72                 df.at[idx, 'T21'] = 1
73             if 'T18' in value_str or '18' in value_str:
74                 df.at[idx, 'T18'] = 1
75             if 'T13' in value_str or '13' in value_str:
76                 df.at[idx, 'T13'] = 1
77
78         df['染色体异常'] = df[['T21', 'T18', 'T13']].max(axis=1)
79         print("使用'染色体的非整倍体'列创建目标变量")
80     else:
81         print("错误: 无法找到创建目标变量所需的列")
82         return None
83
84     print(f"\n目标变量分布:")
85     print(df['染色体异常'].value_counts())
86     print(f"异常样本比例: {df['染色体异常'].mean()*100:.2f}%")
87
88     return df
89
90 # 准备特征函数保持不变
91 def prepare_features(df):
92     """准备特征矩阵和目标向量"""
93     # 可能的特征列 - 只使用数值型特征
94     possible_features = [
95         '孕妇年龄', '孕妇BMI', '怀孕次数', '生产次数', # 基本信息
96         'X染色体的Z值', '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值', # Z值

```

```

97         'GC含量', '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC含量', # GC相关
98         '原始读段数', '唯一比对的读段数', '重复读段的比例', '在参考基因组上比对的比例', '被过滤掉读段数
          的比例' # 测序质量
99     ]
100
101     # 只保留实际存在且为数值型的列
102     available_features = []
103     for col in possible_features:
104         if col in df.columns:
105             # 检查是否为数值型
106             if pd.api.types.is_numeric_dtype(df[col]):
107                 available_features.append(col)
108             else:
109                 print(f"警告: 列 {col} 不是数值型, 将被忽略")
110
111     print(f"\n使用的数值型特征: {available_features}")
112
113     if not available_features:
114         print("错误: 没有找到任何数值型特征列")
115         return None, None
116
117     X = df[available_features]
118     y = df['染色体异常']
119
120     # 确保所有特征都是数值型
121     for col in X.columns:
122         if not pd.api.types.is_numeric_dtype(X[col]):
123             print(f"错误: 列 {col} 不是数值型, 但被包含在特征中")
124             return None, None
125
126     return X, y
127
128 # 点二列相关系数筛选函数保持不变
129 def filter_features_point_biserial(X, y, threshold=0.1, top_k=None):
130     """
131     使用点二列相关系数进行初步特征筛选
132     """
133     print("\n=== 使用点二列相关系数进行初步特征筛选 ===")
134
135     # 标准化特征 (确保公平比较)
136     scaler = StandardScaler()
137     X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns, index=X.index)
138
139     correlations = []
140     p_values = []
141
142     # 计算每个特征与目标变量的点二列相关系数
143     for feature in X_scaled.columns:
144         # 移除缺失值
145         mask = ~(X_scaled[feature].isnull() | y.isnull())
146         x_vals = X_scaled.loc[mask, feature]
147         y_vals = y.loc[mask]
148
149         # 计算点二列相关系数
150         corr, p_val = pointbiserialr(x_vals, y_vals)

```

```

151     correlations.append(corr)
152     p_values.append(p_val)
153
154 # 创建结果DataFrame
155 corr_df = pd.DataFrame({
156     'feature': X_scaled.columns,
157     'correlation': correlations,
158     'p_value': p_values,
159     'abs_correlation': np.abs(correlations)
160 }).sort_values('abs_correlation', ascending=False)
161
162 print("特征与目标变量的点二列相关系数:")
163 print(corr_df)
164
165 # 绘制相关系数图（改进版，与selectedCode风格一致）
166 fig, ax = plt.subplots(figsize=(12, 8))
167
168 y_pos = np.arange(len(corr_df))
169 bars = ax.barh(y_pos, corr_df['correlation'], color=colors[0], alpha=0.8)
170 ax.set_yticks(y_pos)
171 ax.set_yticklabels(corr_df['feature'])
172 ax.set_xlabel('点二列相关系数')
173 ax.set_title('特征与目标变量的点二列相关系数（标准化后）')
174 ax.grid(True, alpha=0.3)
175
176 # 在柱子上添加数值
177 for i, (bar, corr) in enumerate(zip(bars, corr_df['correlation'])):
178     ax.text(corr + (0.01 if corr >= 0 else -0.01), i, f'{corr:.3f}',
179            ha='left' if corr >= 0 else 'right', va='center', fontsize=9)
180
181 plt.tight_layout()
182 plt.savefig('point_biserial_correlations.png')
183 plt.close()
184
185 # 绘制相关系数绝对值的条形图
186 fig, ax = plt.subplots(figsize=(12, 8))
187 bars = ax.bar(range(len(corr_df)), corr_df['abs_correlation'],
188             color=colors[1], alpha=0.8)
189 ax.set_xlabel('特征')
190 ax.set_ylabel('点二列相关系数绝对值')
191 ax.set_title('特征与目标变量的点二列相关系数绝对值')
192 ax.set_xticks(range(len(corr_df)))
193 ax.set_xticklabels(corr_df['feature'], rotation=45, ha='right')
194 ax.grid(True, alpha=0.3)
195
196 # 在柱子上添加数值
197 for bar, abs_corr in zip(bars, corr_df['abs_correlation']):
198     height = bar.get_height()
199     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
200            f'{abs_corr:.3f}', ha='center', va='bottom', fontsize=9)
201
202 plt.tight_layout()
203 plt.savefig('point_biserial_correlations_abs.png')
204 plt.close()
205

```

```

206 # 根据阈值或top_k筛选特征
207 if top_k is not None:
208     selected_features = corr_df.head(top_k)['feature'].tolist()
209     print(f"\n选择前 {top_k} 个最相关的特征: {selected_features}")
210 else:
211     selected_features = corr_df[corr_df['abs_correlation'] >= threshold]['feature'].tolist()
212     print(f"\n选择相关系数绝对值 >= {threshold} 的特征: {selected_features}")
213
214 if not selected_features:
215     print("警告: 没有特征满足筛选条件, 使用所有特征")
216     return X
217
218 return X[selected_features] # 返回原始数据, 标准化只在筛选时使用
219
220 # 处理不平衡数据函数保持不变
221 def handle_imbalanced_data(X, y, method='smote'):
222     """处理不平衡数据"""
223     print(f"\n处理不平衡数据 - 方法: {method}")
224     print(f"原始数据分布: {Counter(y)}")
225
226     # 确保X是数值型
227     if not all(pd.api.types.is_numeric_dtype(X[col]) for col in X.columns):
228         print("错误: 特征矩阵包含非数值型数据")
229         return X, y
230
231     if method == 'smote':
232         sampler = SMOTE(random_state=42)
233     elif method == 'adasyn':
234         sampler = ADASYN(random_state=42)
235     elif method == 'smoteenn':
236         sampler = SMOTEENN(random_state=42)
237     else:
238         return X, y # 不处理
239
240     X_res, y_res = sampler.fit_resample(X, y)
241     print(f"处理后数据分布: {Counter(y_res)}")
242
243     return X_res, y_res
244
245 # 新增函数: 绘制交叉验证结果
246 def plot_cv_results(cv_results, model_name, scoring_metrics):
247     """绘制交叉验证结果"""
248     n_folds = len(cv_results['test_accuracy'])
249     x = np.arange(n_folds)
250     width = 0.15
251
252     fig, ax = plt.subplots(figsize=(12, 8))
253
254     for i, metric in enumerate(scoring_metrics):
255         scores = cv_results[f'test_{metric}']
256         ax.bar(x + i*width, scores, width, label=metric, alpha=0.8)
257
258     ax.set_xlabel('交叉验证折数')
259     ax.set_ylabel('分数')
260     ax.set_title(f'{model_name} 交叉验证结果')

```

```

261     ax.set_xticks(x + width*2)
262     ax.set_xticklabels([f'Fold {i+1}' for i in range(n_folds)])
263     ax.legend(loc='lower right')
264     ax.grid(True, alpha=0.3)
265
266     plt.tight_layout()
267     plt.savefig(f'{model_name}_cv_results.png')
268     plt.close()
269
270     # 绘制箱线图
271     fig, ax = plt.subplots(figsize=(10, 6))
272     scores_data = [cv_results[f'test_{metric}'] for metric in scoring_metrics]
273     ax.boxplot(scores_data, labels=scoring_metrics)
274     ax.set_title(f'{model_name} 交叉验证分数分布')
275     ax.set_ylabel('分数')
276     ax.grid(True, alpha=0.3)
277
278     plt.tight_layout()
279     plt.savefig(f'{model_name}_cv_boxplot.png')
280     plt.close()
281
282     # 新增函数：绘制ROC曲线比较
283     def plot_roc_curves(models, X_test, y_test, model_names):
284         """绘制ROC曲线比较"""
285         plt.figure(figsize=(10, 8))
286
287         for i, model in enumerate(models):
288             if hasattr(model, 'predict_proba'):
289                 y_prob = model.predict_proba(X_test)[: , 1]
290             else:
291                 y_prob = model.decision_function(X_test)
292
293             fpr, tpr, _ = roc_curve(y_test, y_prob)
294             auc_score = roc_auc_score(y_test, y_prob)
295
296             plt.plot(fpr, tpr, label=f'{model_names[i]} (AUC = {auc_score:.3f})', color=colors[i], linewidth
                =2)
297
298             plt.plot([0, 1], [0, 1], 'k--', label='随机猜测 (AUC = 0.5)')
299             plt.xlabel('假正率')
300             plt.ylabel('真正率')
301             plt.title('ROC曲线比较')
302             plt.legend(loc='lower right')
303             plt.grid(True, alpha=0.3)
304
305             plt.tight_layout()
306             plt.savefig('roc_curves_comparison.png')
307             plt.close()
308
309     # 新增函数：绘制精确率-召回率曲线比较
310     def plot_precision_recall_curves(models, X_test, y_test, model_names):
311         """绘制精确率-召回率曲线比较"""
312         plt.figure(figsize=(10, 8))
313
314         for i, model in enumerate(models):

```

```

315         if hasattr(model, 'predict_proba'):
316             y_prob = model.predict_proba(X_test)[:, 1]
317         else:
318             y_prob = model.decision_function(X_test)
319
320         precision, recall, _ = precision_recall_curve(y_test, y_prob)
321         avg_precision = np.mean(precision)
322
323         plt.plot(recall, precision, label=f'{model_names[i]} (AP = {avg_precision:.3f})', color=colors[i], linewidth=2)
324
325     plt.xlabel('召回率')
326     plt.ylabel('精确率')
327     plt.title('精确率-召回率曲线比较')
328     plt.legend(loc='upper right')
329     plt.grid(True, alpha=0.3)
330
331     plt.tight_layout()
332     plt.savefig('precision_recall_curves_comparison.png')
333     plt.close()
334
335 # 修改后的训练和评估函数
336 def train_and_evaluate_model(X, y, model_name='random_forest', use_cross_validation=True):
337     """训练和评估模型"""
338     # 检查数据是否有效
339     if X is None or y is None or len(X) == 0 or len(y) == 0:
340         print("错误: 无效的数据")
341         return None, 0, 0, 0, 0, {}
342
343     # 划分训练集和测试集
344     X_train, X_test, y_train, y_test = train_test_split(
345         X, y, test_size=0.3, random_state=42, stratify=y
346     )
347
348     # 选择模型
349     if model_name == 'random_forest':
350         model = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
351     elif model_name == 'xgboost':
352         # 计算正负样本比例
353         scale_pos_weight = len(y[y==0])/len(y[y==1]) if len(y[y==1]) > 0 else 1
354         model = XGBClassifier(random_state=42, scale_pos_weight=scale_pos_weight)
355     elif model_name == 'svm':
356         model = SVC(random_state=42, class_weight='balanced', probability=True)
357     else:
358         model = RandomForestClassifier(random_state=42)
359
360     # 创建预处理管道
361     numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
362     numeric_transformer = Pipeline(steps=[
363         ('scaler', StandardScaler())
364     ])
365
366     preprocessor = ColumnTransformer(
367         transformers=[
368             ('num', numeric_transformer, numeric_features)

```

```

369     ])
370
371     # 创建完整管道
372     clf = Pipeline(steps=[
373         ('preprocessor', preprocessor),
374         ('classifier', model)
375     ])
376
377     cv_results = {}
378     # 使用交叉验证
379     if use_cross_validation:
380         print(f"\n对 {model_name} 进行交叉验证...")
381
382         # 使用分层K折交叉验证
383         cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
384
385         # 定义评分指标
386         scoring_metrics = ['accuracy', 'f1', 'roc_auc', 'precision', 'recall']
387
388         # 执行交叉验证
389         for metric in scoring_metrics:
390             cv_scores = cross_val_score(clf, X, y, cv=cv, scoring=metric)
391             cv_results[f'test_{metric}'] = cv_scores
392             print(f"{model_name} 交叉验证 {metric}: {cv_scores.mean():.4f} (±{cv_scores.std():.4f})")
393
394         # 绘制交叉验证结果
395         plot_cv_results(cv_results, model_name, scoring_metrics)
396
397     # 训练最终模型
398     clf.fit(X_train, y_train)
399
400     # 预测
401     y_pred = clf.predict(X_test)
402     y_prob = clf.predict_proba(X_test)[:, 1]
403
404     # 评估模型
405     print(f"\n{model_name} 模型评估:")
406     print(classification_report(y_test, y_pred))
407
408     # 计算各种评估指标
409     accuracy = accuracy_score(y_test, y_pred)
410     precision = precision_score(y_test, y_pred)
411     recall = recall_score(y_test, y_pred)
412     auc_score = roc_auc_score(y_test, y_prob)
413     f1 = f1_score(y_test, y_pred)
414
415     print(f"准确率: {accuracy:.4f}")
416     print(f"精确率: {precision:.4f}")
417     print(f"召回率: {recall:.4f}")
418     print(f"AUC: {auc_score:.4f}")
419     print(f"F1分数: {f1:.4f}")
420
421     # 绘制混淆矩阵
422     cm = confusion_matrix(y_test, y_pred)
423     plt.figure(figsize=(6, 5))

```

```

424     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
425     plt.title(f'{model_name} 混淆矩阵')
426     plt.ylabel('真实标签')
427     plt.xlabel('预测标签')
428     plt.tight_layout()
429     plt.savefig(f'{model_name}_confusion_matrix.png')
430     plt.close()
431
432     # 绘制ROC曲线
433     fpr, tpr, _ = roc_curve(y_test, y_prob)
434     plt.figure(figsize=(8, 6))
435     plt.plot(fpr, tpr, color=colors[0], lw=2, label=f'ROC曲线 (AUC = {auc_score:.3f})')
436     plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='随机猜测')
437     plt.xlim([0.0, 1.0])
438     plt.ylim([0.0, 1.05])
439     plt.xlabel('假正率')
440     plt.ylabel('真正率')
441     plt.title(f'{model_name} ROC曲线')
442     plt.legend(loc='lower right')
443     plt.grid(True, alpha=0.3)
444     plt.tight_layout()
445     plt.savefig(f'{model_name}_roc_curve.png')
446     plt.close()
447
448     # 绘制精确率-召回率曲线
449     precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_prob)
450     plt.figure(figsize=(8, 6))
451     plt.plot(recall_vals, precision_vals, color=colors[1], lw=2, label=f'精确率-召回率曲线')
452     plt.xlabel('召回率')
453     plt.ylabel('精确率')
454     plt.title(f'{model_name} 精确率-召回率曲线')
455     plt.legend(loc='upper right')
456     plt.grid(True, alpha=0.3)
457     plt.tight_layout()
458     plt.savefig(f'{model_name}_precision_recall_curve.png')
459     plt.close()
460
461     return clf, auc_score, f1, accuracy, recall, cv_results
462
463 # 新增函数：绘制模型指标比较图
464 def plot_metrics_comparison(metrics_df):
465     """绘制模型指标比较图"""
466     metrics = metrics_df.columns[1:] # 排除模型名列
467
468     # 计算需要的行数和列数
469     n_metrics = len(metrics)
470     n_cols = min(3, n_metrics) # 每行最多3列
471     n_rows = (n_metrics + n_cols - 1) // n_cols # 向上取整
472
473     fig, axes = plt.subplots(n_rows, n_cols, figsize=(5*n_cols, 5*n_rows))
474
475     # 如果只有一行或一列，确保axes是二维数组
476     if n_rows == 1 and n_cols == 1:
477         axes = np.array([[axes]])
478     elif n_rows == 1:

```

```

479     axes = axes.reshape(1, -1)
480 elif n_cols == 1:
481     axes = axes.reshape(-1, 1)
482
483 # 绘制每个指标的条形图
484 for i, metric in enumerate(metrics):
485     row = i // n_cols
486     col = i % n_cols
487     ax = axes[row, col]
488
489     bars = ax.bar(range(len(metrics_df)), metrics_df[metric],
490                   color=colors[:len(metrics_df)], alpha=0.8)
491     ax.set_xlabel('模型')
492     ax.set_ylabel(metric)
493     ax.set_title(f'{metric} 比较')
494     ax.set_xticks(range(len(metrics_df)))
495     ax.set_xticklabels(metrics_df['模型'], rotation=45)
496
497 # 在柱子上添加数值
498 for bar in bars:
499     height = bar.get_height()
500     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
501             f'{height:.3f}', ha='center', va='bottom', fontsize=9)
502
503     ax.grid(True, alpha=0.3)
504
505 # 隐藏多余的子图
506 for i in range(n_metrics, n_rows * n_cols):
507     row = i // n_cols
508     col = i % n_cols
509     axes[row, col].set_visible(False)
510
511 plt.tight_layout()
512 plt.savefig('metrics_comparison.png')
513 plt.close()
514
515 # 绘制雷达图
516 fig = plt.figure(figsize=(10, 10))
517 ax = fig.add_subplot(111, polar=True)
518
519 # 准备数据
520 categories = list(metrics)
521 N = len(categories)
522
523 # 计算每个模型的角度
524 angles = [n / float(N) * 2 * np.pi for n in range(N)]
525 angles += angles[:1] # 闭合雷达图
526
527 # 绘制每个模型的雷达图
528 for i, row in metrics_df.iterrows():
529     values = row[metrics].values.flatten().tolist()
530     values += values[:1] # 闭合雷达图
531     ax.plot(angles, values, linewidth=2, linestyle='solid',
532            label=row['模型'], color=colors[i])
533     ax.fill(angles, values, alpha=0.1, color=colors[i])

```

```

534
535 # 添加类别标签
536 plt.xticks(angles[:-1], categories)
537
538 # 添加图例
539 plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
540
541 # 添加标题
542 plt.title('模型性能雷达图比较', size=20, y=1.05)
543
544 plt.tight_layout()
545 plt.savefig('radar_chart_comparison.png')
546 plt.close()
547
548 # 修改后的比较采样方法函数
549 def compare_sampling_methods(X, y):
550     """比较不同的采样方法"""
551     methods = ['none', 'smote', 'smoteenn', 'adasyn']
552     results = []
553
554     for method in methods:
555         print(f"\n=== 使用方法: {method} ===")
556         X_res, y_res = handle_imbalanced_data(X, y, method)
557
558         # 使用随机森林评估每种方法
559         model, auc, f1, accuracy, recall, _ = train_and_evaluate_model(X_res, y_res, 'random_forest',
560                                 use_cross_validation=False)
561         results.append({
562             'method': method,
563             'auc': auc,
564             'f1': f1,
565             'accuracy': accuracy,
566             'recall': recall,
567             'distribution': Counter(y_res)
568         })
569
570 # 比较结果
571 results_df = pd.DataFrame(results)
572 print("\n=== 不同采样方法比较 ===")
573 print(results_df[['method', 'accuracy', 'recall', 'auc', 'f1']])
574
575 # 绘制比较图 (改进版, 与selectedCode风格一致)
576 fig, ax = plt.subplots(figsize=(12, 8))
577 x_pos = np.arange(len(results_df))
578 width = 0.35
579
580 bars1 = ax.bar(x_pos - width/2, results_df['auc'], width, label='AUC', color=colors[0], alpha=0.8)
581 bars2 = ax.bar(x_pos + width/2, results_df['f1'], width, label='F1 Score', color=colors[1], alpha
582               =0.8)
583
584 ax.set_xlabel('采样方法')
585 ax.set_ylabel('分数')
586 ax.set_title('不同采样方法性能比较')
587 ax.set_xticks(x_pos)
588 ax.set_xticklabels(results_df['method'])

```

```

587 ax.legend()
588 ax.grid(True, alpha=0.3)
589
590 # 在柱子上添加数值
591 for bar, auc_val in zip(bars1, results_df['auc']):
592     height = bar.get_height()
593     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
594             f'{auc_val:.3f}', ha='center', va='bottom', fontsize=9)
595
596 for bar, f1_val in zip(bars2, results_df['f1']):
597     height = bar.get_height()
598     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
599             f'{f1_val:.3f}', ha='center', va='bottom', fontsize=9)
600
601 plt.tight_layout()
602 plt.savefig('sampling_methods_comparison.png')
603 plt.close()
604
605 return results_df
606
607 # 超参数调优函数保持不变
608 def hyperparameter_tuning(X, y, model_name='random_forest'):
609     """超参数调优"""
610     print(f"\n=== 对 {model_name} 进行超参数调优 ===")
611
612     # 划分训练集和测试集
613     X_train, X_test, y_train, y_test = train_test_split(
614         X, y, test_size=0.3, random_state=42, stratify=y
615     )
616
617     # 创建预处理管道
618     numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
619     numeric_transformer = Pipeline(steps=[
620         ('scaler', StandardScaler())
621     ])
622
623     preprocessor = ColumnTransformer(
624         transformers=[
625             ('num', numeric_transformer, numeric_features)
626         ])
627
628     # 根据模型选择不同的参数网格
629     if model_name == 'random_forest':
630         model = RandomForestClassifier(random_state=42, class_weight='balanced')
631         param_dist = {
632             'classifier__n_estimators': [50, 100, 200],
633             'classifier__max_depth': [None, 10, 20, 30],
634             'classifier__min_samples_split': [2, 5, 10],
635             'classifier__min_samples_leaf': [1, 2, 4]
636         }
637     elif model_name == 'xgboost':
638         scale_pos_weight = len(y[y==0])/len(y[y==1]) if len(y[y==1]) > 0 else 1
639         model = XGBClassifier(random_state=42, scale_pos_weight=scale_pos_weight)
640         param_dist = {
641             'classifier__n_estimators': [50, 100, 200],

```

```

642         'classifier__max_depth': [3, 6, 9],
643         'classifier__learning_rate': [0.01, 0.1, 0.2],
644         'classifier__subsample': [0.8, 0.9, 1.0]
645     }
646 elif model_name == 'svm':
647     model = SVC(random_state=42, class_weight='balanced', probability=True)
648     param_dist = {
649         'classifier__C': [0.1, 1, 10, 100],
650         'classifier__gamma': ['scale', 'auto', 0.01, 0.1, 1],
651         'classifier__kernel': ['rbf', 'linear']
652     }
653 else:
654     print(f"暂不支持 {model_name} 的超参数调优")
655     return None
656
657 # 创建完整管道
658 clf = Pipeline(steps=[
659     ('preprocessor', preprocessor),
660     ('classifier', model)
661 ])
662
663 # 使用随机搜索进行超参数调优
664 random_search = RandomizedSearchCV(
665     clf, param_distributions=param_dist, n_iter=20,
666     cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
667     scoring='f1', random_state=42, n_jobs=-1
668 )
669
670 # 执行随机搜索
671 random_search.fit(X_train, y_train)
672
673 # 输出最佳参数
674 print(f"最佳参数: {random_search.best_params_}")
675 print(f"最佳交叉验证分数: {random_search.best_score_:.4f}")
676
677 # 在测试集上评估最佳模型
678 best_model = random_search.best_estimator_
679 y_pred = best_model.predict(X_test)
680 y_prob = best_model.predict_proba(X_test)[:, 1]
681
682 # 计算评估指标
683 accuracy = accuracy_score(y_test, y_pred)
684 precision = precision_score(y_test, y_pred)
685 recall = recall_score(y_test, y_pred)
686 auc_score = roc_auc_score(y_test, y_prob)
687 f1 = f1_score(y_test, y_pred)
688
689 print(f"测试集准确率: {accuracy:.4f}")
690 print(f"测试集精确率: {precision:.4f}")
691 print(f"测试集召回率: {recall:.4f}")
692 print(f"测试集 AUC: {auc_score:.4f}")
693 print(f"测试集 F1 分数: {f1:.4f}")
694
695 # 绘制混淆矩阵
696 cm = confusion_matrix(y_test, y_pred)

```

```

697 plt.figure(figsize=(6, 5))
698 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
699 plt.title(f'{model_name} 超参数调优后混淆矩阵')
700 plt.ylabel('真实标签')
701 plt.xlabel('预测标签')
702 plt.tight_layout()
703 plt.savefig(f'{model_name}_tuned_confusion_matrix.png')
704 plt.close()
705
706 # 绘制ROC曲线
707 fpr, tpr, _ = roc_curve(y_test, y_prob)
708 plt.figure(figsize=(8, 6))
709 plt.plot(fpr, tpr, color=colors[0], lw=2, label=f'ROC曲线 (AUC = {auc_score:.3f})')
710 plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='随机猜测')
711 plt.xlim([0.0, 1.0])
712 plt.ylim([0.0, 1.05])
713 plt.xlabel('假正率')
714 plt.ylabel('真正率')
715 plt.title(f'{model_name} 超参数调优后 ROC曲线')
716 plt.legend(loc='lower right')
717 plt.grid(True, alpha=0.3)
718 plt.tight_layout()
719 plt.savefig(f'{model_name}_tuned_roc_curve.png')
720 plt.close()
721
722 return best_model, {'accuracy': accuracy, 'precision': precision, 'recall': recall, 'auc': auc_score
723                     , 'f1': f1}
724
725 # 嵌入式特征选择函数保持不变
726 def embedded_feature_selection(X, y, model_name='random_forest', top_k=10):
727     """
728     使用嵌入式方法进行特征选择（基于树模型的特征重要性）
729     """
730     print(f"\n=== 使用 {model_name} 进行嵌入式特征选择 ===")
731
732     # 划分训练集和测试集
733     X_train, X_test, y_train, y_test = train_test_split(
734         X, y, test_size=0.3, random_state=42, stratify=y
735     )
736
737     # 选择模型
738     if model_name == 'random_forest':
739         model = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
740     elif model_name == 'xgboost':
741         scale_pos_weight = len(y[y==0])/len(y[y==1]) if len(y[y==1]) > 0 else 1
742         model = XGBClassifier(random_state=42, scale_pos_weight=scale_pos_weight)
743     else:
744         print(f"暂不支持 {model_name} 的嵌入式特征选择")
745         return X
746
747     # 创建预处理管道
748     numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
749     numeric_transformer = Pipeline(steps=[
750         ('scaler', StandardScaler())
751     ])

```

```

751
752 preprocessor = ColumnTransformer(
753     transformers=[
754         ('num', numeric_transformer, numeric_features)
755     ])
756
757 # 创建完整管道
758 clf = Pipeline(steps=[
759     ('preprocessor', preprocessor),
760     ('classifier', model)
761 ])
762
763 # 训练模型
764 clf.fit(X_train, y_train)
765
766 # 获取特征重要性
767 if hasattr(clf.named_steps['classifier'], 'feature_importances_'):
768     feature_importances = clf.named_steps['classifier'].feature_importances_
769     features = X.columns
770
771     # 创建特征重要性DataFrame
772     importance_df = pd.DataFrame({
773         'feature': features,
774         'importance': feature_importances
775     }).sort_values('importance', ascending=False)
776
777     print("特征重要性排名:")
778     print(importance_df)
779
780     # 绘制特征重要性图（改进版，与selectedCode风格一致）
781     fig, ax = plt.subplots(figsize=(12, 8))
782     y_pos = np.arange(len(importance_df.head(15)))
783     bars = ax.barh(y_pos, importance_df['importance'].head(15), color=colors[2], alpha=0.8)
784     ax.set_yticks(y_pos)
785     ax.set_yticklabels(importance_df['feature'].head(15))
786     ax.set_xlabel('重要性')
787     ax.set_title(f'{model_name} 特征重要性')
788     ax.grid(True, alpha=0.3)
789
790     # 在柱子上添加数值
791     for i, (bar, importance) in enumerate(zip(bars, importance_df['importance'].head(15))):
792         ax.text(importance + 0.001, i, f'{importance:.3f}',
793             ha='left', va='center', fontsize=9)
794
795     plt.tight_layout()
796     plt.savefig(f'{model_name}_feature_importance.png')
797     plt.close()
798
799     # 绘制特征重要性条形图
800     fig, ax = plt.subplots(figsize=(12, 8))
801     x_pos = np.arange(len(importance_df.head(15)))
802     bars = ax.bar(x_pos, importance_df['importance'].head(15), color=colors[3], alpha=0.8)
803     ax.set_xlabel('特征')
804     ax.set_ylabel('重要性')
805     ax.set_title(f'{model_name} 特征重要性条形图')

```

```

806     ax.set_xticks(x_pos)
807     ax.set_xticklabels(importance_df['feature'].head(15), rotation=45, ha='right')
808     ax.grid(True, alpha=0.3)
809
810     # 在柱子上添加数值
811     for bar, importance in zip(bars, importance_df['importance'].head(15)):
812         height = bar.get_height()
813         ax.text(bar.get_x() + bar.get_width()/2., height + 0.001,
814               f'{importance:.3f}', ha='center', va='bottom', fontsize=9)
815
816     plt.tight_layout()
817     plt.savefig(f'{model_name}_feature_importance_bar.png')
818     plt.close()
819
820     # 选择前k个最重要的特征
821     selected_features = importance_df.head(top_k)['feature'].tolist()
822     print(f"\n选择前 {top_k} 个最重要的特征: {selected_features}")
823
824     return X[selected_features]
825 else:
826     print(f'{model_name} 模型不支持特征重要性分析')
827     return X
828
829 # 修改后的模型准确率和召回率比较图函数
830 def plot_model_metrics_comparison(model_results_df):
831     """绘制模型准确率和召回率比较图"""
832     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
833
834     # 准确率比较
835     bars1 = ax1.bar(range(len(model_results_df)), model_results_df['accuracy'],
836                   color=colors[0], alpha=0.8)
837     ax1.set_xlabel('模型')
838     ax1.set_ylabel('准确率')
839     ax1.set_title('不同模型准确率比较')
840     ax1.set_xticks(range(len(model_results_df)))
841     # 修改这里: 使用 '模型' 而不是 'model'
842     ax1.set_xticklabels(model_results_df['模型'])
843     ax1.grid(True, alpha=0.3)
844
845     # 在柱子上添加数值
846     for bar, acc_val in zip(bars1, model_results_df['accuracy']):
847         height = bar.get_height()
848         ax1.text(bar.get_x() + bar.get_width()/2., height + 0.01,
849               f'{acc_val:.3f}', ha='center', va='bottom', fontsize=9)
850
851     # 召回率比较
852     bars2 = ax2.bar(range(len(model_results_df)), model_results_df['recall'],
853                   color=colors[1], alpha=0.8)
854     ax2.set_xlabel('模型')
855     ax2.set_ylabel('召回率')
856     ax2.set_title('不同模型召回率比较')
857     ax2.set_xticks(range(len(model_results_df)))
858     # 修改这里: 使用 '模型' 而不是 'model'
859     ax2.set_xticklabels(model_results_df['模型'])
860     ax2.grid(True, alpha=0.3)

```

```

861
862 # 在柱子上添加数值
863 for bar, rec_val in zip(bars2, model_results_df['recall']):
864     height = bar.get_height()
865     ax2.text(bar.get_x() + bar.get_width()/2., height + 0.01,
866             f'{rec_val:.3f}', ha='center', va='bottom', fontsize=9)
867
868 plt.tight_layout()
869 plt.savefig('model_accuracy_recall_comparison.png')
870 plt.close()
871
872 # 绘制准确率和召回率的组合图
873 fig, ax = plt.subplots(figsize=(10, 6))
874 x_pos = np.arange(len(model_results_df))
875 width = 0.35
876
877 bars1 = ax.bar(x_pos - width/2, model_results_df['accuracy'], width,
878               label='准确率', color=colors[0], alpha=0.8)
879 bars2 = ax.bar(x_pos + width/2, model_results_df['recall'], width,
880               label='召回率', color=colors[1], alpha=0.8)
881
882 ax.set_xlabel('模型')
883 ax.set_ylabel('分数')
884 ax.set_title('模型准确率和召回率比较')
885 ax.set_xticks(x_pos)
886 # 修改这里：使用 '模型' 而不是 'model'
887 ax.set_xticklabels(model_results_df['模型'])
888 ax.legend()
889 ax.grid(True, alpha=0.3)
890
891 # 在柱子上添加数值
892 for bar, acc_val in zip(bars1, model_results_df['accuracy']):
893     height = bar.get_height()
894     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
895            f'{acc_val:.3f}', ha='center', va='bottom', fontsize=9)
896
897 for bar, rec_val in zip(bars2, model_results_df['recall']):
898     height = bar.get_height()
899     ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
900            f'{rec_val:.3f}', ha='center', va='bottom', fontsize=9)
901
902 plt.tight_layout()
903 plt.savefig('model_accuracy_recall_combined.png')
904 plt.close()
905
906 # 修改后的最终模型比较图函数
907 def plot_final_model_comparison(model_results_df):
908     """绘制最终模型比较图（与selectedCode风格一致）"""
909     metrics = ['auc', 'f1', 'accuracy', 'precision', 'recall']
910
911     fig, ax = plt.subplots(figsize=(12, 8))
912     x_pos = np.arange(len(model_results_df))
913     width = 0.15
914
915     for i, metric in enumerate(metrics):

```

```

916         ax.bar(x_pos + i*width, model_results_df[metric], width, label=metric, color=colors[i], alpha
917                =0.8)
918
919     ax.set_xlabel('模型')
920     ax.set_ylabel('分数')
921     ax.set_title('不同模型性能比较')
922     ax.set_xticks(x_pos + width*2)
923     # 修改这里: 使用 '模型' 而不是 'model'
924     ax.set_xticklabels(model_results_df['模型'])
925     ax.legend()
926     ax.grid(True, alpha=0.3)
927
928     plt.tight_layout()
929     plt.savefig('model_comparison.png')
930     plt.close()
931
932 # 修改后的主函数
933 def main():
934     """主函数"""
935     # 文件路径
936     input_file = "./python_code/Q4/处理后的女胎NIPT数据.xlsx"
937
938     # 加载和预处理数据
939     df = load_and_preprocess_data(input_file)
940     if df is None:
941         print("数据加载失败, 程序退出")
942         return
943
944     # 准备特征和目标
945     X, y = prepare_features(df)
946     if X is None or y is None:
947         print("特征准备失败, 程序退出")
948         return
949
950     # 检查是否有任何非数值型数据
951     print("\n检查数据类型...")
952     for col in X.columns:
953         if not pd.api.types.is_numeric_dtype(X[col]):
954             print(f"警告: 列 {col} 包含非数值型数据, 将被删除")
955             X = X.drop(columns=[col])
956
957     # 如果删除了列, 重新检查
958     if len(X.columns) == 0:
959         print("错误: 没有可用的数值型特征列")
960         return
961
962     print(f"初始特征: {list(X.columns)}")
963
964     # 第一步: 初步筛选 - 使用点二列相关系数
965     X_filtered = filter_features_point_biserial(X, y, threshold=0.05, top_k=15)
966
967     # 第二步: 精细筛选 - 使用嵌入式方法 (基于树模型的特征重要性)
968     # X_final = embedded_feature_selection(X_filtered, y, model_name='random_forest', top_k=10)
969     X_final = X_filtered

```

```

970 print(f"\n最终选择的特征: {list(X_final.columns)}")
971
972 # 比较不同的采样方法
973 results_df = compare_sampling_methods(X_final, y)
974
975 # 选择最佳方法
976 best_method = results_df.loc[results_df['f1'].idxmax(), 'method']
977 print(f"\n最佳采样方法: {best_method}")
978
979 # 应用最佳采样方法
980 X_res, y_res = handle_imbalanced_data(X_final, y, best_method)
981
982 # 训练不同模型并进行交叉验证
983 # 在 main() 函数中找到构建 model_results 的部分, 修改如下:
984
985 # 训练不同模型并进行交叉验证
986 models = ['random_forest', 'xgboost', 'svm']
987 trained_models = []
988 model_results = []
989 model_names = []
990
991 for model_name in models:
992     print(f"\n=== 训练 {model_name} 模型 ===")
993     model, auc, f1, accuracy, recall, cv_results = train_and_evaluate_model(X_res, y_res, model_name
994                                     , use_cross_validation=True)
995
996     # 计算精确率
997     X_train, X_test, y_train, y_test = train_test_split(
998         X_res, y_res, test_size=0.3, random_state=42, stratify=y_res
999     )
1000     y_pred = model.predict(X_test)
1001     precision = precision_score(y_test, y_pred)
1002
1003     model_results.append({
1004         '模型': model_name,
1005         'auc': auc,
1006         'f1': f1,
1007         'accuracy': accuracy,
1008         'precision': precision, # 这里应该是计算出的精度值, 而不是函数
1009         'recall': recall
1010     })
1011     trained_models.append(model)
1012     model_names.append(model_name)
1013
1014 # 创建模型指标DataFrame
1015 model_results_df = pd.DataFrame(model_results)
1016 print("\n=== 不同模型性能比较 ===")
1017 print(model_results_df)
1018
1019 # 绘制模型指标比较图
1020 plot_metrics_comparison(model_results_df)
1021
1022 # 绘制ROC曲线比较
1023 X_train, X_test, y_train, y_test = train_test_split(
1024     X_res, y_res, test_size=0.3, random_state=42, stratify=y_res

```

```

1024 )
1025 plot_roc_curves(trained_models, X_test, y_test, model_names)
1026
1027 # 绘制精确率-召回率曲线比较
1028 plot_precision_recall_curves(trained_models, X_test, y_test, model_names)
1029
1030 # 绘制模型比较图
1031 plot_final_model_comparison(model_results_df)
1032
1033 # 绘制准确率和召回率比较图
1034 plot_model_metrics_comparison(model_results_df)
1035
1036 # 选择最佳模型
1037 best_model_name = model_results_df.loc[model_results_df['f1'].idxmax(), '模型']
1038 print(f"\n最佳模型: {best_model_name}")
1039
1040 # 对最佳模型进行超参数调优
1041 best_model_tuned, tuned_metrics = hyperparameter_tuning(X_res, y_res, best_model_name)
1042
1043 # 重新训练最终模型
1044 print(f"\n=== 训练最终模型: {best_model_name} ===")
1045 final_model, final_auc, final_f1, final_accuracy, final_recall, _ = train_and_evaluate_model(X_res,
1046     y_res, best_model_name, use_cross_validation=True)
1047
1048 # 特征重要性 (如果适用)
1049 if hasattr(final_model.named_steps['classifier'], 'feature_importances_'):
1050     feature_importances = final_model.named_steps['classifier'].feature_importances_
1051     features = X_final.columns
1052
1053 # 创建特征重要性DataFrame
1054 importance_df = pd.DataFrame({
1055     'feature': features,
1056     'importance': feature_importances
1057 }).sort_values('importance', ascending=False)
1058
1059 print("\n最终模型特征重要性:")
1060 print(importance_df)
1061
1062 # 绘制特征重要性图 (改进版, 与selectedCode风格一致)
1063 fig, ax = plt.subplots(figsize=(12, 8))
1064 y_pos = np.arange(len(importance_df.head(15)))
1065 bars = ax.barh(y_pos, importance_df['importance'].head(15), color=colors[4], alpha=0.8)
1066 ax.set_yticks(y_pos)
1067 ax.set_yticklabels(importance_df['feature'].head(15))
1068 ax.set_xlabel('重要性')
1069 ax.set_title('最终模型 - Top 15 特征重要性')
1070 ax.grid(True, alpha=0.3)
1071
1072 # 在柱子上添加数值
1073 for i, (bar, importance) in enumerate(zip(bars, importance_df['importance'].head(15))):
1074     ax.text(importance + 0.001, i, f'{importance:.3f}',
1075         ha='left', va='center', fontsize=9)
1076
1077 plt.tight_layout()
1078 plt.savefig('final_feature_importance.png')

```

```

1078     plt.close()
1079
1080     # 绘制特征重要性条形图
1081     fig, ax = plt.subplots(figsize=(12, 8))
1082     x_pos = np.arange(len(importance_df.head(15)))
1083     bars = ax.bar(x_pos, importance_df['importance'].head(15), color=colors[5], alpha=0.8)
1084     ax.set_xlabel('特征')
1085     ax.set_ylabel('重要性')
1086     ax.set_title('最终模型 - 特征重要性条形图')
1087     ax.set_xticks(x_pos)
1088     ax.set_xticklabels(importance_df['feature'].head(15), rotation=45, ha='right')
1089     ax.grid(True, alpha=0.3)
1090
1091     # 在柱子上添加数值
1092     for bar, importance in zip(bars, importance_df['importance'].head(15)):
1093         height = bar.get_height()
1094         ax.text(bar.get_x() + bar.get_width()/2., height + 0.001,
1095                 f'{importance:.3f}', ha='center', va='bottom', fontsize=9)
1096
1097     plt.tight_layout()
1098     plt.savefig('final_feature_importance_bar.png')
1099     plt.close()
1100
1101     print("\n=== 分析完成 ===")
1102
1103 if __name__ == "__main__":
1104     main()

```